

1 Vorspann: Sprachen

Def. 1.1: Ein *Alphabet* ist eine nicht leere, endliche Menge von *Zeichen*.

Vorlesung:
21.04.26

Zeichen sind hier beliebige abstrakte Symbole.

Bsp.: für Alphabete, die in dieser Vorlesung, im täglichen Umgang mit Computern oder in der Forschung an unserem Lehrstuhl eine Rolle spielen

- $\{a, \dots, z\}$
- $\{0, 1\}$
- $\{\text{rot}, \text{gelb}, \text{grün}\}$ (Ampelfarben)
- Die Menge aller ASCII-Symbole
- Die Menge aller Statements eines Computerprogramms

Wir verwenden typischerweise den griechischen Buchstaben Σ als Namen für ein Alphabet und die lateinischen Buchstaben a, b, c als Namen für Zeichen. Im Folgenden sei Σ immer ein beliebiges Alphabet.

Def. 1.2: Ein *Wort* ist eine endliche Folge von Elementen aus Σ . Die leere Folge nennen wir das *leere Wort* geschrieben als ε . Wir bezeichnen die Menge aller Wörter mit Σ^* und die Menge aller nicht leeren Wörter mit $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$.

Hier die induktive Definition.

1. $\varepsilon \in \Sigma^*$.
2. $a \in \Sigma$ und $w \in \Sigma^*$ impliziert $a.w \in \Sigma^*$.

◇

Hier einige Konventionen:

- Wir schreiben ein Wort immer ohne Trennsymbole wie z.B. Komma. Also **einhorn** statt **e,i,n,h,o,r,n**.
- Wir verwenden ε für das leere Wort.
- Wir verwenden typischerweise u, v, w als Namen für Wörter.

Bsp.: für Wörter über $\Sigma = \{a, \dots, z\}$

- rambo (Länge 5)
- eis, ies (beide Länge 3 aber ungleich)
- ε (Länge 0)

Aus der induktiven Definition ergibt sich das folgende Beweisprinzip für Aussagen $\varphi(w)$ über Worte w :

$$\frac{\varphi(\varepsilon) \quad \forall a, w. \varphi(w) \Rightarrow \varphi(a.w)}{\forall w \in \Sigma^*. \varphi(w)}$$

Dieses Prinzip können wir auch verwenden um Funktionen auf Worten zu definieren. Als erstes Beispiel definieren wir die Länge eines Wortes, d.h. die Anzahl der Symbole des Wortes.

Def. 1.3 (Länge eines Worts): Die *Länge* eines Worts, $|_| : \Sigma^* \rightarrow \mathbb{N}$, wird für alle $w \in \Sigma^*$ induktiv definiert durch

1. $|\varepsilon| = 0$;
2. für alle $a \in \Sigma$ und $w \in \Sigma^*$: $|a.w| = 1 + |w|$.

◇

Wörter lassen sich „verketten“/„aneinanderhängen“. Die entsprechende Operation heißt *Konkatenation*, geschrieben „ $\cdot$ “ (wie Multiplikation).

Def. 1.4 (Konkatenation von Wörtern): Die *Konkatenation*, $\cdot : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, wird für alle $v \in \Sigma^*$ induktiv definiert durch

1. $\varepsilon \cdot v = v$;
2. für alle $a \in \Sigma$ und $u \in \Sigma^*$: $(a.u) \cdot v = a.(u \cdot v)$.

◇

Lemma 1.1: Eigenschaften der Konkatenation: $(\Sigma^*, \varepsilon, \cdot)$ ist ein Monoid:

1. $\varepsilon \cdot v = v$ (leeres Wort ist linksneutrales Element);
2. $v \cdot \varepsilon = v$ (leeres Wort ist rechtsneutrales Element);
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ (Assoziativität).

BEWEIS: 1. $\varepsilon \cdot v = v$ (Definition)

2. $v \cdot \varepsilon = v$ Induktion über v :

- IA: $(v = \varepsilon) \ \varepsilon \cdot \varepsilon = \varepsilon$ (Definition)
- IS: $(v \rightsquigarrow a.v) \ (a.v) \cdot \varepsilon = a.(v \cdot \varepsilon) \stackrel{IV}{=} a.v$

3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ Induktion über u :

- IA: $(u = \varepsilon) \ \varepsilon \cdot (v \cdot w) = (v \cdot w) = (\varepsilon \cdot v) \cdot w$ (Definition)
- IS: $(u \rightsquigarrow a.u) \ (a.u) \cdot (v \cdot w) \stackrel{Def}{=} a.(u \cdot (v \cdot w)) \stackrel{IV}{=} a.((u \cdot v) \cdot w) \stackrel{Def}{=} ((a.u) \cdot v) \cdot w$

□

Bsp.:

- $\mathbf{eis} \cdot \mathbf{rambo} = \mathbf{eisrambo}$
- $\mathbf{rambo} \cdot \varepsilon = \mathbf{rambo} = \varepsilon \cdot \mathbf{rambo}$

Der Konkatenationsoperator „ $\cdot$ “ wird oft weggelassen (ähnlich wie der Multiplikationsoperator in der Arithmetik). Ebenso können durch die Assoziativität Klammern weggelassen werden.

$w_1 w_2 w_3$ steht also auch für $w_1 \cdot w_2 \cdot w_3$, für $(w_1 \cdot w_2) \cdot w_3$ und für $w_1 \cdot (w_2 \cdot w_3)$

Bemerkung: Die Zeichenfolge $\mathbf{rambo}\varepsilon$ ist *kein* Wort. Diese Zeichenfolge ist lediglich eine Notation für eine Konkatenationsoperation, die ein Wort der Länge 5 (nämlich $\mathbf{rambo}$) beschreibt.

Wörter lassen sich außerdem *potenzieren*:

Def. 1.5: Die *Potenzierung* von Wörtern, $\cdot^* : \Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, ist induktiv definiert durch

1. $w^0 = \varepsilon$
2. $w^{n+1} = w \cdot w^n$ ◇

Bsp.: $\mathbf{eis}^3 \stackrel{(2.)}{=} \mathbf{eis} \cdot \mathbf{eis}^2 \stackrel{\text{zweimal } (2.)}{=} \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis}^0 \stackrel{(1.)}{=} \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis} \cdot \varepsilon = \mathbf{eiseiseis}$

Def. 1.6: Eine *Sprache* über Σ ist eine Menge $L \subseteq \Sigma^*$. Gleichwertig: $L \in \mathcal{P}(\Sigma^*)$. ◇

Bsp.:

- $\{\text{eis}, \text{rambo}\}$
- $\{w \in \{0, 1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$
- $\{\}$ (die „leere Sprache“)
- $\{\varepsilon\}$ (ist verschieden von der leeren Sprache)
- Σ^*

Sämtliche Mengenoperationen sind auch Sprachoperationen, insbesondere Schnitt ($L_1 \cap L_2$), Vereinigung ($L_1 \cup L_2$), Differenz ($L_1 \setminus L_2$) und Komplement ($\overline{L_1} = \Sigma^* \setminus L_1$).

Weitere Operationen auf Sprachen sind Konkatenation und Potenzierung, sowie der *Kleene-Abschluss*.

Def. 1.7 (Konkatenation und Potenzierung von Sprachen): Seien $U, V \subseteq \Sigma^*$. Dann ist die *Konkatenation* von U und V definiert durch

$$U \cdot V = \{uv \mid u \in U, v \in V\}$$

und die *Potenzierung* von U induktiv definiert durch

1. $U^0 = \{\varepsilon\}$
2. $U^{n+1} = U \cdot U^n$

◇

Bsp.:

- $\{\text{eis}, \varepsilon\} \cdot \{\text{rambo}\} = \{\text{eisrambo}, \text{rambo}\}$
- $\{\text{eis}, \varepsilon\} \cdot \{\} = \{\}$
- $\{\}^0 = \{\varepsilon\}$
- $\{\}^4 = \{\}$
- $\{\text{eis}, \varepsilon\}^2 = \{\varepsilon, \text{eis}, \text{eiseis}\}$

Wie bei der Konkatenation von Wörtern dürfen wir den Konkatenationsoperator auch weglassen.

Def. 1.8 (Kleene-Abschluss, Kleene-Stern, Kleene-Plus): Gegeben eine Sprache $U \subseteq \Sigma^*$, definieren wir den *Kleene-Abschluss* U^* wie folgt.

$$U^* = \bigcup_{n \in \mathbb{N}} U^n$$

Wie nennen den Operator $\cdot^* : \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, der für eine gegebene Sprache den Kleene-Abschluss liefert, den *Kleene-Stern*. Analog definieren wir den *Kleene-Plus* Operator $\cdot^+ : \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$ als

$$U^+ = \bigcup_{n \geq 1} U^n.$$

◇

Es gilt also $U^* = U^+ \cup \{\varepsilon\}$.

Zum Abschluss eine alternative Definition, die in manchen Textbüchern oder manchmal in der Literatur verwendet wird. In diesem Skript werden wir sie nicht weiter verwenden.

Def. 1.9 (Alternative Definition für Σ^*): Sei $n \in \mathbb{N}$.

- $\Sigma^n = \{1, \dots, n\} \rightarrow \Sigma$ Worte der Länge n
- insbesondere $\Sigma^0 = \{\}$ $\rightarrow \Sigma$ enthält nur ein Element, ε
- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$
- Konkatination: $u \in \Sigma^n, v \in \Sigma^m \Rightarrow u \cdot v \in \Sigma^{n+m}$ definiert durch

$$(u \cdot v)(i) = \begin{cases} u(i) & 1 \leq i \leq n \\ v(i - n) & n < i \leq n + m \end{cases}$$

◇

2 Reguläre Sprachen und endliche Automaten

Vorlesung:
22.04.26

Wie können wir potentiell unendlich große Mengen von Wörtern darstellen? Eine Lösung für dieses Problem sahen wir bereits im vorherigen Kapitel, als wir die (unendlich große) Menge der binär codierten Primzahlen mit Hilfe der folgenden Sprache darstellten.

$$L_{\text{prim}} = \{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$$

Ein weiteres Beispiel ist die folgende Sprache.

$$L_{\text{even}} = \{w \in \{0,1\}^* \mid \text{die Anzahl der Einsen in } w \text{ ist gerade.}\}$$

Eine häufig interessante Fragestellung für ein gegebenes Wort w und eine Sprache L ist: „Ist w in L enthalten?“ (Also: „Gilt $w \in L$?“) Wir nennen dieses Entscheidungsproblem das *Wortproblem*. Eine konkrete Instanz des Wortproblems wäre z.B. „ $1100101 \in L_{\text{prim}}$?“ oder „ $1100101 \in L_{\text{even}}$?“

Die obige Darstellung der unendlichen Mengen L_{prim} und L_{even} ist zwar sehr kompakt, wir können daraus aber nicht direkt ein Vorgehen zur Lösung des Wortproblems ableiten. Wir müssen zunächst verstehen, was die Begriffe „Binärcodierung“, „Primzahl“ oder „gerade Anzahl“ bedeuten und für L_{prim} und L_{even} jeweils einen Algorithmus zur Entscheidung entwickeln.

In diesem Kapitel werden wir mit *endlichen Automaten* einen weiteren Formalismus kennenlernen, um (potentiell unendlich große) Mengen von Wörtern darzustellen. Ein Vorteil dieser Darstellung ist, dass es einen einheitlichen und effizienten Algorithmus für das Wortproblem gibt. Wir werden aber auch sehen, dass sich nicht jede Sprache (z.B. L_{prim}) mit Hilfe eines endlichen Automaten darstellen lässt.

2.1 Endliche Automaten

Wir beschreiben zunächst informell die Bestandteile eines endlichen Automaten:

Endliches Band (read-only; jede Zelle enthält ein $a_i \in \Sigma$; der Inhalt des Bands ist das *Eingabewort* bzw. die *Eingabe*)

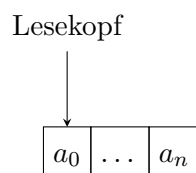


Abb. 1: Endliches Band

Lesekopf

- Der *Lesekopf* zeigt auf ein Feld des Bands, oder hinter das letzte Feld.
- Er bewegt sich feldweise nach rechts; andere Bewegungen (Vor- bzw. Zurückspulen) sind nicht möglich.
- Wenn er hinter das letzte Zeichen zeigt, *stoppt* der Automat. Er muss sich nun „entscheiden“ ob er das Wort *akzeptiert* oder nicht.

Zustände Zu jedem Zeitpunkt befindet sich der Automat in einem Zustand q aus einer *endlichen* Zustandsmenge Q .

Startzustand $q^{\text{init}} \in Q$

Akzeptierende Zustände $F \subseteq Q$

Transitionsfunktion Im Zustand q beim Lesen von a gehe in Zustand $q' := \delta(q, a)$.

Der endliche Automat akzeptiert eine Eingabe, falls er in einem akzeptierenden Zustand stoppt.

Bsp. 2.1: Aufgabe:

„Erkenne alle Stapel von Macarons, in denen höchstens ein grünes Macaron vorkommt.“



Ein passendes Alphabet wäre $\Sigma = \{\text{grün}, \text{nicht-grün}\}$. Wir definieren die folgenden Zustände. (Die Metapher hier ist: „wenn ich mehr als ein grünes Macaron esse, wird mir übel, und das wäre nicht akzeptabel“.)

Zustand	Bedeutung
q_0	„alles gut“
q_1	„mir wird schon flau“
q_2	„mir ist übel“

Der Startzustand ist q_0 . Akzeptierende Zustände sind q_0 und q_1 . Die Transitionsfunktion δ ist durch die folgende Tabelle gegeben.

¹Von links nach rechts:

By Mariajudit - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=48726001>
 By Michelle Naherny - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=44361114>
 By Keven Law - originally posted to Flickr as What's your Colour???, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=6851868>

	grün	nicht-grün	
q_0	q_1	q_0	wechsle nach q_1 falls grün , ansonsten verweile
q_1	q_2	q_1	wechsle nach q_2 falls grün , ansonsten verweile
q_2	q_2	q_2	verweile, da es nichts mehr zu retten gibt

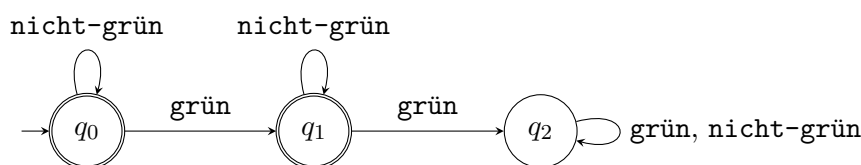
Def. 2.1 (DEA): Ein *deterministischer endlicher Automat* (DEA)² ist ein 5-Tupel

$$\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \rightarrow Q$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\text{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende Zustände* nennen. $\diamond$

DEAs lassen sich auch graphisch darstellen. Dabei gibt man für den Automaten einen gerichteten Graphen an. Die Knoten des Graphen sind die Zustände, und mit Zeichen beschriftete Kanten zeigen, welchen Zustandsübergang die Transitionsfunktion für das nächste Zeichen erlaubt. Der Startzustand ist mit einem unbeschrifteten Pfeil markiert, akzeptierende Zustände sind doppelt eingekreist. Hier ist die graphische Darstellung von A_{Macaron} aus Beispiel 2.1:



Die folgenden beiden Definitionen erlauben uns mit Hilfe eines DEA eine Sprache zu charakterisieren. Hierbei sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein DEA.

Def. 2.2 (Induktive Erweiterung von δ auf Wörter): Die *induktive Erweiterung* von $\delta : Q \times \Sigma \rightarrow Q$ auf Wörter, $\tilde{\delta} : Q \times \Sigma^* \rightarrow Q$, ist (induktiv) definiert durch

1. $\tilde{\delta}(q, \varepsilon) = q$ (Wortende erreicht)

²engl. DFA $\triangleq$ deterministic finite automaton

$$2. \tilde{\delta}(q, aw) = \tilde{\delta}(\delta(q, a), w) \quad (\text{Rest im Folgezustand verarbeiten}) \quad \diamond$$

Def. 2.3 (Durch einen DEA akzeptierte Sprache): Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$. Ein Wort $w \in \Sigma^*$ wird von $\mathcal{A}$ *akzeptiert*, falls $\tilde{\delta}(q^{\text{init}}, w) \in F$. Die *von $\mathcal{A}$ akzeptierte Sprache*, geschrieben $L(\mathcal{A})$, ist die Menge aller Wörter, die von $\mathcal{A}$ akzeptiert werden. D.h.,

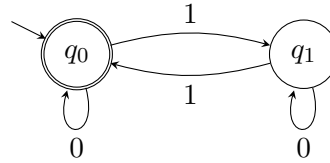
$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \tilde{\delta}(q^{\text{init}}, w) \in F\}.$$

Eine durch einen DEA akzeptierte Sprache heißt *regulär*. $\diamond$

Bsp. 2.2: Ein möglicher DEA für die Sprache

$$L_{\text{even}} = \{w \in \{0, 1\}^* \mid \text{Die Anzahl der Einsen in } w \text{ ist gerade.}\}$$

aus der Einleitung dieses Kapitels hat die folgende graphische Repräsentation.



Frage: Angenommen wir haben zwei DEAs A_1 und A_2 , können wir dann immer einen DEA konstruieren der den Durchschnitt $L(A_1) \cap L(A_2)$ akzeptiert?

Idee: Lasse beide DEAs parallel laufen, akzeptiere nur wenn beide DEAs akzeptieren.

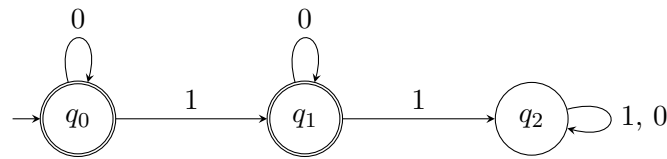
Frage: Lassen sich zwei *parallel laufende* DEAs in einem einzigen DEA implementieren?

Die nächste Konstruktion und der drauf folgende Satz liefern eine positive Antwort auf diese Frage.

Def. 2.4 (Produktautomat für Durchschnitt): Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\text{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\text{init}}, F_2)$ DEAs. Wir definieren den *Produktautomaten für Durchschnitt* als den DEA $A_{\cap} = (\Sigma, Q_{\cap}, \delta_{\cap}, q_{\cap}^{\text{init}}, F_{\cap})$ mit den folgenden Komponenten.

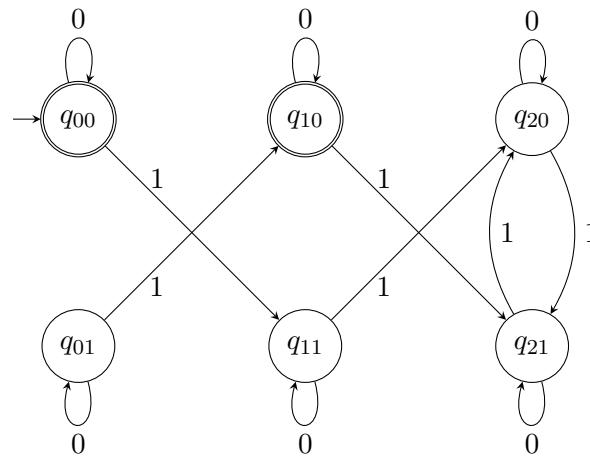
$$\begin{aligned} Q_{\cap} &= Q_1 \times Q_2 \\ \delta_{\cap}((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, a)) \quad , \quad \text{für alle } a \in \Sigma \\ q_{\cap}^{\text{init}} &= (q_1^{\text{init}}, q_2^{\text{init}}) \\ F_{\cap} &= F_1 \times F_2 \end{aligned} \quad \diamond$$

Im folgenden Beispiel sei A_1 der DEA über dem Alphabet $\Sigma = \{0, 1\}$, dessen graphische Repräsentation nahezu mit A_{Macaron} identisch ist.



Bsp. 2.3: Der Produktautomat für den Durchschnitt von A_1 und A_{even} hat die folgende graphische Repräsentation, wobei wir um Platz zu sparen „ q_{ij} “ statt „ (q_i, q_j) “ schreiben.

Vorlesung:
28.04.26



Satz 2.1: Für beliebige DEAs A_1 und A_2 akzeptiert der entsprechende Produktautomat für Durchschnitt die Sprache $L(A_1) \cap L(A_2)$.

BEWEIS (Beschreibung des Agda-Beweises von `Intersection.correct`): Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\text{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\text{init}}, F_2)$ zwei DEAs. Der Produktautomat $A_\cap$ hat Zustandsmenge $Q_1 \times Q_2$, Startzustand $(q_1^{\text{init}}, q_2^{\text{init}})$, Übergangsfunktion

$$\delta_\cap((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$$

und Endzustände $F_\cap = F_1 \times F_2$.

Die zentrale Aussage des Agda-Beweises ist `Intersection.left-right`. Sie zeigt für alle $q_1 \in Q_1$, $q_2 \in Q_2$ und alle Wörter $w \in \Sigma^*$ die Äquivalenz

$$\tilde{\delta}_\cap((q_1, q_2), w) \in F_\cap \iff \tilde{\delta}_1(q_1, w) \in F_1 \text{ und } \tilde{\delta}_2(q_2, w) \in F_2.$$

Wir führen eine Induktion über w durch.

I.A. ($w = \varepsilon$): Nach Definition von $\tilde{\delta}$ gilt

$$\tilde{\delta}_{\cap}((q_1, q_2), \varepsilon) = (q_1, q_2).$$

Da $F_{\cap} = F_1 \times F_2$ ist, gilt

$$(q_1, q_2) \in F_{\cap} \iff q_1 \in F_1 \text{ und } q_2 \in F_2.$$

Das ist genau die gewünschte Aussage für ε . Im Agda-Beweis sind beide Richtungen dieser Äquivalenz deshalb die Identitätsfunktion.

I.S. ($w \rightsquigarrow aw$): Sei $a \in \Sigma$ und $w \in \Sigma^*$. Dann gilt:

$$\begin{array}{ll} \tilde{\delta}_{\cap}((q_1, q_2), aw) \in F_{\cap} \\ \text{Def. } \tilde{\delta} \quad \text{gdw} \quad \tilde{\delta}_{\cap}(\delta_{\cap}((q_1, q_2), a), w) \in F_{\cap} \\ \text{Def. } \delta_{\cap} \quad \text{gdw} \quad \tilde{\delta}_{\cap}((\delta_1(q_1, a), \delta_2(q_2, a)), w) \in F_{\cap} \\ \text{I.V.} \quad \text{gdw} \quad \tilde{\delta}_1(\delta_1(q_1, a), w) \in F_1 \text{ und } \tilde{\delta}_2(\delta_2(q_2, a), w) \in F_2 \\ \text{Def. } \tilde{\delta} \quad \text{gdw} \quad \tilde{\delta}_1(q_1, aw) \in F_1 \text{ und } \tilde{\delta}_2(q_2, aw) \in F_2. \end{array}$$

Damit ist `Intersection.left-right` bewiesen.

Für `Intersection.correct` wird diese Aussage nur noch auf die Startzustände angewendet. Für ein beliebiges Wort $w \in \Sigma^*$ erhalten wir:

$$\begin{array}{ll} w \in L(A_{\cap}) & \begin{array}{ll} \text{Def. Sprache} & \tilde{\delta}_{\cap}((q_1^{\text{init}}, q_2^{\text{init}}), w) \in F_{\cap} \\ \text{left-right} & \text{gdw} \quad \tilde{\delta}_1(q_1^{\text{init}}, w) \in F_1 \text{ und } \tilde{\delta}_2(q_2^{\text{init}}, w) \in F_2 \\ \text{Def. Sprache} & \text{gdw} \quad w \in L(A_1) \text{ und } w \in L(A_2) \\ \text{Def. } \cap & \text{gdw} \quad w \in L(A_1) \cap L(A_2). \end{array} \end{array}$$

Also gilt $L(A_{\cap}) = L(A_1) \cap L(A_2)$. In Agda wird die punktweise Äquivalenz durch die Hilfsfunktion für die Verteilung von $\forall$ über $\times$ in die beiden Inklusionen der Sprachgleichheit umgepackt. $\square$

2.2 Minimierung endlicher Automaten

Beobachtung: Der Zustand q_{01} im Beispiel 2.3 scheint nutzlos. Wir charakterisieren die Nützlichkeit eines Zustands in einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$ formal wie folgt.

Def. 2.5: Ein Zustand $q \in Q$ heißt *erreichbar*, falls ein $w \in \Sigma^*$ existiert, sodass $\tilde{\delta}(q^{\text{init}}, w) = q$. $\diamond$

Bemerkung: Die Menge der erreichbaren Zustände kann mit dem folgenden Verfahren in $O(|Q| \cdot |\Sigma|)$ berechnet werden.

- Fasse $\mathcal{A}$ als Graph auf.
- Wende Tiefensuche ab q^{init} an und markiere dabei alle besuchten Zustände.
- Genau die markierten Zustände sind erreichbar.

Beobachtung: Auch nach dem Entfernen der nicht erreichbaren Zustände q_{01} und q_{10} scheint der DEA aus Bsp. 2.3 unnötig groß zu sein. Das Verhalten des DEA in den Zuständen q_{11} , q_{20} und q_{21} ist sehr ähnlich. Wir charakterisieren diese „Ähnlichkeit“ formal wie folgt.

Def. 2.6: Zwei Zustände $p, q \in Q$ eines DEA sind *äquivalent*, geschrieben $p \equiv q$, falls

$$\forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \text{ gdw } \tilde{\delta}(q, w) \in F \quad \diamond$$

Bsp. 2.4: Für Bsp. 2.3 gilt: Die Zustände q_{11} , q_{20} und q_{21} sind paarweise äquivalent. Die Zustände q_{00} und q_{10} sind äquivalent. Keine weiteren Zustandspaare sind äquivalent.

Geschrieben als Menge von Paaren sieht die Relation $\equiv \subseteq Q \times Q$ also wie folgt aus:

$$\{(q_{00}, q_{10}), (q_{10}, q_{00}), (q_{11}, q_{20}), (q_{20}, q_{11}), (q_{20}, q_{21}), (q_{21}, q_{20}), (q_{21}, q_{11}), (q_{11}, q_{21})\}$$

Lemma 2.2: Die Relation „ $\equiv$ “ ist eine *Äquivalenzrelation*.

BEWEIS: Zu zeigen ist, dass $\equiv$ reflexiv, transitiv und symmetrisch ist.

- Die Relation $\equiv$ ist offensichtlich reflexiv.
- Die Symmetrie und Transitivität von $\equiv$ folgen aus der Symmetrie und Transitivität der logischen Interpretation von „genau dann, wenn“ (gdw).

□

Bsp. 2.5: Für den DEA aus Bsp. 2.3 hat die Relation $\equiv$ drei Äquivalenzklassen.³

$$\begin{aligned} [q_{00}] &= \{q_{00}, q_{10}\}, \\ [q_{01}] &= \{q_{01}\}, \\ [q_{11}] &= \{q_{11}, q_{20}, q_{21}\} \end{aligned}$$

³Den Zustand q_{11} als Repräsentanten für die dritte Äquivalenzklasse zu wählen ist eine völlig willkürliche Entscheidung. Wir könnten genauso gut q_{20} oder q_{21} wählen.

Idee: „Verschmelze“ alle Zustände aus einer Äquivalenzklasse zu einem einzigen Zustand. Bedenken: Bei einem DEA hat jeder Zustand hat für jedes Zeichen einen Nachfolger. Wenn wir Zustände verschmelzen, könnte es mehrere Nachfolger geben und das Resultat wäre kein wohldefinierter DEA mehr.

Das folgende Lemma zeigt, dass unsere Bedenken nicht gerechtfertigt sind. Sind zwei Zustände äquivalent, so sind auch für jedes Zeichen ihre Nachfolger äquivalent.

Lemma 2.3: Für alle $p, q \in Q$ gilt:

$$p \equiv q \Rightarrow \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a)$$

BEWEIS:

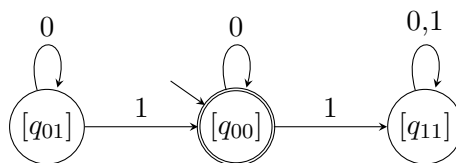
$$\begin{aligned} p \equiv q & \quad \text{gdw} \quad \forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \Leftrightarrow \tilde{\delta}(q, w) \in F \\ & \quad \text{gdw} \quad (p \in F \Leftrightarrow q \in F) \wedge \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(p, aw) \in F \Leftrightarrow \tilde{\delta}(q, aw) \in F \\ & \quad \text{impliziert} \quad \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(\delta(p, a), w) \in F \Leftrightarrow \tilde{\delta}(\delta(q, a), w) \in F \\ & \quad \text{gdw} \quad \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a) \quad \square \end{aligned}$$

Wir formalisieren das „Verschmelzen“ von Zuständen wie folgt.

Def. 2.7: Der Äquivalenzklassenautomat $\mathcal{A}_{\equiv} = (Q_{\equiv}, \Sigma, \delta_{\equiv}, q_{\equiv}^{\text{init}}, F_{\equiv})$ zu einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$ ist bestimmt durch:

$$\begin{aligned} Q_{\equiv} &= \{[q] \mid q \in Q\} \\ \delta_{\equiv}([q], a) &= [\delta(q, a)] \quad \text{wobei } q \in q_{\equiv} \text{ beliebig gewählt (also } q_{\equiv} = [q]) \\ q_{\equiv}^{\text{init}} &= [q^{\text{init}}] \\ F_{\equiv} &= \{[q] \mid q \in F\} \quad \diamond \end{aligned}$$

Bsp. 2.6: Der Äquivalenzklassenautomat $\mathcal{A}_{\equiv}$ zum DEA aus Bsp. 2.3 hat das folgende Zustandsdiagramm.



Vorlesung:
29.04.26

Satz 2.4: Der Äquivalenzklassenautomat ist wohldefiniert und $L(\mathcal{A}_{\equiv}) = L(\mathcal{A})$.

BEWEIS:

1. Wohldefiniert: Es gilt zu zeigen, dass $\delta_{\equiv}([q], a) = [\delta(q, a)]$ nicht abhängig von der Wahl des Repräsentanten $q \in [q]$ ist. Dies folgt direkt aus Lemma 2.3.
2. $L(\mathcal{A}) = L(\mathcal{A}_{\equiv})$: Zunächst zeigen wir per Induktion über w , dass für alle $w \in \Sigma^*$ und alle $q \in Q$ die folgende Äquivalenz gilt.

$$\varphi(w) := \tilde{\delta}(q, w) \in F \Leftrightarrow \tilde{\delta}_{\equiv}([q], w) \in F_{\equiv}$$

$$\text{I.A. } (\varphi(\varepsilon)): \tilde{\delta}(q, \varepsilon) = q \in F \Leftrightarrow \tilde{\delta}_{\equiv}([q], \varepsilon) = [q] \in F_{\equiv}$$

$$\text{I.S. } (\forall a, w : \varphi(w) \Rightarrow \varphi(aw)):$$

$$\begin{aligned} \tilde{\delta}(q, aw) \in F &\Leftrightarrow \tilde{\delta}(\delta(q, a), w) \in F \\ &\xLeftrightarrow{\text{I.V.}} \tilde{\delta}_{\equiv}([\delta(q, a)], w) \in F_{\equiv} \\ &\Leftrightarrow \tilde{\delta}_{\equiv}(\delta_{\equiv}([q], a), w) \in F_{\equiv} \\ &\Leftrightarrow \tilde{\delta}_{\equiv}([q], aw) \in F_{\equiv} \end{aligned}$$

Damit zeigen wir nun $L(\mathcal{A}) = L(\mathcal{A}_{\equiv})$. Sei $w \in \Sigma^*$.

$$\begin{aligned} w \in L(\mathcal{A}) &\Leftrightarrow \tilde{\delta}(q^{\text{init}}, w) \in F \\ &\Leftrightarrow \tilde{\delta}_{\equiv}([q^{\text{init}}], w) \in F_{\equiv} \\ &\Leftrightarrow w \in L(\mathcal{A}_{\equiv}) \end{aligned}$$

□

In den Übungen werden wir ein Verfahren mit Laufzeit $O(|Q|^4 \cdot |\Sigma|)$ zur Konstruktion des Äquivalenzklassenautomaten kennenlernen. Es gibt aber auch schnellere Verfahren. Mit dem Algorithmus von Hopcroft kann $\mathcal{A}_{\equiv}$ in $O(|Q||\Sigma| \log |Q|)$ erzeugt werden.

Wir werden später ($\rightarrow$ Satz von Myhill-Nerode) sehen, dass $\mathcal{A}_{\equiv}$ ein DEA ist, der $L(\mathcal{A})$ akzeptiert, sodass kein DEA, der ebenfalls $L(\mathcal{A})$ akzeptiert, weniger Zustände haben kann.

Def. 2.8: Eine Äquivalenzrelation $R \subseteq \Sigma^* \times \Sigma^*$ heißt *rechtskongruent*, falls

$$(u, v) \in R \quad \Rightarrow \quad \forall w \in \Sigma^* : (u \cdot w, v \cdot w) \in R. \quad \diamond$$

Motivation: Ist eigentlich jede Sprache regulär? Falls nein, wie können wir nicht-reguläre Sprachen finden? Wie können wir beweisen dass eine Sprache nicht regulär ist? In weiteren Verlauf dieses Unterkapitels wollen wir charakteristische Merkmale von reguläre Sprachen finden. Die Idee dazu basiert auf den folgenden Überlegungen.

Angenommen, wir haben für eine gegebene Sprache L einen ersten Entwurf für einen DEA $\mathcal{A}$ konstruiert. In diesem führen sowohl das Wort $u \in \Sigma^*$ als auch das Wort $v \in \Sigma^*$ in den Zustand q_{42} (formal: $\tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v) = q_{42}$).

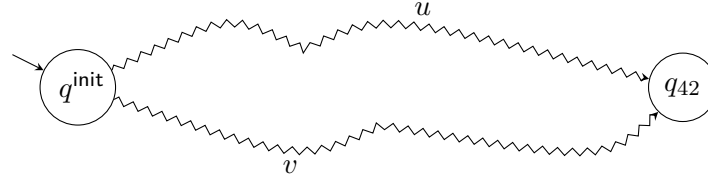


Abb. 2: Schematische Darstellung eines DEA, mit $\tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v) = q_{42}$

- Angenommen, es gibt ein Wort $w \in \Sigma^*$, dass die Äquivalenz $uw \in L \Leftrightarrow vw \in L$ verletzt. Dann muss der Entwurf noch einen Fehler enthalten und wir müssen dafür sorgen, dass q^{init} nicht sowohl von u als auch von v in q_{42} überführt wird.
- Angenommen, es gilt für alle Wörter $w \in \Sigma^*$, dass $uw \in L \Leftrightarrow vw \in L$. Dann wissen wir, dass es eine gute Idee war, q^{init} mit u und v in den gleichen Zustand zu führen.

Angenommen, wir finden nicht nur für zwei Wörter u, v ein Wort w , sodass die Äquivalenz $uw \in L \Leftrightarrow vw \in L$ nicht gilt, sondern wir finden solch ein w für jedes Paar aus einer unendlich großen Menge von Wörtern U .

$$(\text{Formal: } \forall u, v \in U : u \neq v \Rightarrow \exists w^{uv} \in \Sigma^* : \neg(uw^{uv} \in L \Leftrightarrow vw^{uv} \in L))$$

Dann wissen wir, dass all unsere Reparaturversuche hoffnungslos sind, denn der DEA $\mathcal{A}$ bräuchte für jedes $u \in U$ einen eigenen Zustand. Doch ein DEA darf nur endlich viele Zustände haben. Die Sprache L kann also nicht regulär sein.

Im weiteren Verlauf des Unterkapitels werden wir genau diese Überlegungen formal ausarbeiten.

Def. 2.9: Für einen DEA $\mathcal{A}$ definiere

$$R_{\mathcal{A}} = \{(u, v) \mid \tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v)\}.$$

Beobachtung 1 $R_{\mathcal{A}}$ ist eine Äquivalenzrelation.

Dies folgt daraus, dass „ $=$ “ eine Äquivalenzrelation ist.

Beobachtung 2 $R_{\mathcal{A}}$ ist rechtskongruent.

Dies wird in den Übungen bewiesen.

Beobachtung 3 Wir haben pro Zustand, der von q^{init} erreichbar ist, genau eine Äquivalenzklasse. Der Index von $R_{\mathcal{A}}$ ist also die Anzahl der erreichbaren Zustände.

Für den DEA aus Bsp. 2.3 hat $R_{\mathcal{A}}$ die folgenden Äquivalenzklassen.

$$\begin{aligned} [\varepsilon] &= \{0^n \mid n \in \mathbb{N}\} \\ [1] &= \{0^n 10^m \mid n, m \in \mathbb{N}\} \\ [11] &= \{w \mid \text{Anzahl von Einsen in } w \text{ ist gerade und } \geq 2\} \\ [111] &= \{w \mid \text{Anzahl von Einsen in } w \text{ ist ungerade und } \geq 2\} \end{aligned} \quad \diamond$$

Def. 2.10: Für eine Sprache $L \subseteq \Sigma^*$ ist die *Nerode-Relation* wie folgt definiert.

$$R_L = \{(u, v) \mid \forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L\} \quad \diamond$$

Lemma 2.5: Die Nerode-Relation ist eine Äquivalenzrelation.

Dies folgt daraus, dass „ $\Leftrightarrow$ “ eine Äquivalenzrelation ist.

Lemma 2.6: Die Nerode-Relation ist rechtskongruent.

Dafür starten wir mit folgender Beobachtung.

Lemma 2.7: $\forall u, v \in \Sigma^*. \forall a \in \Sigma. (u, v) \in R_L \Rightarrow (ua, va) \in R_L$

BEWEIS:

$$\begin{aligned} &(u, v) \in R_L \\ \Leftrightarrow &\forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L \\ &\text{wähle alle } w \text{ der Form } w = aw' \\ \Rightarrow &\forall a \in \Sigma : \forall w' \in \Sigma^* : uaw' \in L \Leftrightarrow vaw' \in L \\ \Leftrightarrow &(ua, va) \in R_L \end{aligned}$$

D.h. $(u, v) \in R_L \Rightarrow \forall a \in \Sigma : (ua, va) \in R_L$. □

BEWEIS (Lemma 2.6): Wir wählen die folgende Induktionsbehauptung

$$\forall w \in \Sigma^* : \forall (u, v) \in R_L : (uw, vw) \in R_L$$

und führen eine Induktion über w durch.

I.A. ($w = \varepsilon$): $(u\varepsilon, v\varepsilon) = (u, v) \in R_L$

I.S. ($w \rightsquigarrow a.w$): Zeige nun $\forall (u, v) \in R_L : (uaw, vaw) \in R_L$.

Sei $(u, v) \in R_L$, dann ist nach Lemma 2.7 auch $(ua, va) \in R_L$. Daraus liefert die Induktionsbehauptung sofort $(uaw, vaw) \in R_L$. □

Bsp. 2.7: Sei $\Sigma = \{0, 1\}$. Die Sprache $L = \{w \in \Sigma^* \mid \text{vorletztes Zeichen von } w \text{ ist } 1\}$ hat die folgenden Äquivalenzklassen bezüglich der Nerode-Relation.

$$\begin{aligned} [\varepsilon] &= \{w \mid w \text{ endet mit } 00\} \cup \{\varepsilon, 0\} \\ [1] &= \{w \mid w \text{ endet mit } 01\} \cup \{1\} \\ [10] &= \{w \mid w \text{ endet mit } 10\} \\ [11] &= \{w \mid w \text{ endet mit } 11\} \end{aligned}$$

Bsp. 2.8: Für ein beliebiges Alphabet Σ gilt:

1. Die Sprache $L = \{\varepsilon\}$ hat genau zwei Äquivalenzklassen bezüglich der Nerode-Relation. Eine Äquivalenzklasse ist $\{\varepsilon\}$, die andere ist Σ^+ .
2. Die Sprache $L = \{\}$ hat genau eine Äquivalenzklasse (nämlich Σ^*) bezüglich der Nerode-Relation.

Vorlesung:
05.05.2026

Bsp. 2.9: Sei $\Sigma = \{0, 1\}$. Die Sprache $L_{\text{centered}} = \{0^n 1 0^n \mid n \in \mathbb{N}\}$ hat bezüglich der Nerode-Relation die folgende Menge von Äquivalenzklassen:

$$\{[w'] \mid w' \text{ ist Präfix eines Worts } w \in L_{\text{centered}}\} \cup \{[11]\}$$

Bemerkung 1: Die Äquivalenzklasse $[11]$ enthält alle Wörter, die kein Präfix eines Worts aus L_{centered} sind.

Bemerkung 2: Nicht für alle Präfixe von Wörtern in L_{centered} sind die zugehörigen Äquivalenzklassen paarweise verschieden. Es gilt z.B. $[01] = [0010]$.

Bemerkung 3: Für je zwei verschiedene $k \in \mathbb{N}$ sind die Äquivalenzklassen $[0^k 1]$ verschieden. Es gibt also unendlich viele Äquivalenzklassen.

Bsp. 2.10: Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ hat bezüglich der Nerode-Relation die folgende Menge von Äquivalenzklassen:

$$\{[w] \mid \#_a(w) - \#_b(w) = k, k \in \mathbb{Z}\}$$

Satz 2.8 (Myhill und Nerode): Für Sprachen $L \subseteq \Sigma^*$ sind folgende Aussagen äquivalent.

1. L wird von einem DEA akzeptiert.
2. L ist Vereinigung von Äquivalenzklassen einer rechtskongruenten Äquivalenzrelation mit *endlichem* Index.

3. Die Nerode-Relation R_L hat *endlichen* Index.

BEWEIS: Wir beweisen die paarweise Äquivalenz in drei Schritten:

$$(1) \Rightarrow (2), \quad (2) \Rightarrow (3) \quad \text{und} \quad (3) \Rightarrow (1)$$

(1) $\Rightarrow$ (2) Sei $\mathcal{A}$ ein DEA. Es gilt

$$L(\mathcal{A}) = \{w \mid \tilde{\delta}(q^{\text{init}}, w) \in F\} = \bigcup_{q \in F} \{w \mid \tilde{\delta}(q^{\text{init}}, w) = q\}.$$

Nun sind die Mengen $\{w \mid \tilde{\delta}(q^{\text{init}}, w) = q\}$ für erreichbare $q \in Q$ genau die Äquivalenzklassen der Relation $R_{\mathcal{A}}$ aus Def. 2.9, einer rechtskongruenten Äquivalenzrelation. Der Index von $R_{\mathcal{A}}$ ist die Anzahl der *erreichbaren* Zustände und somit endlich: $\text{Index}(R_{\mathcal{A}}) \leq |Q| < \infty$.

(2) $\Rightarrow$ (3) Sei R eine rechtskongruente Äquivalenzrelation mit endlichem Index, sodass L die Vereinigung von R -Äquivalenzklassen ist.

Es genügt zu zeigen, dass die Nerode-Relation R_L eine Obermenge von R ist.⁴

$$\begin{aligned} (u, v) \in R &\Rightarrow u \in L \Leftrightarrow v \in L, \quad \text{da } L \text{ Vereinigung von Äquivalenzklassen ist} \\ &\Rightarrow \forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L, \quad \text{da } R \text{ rechtskongruent} \\ &\Rightarrow (u, v) \in R_L, \quad \text{nach Definition der Nerode-Relation} \end{aligned}$$

Es gilt also $R \subseteq R_L$ und somit $\text{Index}(R_L) \leq \text{Index}(R) < \infty$.

(3) $\Rightarrow$ (1) Gegeben R_L , konstruiere $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$

- $Q = \{[w]_{R_L} \mid w \in \Sigma^*\}$ endlich, da $\text{Index}(R_L)$ endlich
- $\delta(q, a) = [wa]$ für ein $w \in q$ —wohldefiniert (vgl. Lemma 2.7)
- $q^{\text{init}} = [\varepsilon]$
- $F = \{[w] \mid w \in L\}$

⁴Zur Erklärung: Falls $R \subseteq R_L$, dann gilt $\text{Index}(R) \geq \text{Index}(R_L)$. Intuitiv: Je mehr Elemente eine Äquivalenzrelation R enthält, desto mehr Elemente sind bzgl. dieser Relation äquivalent, d.h. desto weniger unterschiedliche Äquivalenzklassen gibt es.

Wir wollen nun $L(\mathcal{A}) = L$ zeigen. Dafür beweisen wir zunächst per Induktion über w die folgende Eigenschaft.

$$\forall w \in \Sigma^* : \forall v \in \Sigma^* : \tilde{\delta}([v], w) = [v \cdot w]$$

I.A. ($w = \varepsilon$): $\tilde{\delta}([v], \varepsilon) = [v] = [v \cdot \varepsilon]$

I.S. ($w \rightsquigarrow a.w$)

$$\begin{aligned} \tilde{\delta}([v], a.w) &= \tilde{\delta}(\delta([v], a), w) \\ &= \tilde{\delta}([v \cdot a], w) \\ &\stackrel{\text{I.V.}}{=} [(v \cdot a) \cdot w] \\ &= [v \cdot a.w] \end{aligned}$$

Nun zeigen wir $L(\mathcal{A}) = L$ wie folgt:

$$\begin{aligned} w \in L(\mathcal{A}) &\text{ gdw } \tilde{\delta}([\varepsilon], w) \in F \\ &\text{ gdw } [w] \in F, \quad (\text{per Induktion gezeigte Eigenschaft für } v = \varepsilon) \\ &\text{ gdw } w \in L \end{aligned} \quad \square$$

Korollar 2.5: Der im Beweisschritt (3) $\Rightarrow$ (1) konstruierte Automat $\mathcal{A}$ ist ein minimaler Automat (bzgl. der Zustandsanzahl) für eine reguläre Sprache L . $\diamond$

BEWEIS: Sei $\mathcal{A}' = (Q', \dots)$ ein *beliebiger* DEA mit $L(\mathcal{A}') = L$.

Aus „1 $\Rightarrow$ 2“ wissen wir, dass $\text{Index}(R_{\mathcal{A}'}) \leq |Q'|$ gilt.

Aus „2 $\Rightarrow$ 3“ wissen wir, dass $R_{\mathcal{A}'} \subseteq R_L$ und somit $\text{Index}(R_L) \leq \text{Index}(R_{\mathcal{A}'})$ gilt.

Aus „3 $\Rightarrow$ 1“ erhalten wir $\mathcal{A} = (Q, \dots)$ mit $|Q| = \text{Index}(R_L)$.

Wegen $|Q| = \text{Index}(R_L) \leq \text{Index}(R_{\mathcal{A}'}) \leq |Q'|$ hat $\mathcal{A}$ nicht mehr Zustände als $\mathcal{A}'$. $\square$

2.3 Nichtdeterministischer endlicher Automat (NEA)

Aufgabe: Konstruiere für eine natürliche Zahl $n \in \mathbb{N}$ einen DEA für die folgende Sprache.

Vorlesung:
12.05.2026

$$L_n = \{w \in \{0, 1\}^* \mid |w| \geq n \text{ und das } n\text{-letzte Symbol von } w \text{ ist } 1\}$$

Naiver Lösungsversuch:

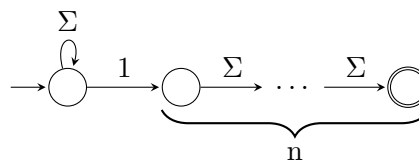


Abb. 3: Nichtdeterministischer Automat für L_n

Problem: Das Zustandsdiagramm beschreibt keinen DEA: Der Startzustand hat zwei ausgehende Kanten für 1.

Untersuche die Sprache mit Hilfe der Nerode-Relation. Beobachtung: Je zwei Wörter der Länge n sind in unterschiedlichen Äquivalenzklassen. Es gibt also mindestens 2^n Äquivalenzklassen; aus Korollar 2.5 wissen wir, dass ein minimaler DEA, der L_n akzeptiert, mindestens 2^n Zustände haben muss.

Idee: Definiere eine neue Art von Automaten, bei dem ein Zustand pro Zeichen mehrere Nachfolger haben darf.

Def. 2.11 (NEA): Ein *nichtdeterministischer endlicher Automat* (NEA)⁵ ist ein 5-Tupel

$$\mathcal{N} = (\Sigma, Q, \delta, q^{\text{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\text{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende Zustände* nennen. $\diamond$

Bemerkung: Die Definition des NEA unterscheidet sich vom DEA nur in der Transitionsfunktion. Beim DEA ist der Bildbereich der Transitionsfunktion die Menge der Zustände Q . Beim NEA ist der Bildbereich die Potenzmenge $\mathcal{P}(Q)$ der Zustandsmenge Q . Analog zu DEAs werden wir auch NEAs mit Hilfe eines Zustandsdiagramms beschrieben. So beschreibt Abb. 3 für jedes $n \in \mathbb{N}$ einen NEA für die Sprache L_n .

Im Folgenden sei $\mathcal{N}$ immer ein NEA.

Def. 2.12 (Lauf eines Automaten): Die Funktion $\text{run} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q^*)$ ist so definiert, dass $\text{run}(q, w)$ die Folgen von Zuständen liefert, die $\mathcal{N}$ vom Zustand q angesetzt auf die Eingabe w durchläuft.

$$\text{run}(q, \varepsilon) = \{q\} \tag{1}$$

$$\text{run}(q, a.w) = \{q.\bar{q} \mid \bar{q} \in \text{run}(q', w), q' \in \delta(q, a)\} \tag{2}$$

Ein Lauf $q_0 \dots q_n \in \text{run}(q_0, w)$ heißt *initial*, falls $q_0 = q^{\text{init}}$. Er heißt *akzeptierend*, falls $q_n \in F$. $\diamond$

⁵engl. NFA $\triangleq$ nondeterministic finite automaton

Def. 2.13 (Akzeptanz): Ein Wort $w \in \Sigma^*$ wird von $\mathcal{N}$ *akzeptiert*, falls $\mathcal{N}$ einen initialen und akzeptierenden Lauf über w hat. Die von $\mathcal{N}$ akzeptierte Sprache ist die Menge der von $\mathcal{N}$ akzeptierten Wörter, d.h.

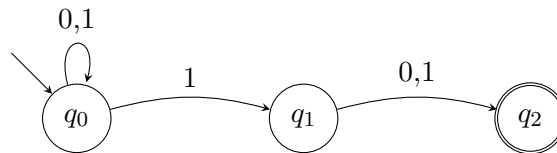
$$L(\mathcal{N}) = \{w \in \Sigma^* \mid \exists \text{ initialer, akzeptierender Lauf von } \mathcal{N} \text{ über } w\}.$$

◇

Bsp. 2.11: Der NEA für die Sprache

$$L_2 = \{w \in \{0, 1\}^* \mid \text{das zweitletzte Zeichen von } w \text{ ist } 1\}$$

hat die folgende graphische Repräsentation.



Bemerkung: Die Frage, ob ein gegebenes Wort w akzeptiert wird (das „Wortproblem“), lässt sich für NEAs nicht mehr so leicht beantworten wie wir es von DEAs gewohnt sind. Ein sinnvolles Vorgehen scheint, jeden initialen Lauf zu betrachten, doch z.B. für das Wort 11 hat obiger NEA bereits drei verschiedene initiale Läufe: $q_0q_0q_0$, $q_0q_0q_1$ und $q_0q_1q_2$.

Bemerkung: Die Definitionen von NEA und DEA in der Literatur sind nicht einheitlich. Es gibt äquivalente NEA-Definitionen, die statt der Transitionsfunktion $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ eine Transitionsrelation $\delta \subseteq Q \times \Sigma \times Q$ verwenden. Es gibt alternative NEA-Definitionen, die eine Menge von Startzuständen erlauben. Alternativ könnte man auch zunächst den NEA einführen und den DEA als Spezialfall dessen definieren (Spezialfall: Das Bild der Transitionsfunktion ist einelementig für alle $q \in Q$ und $a \in \Sigma$).

Bemerkung: Zu jedem DEA $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ gibt es einen NEA, der die gleiche Sprache akzeptiert. Beispiel: der NEA $\mathcal{N} = (\Sigma, Q, \delta_{\text{NEA}}, q^{\text{init}}, F)$ mit $\delta_{\text{NEA}}(q, a) = \{\delta(q, a)\}$, der sich von $\mathcal{A}$ nur in der Transitionsfunktion unterscheidet.

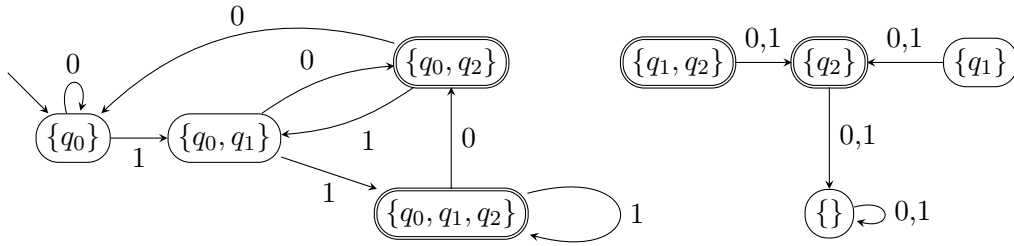
Satz 2.6 (Rabin und Scott): Zu jedem NEA $\mathcal{N}$ mit n Zuständen gibt es einen DEA $\mathcal{A}_{\mathcal{P}}$ mit 2^n Zuständen, sodass $L(\mathcal{A}_{\mathcal{P}}) = L(\mathcal{N})$ gilt.

Zur Vorbereitung des Beweises machen wir zunächst die folgende Definition.

Def. 2.14 (Potenzmengenautomat): Für einen gegebenen NEA $\mathcal{N} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ist der Potenzmengenautomat $\mathcal{A}_{\mathcal{P}}$ wie folgt definiert.

$$\begin{aligned} Q_{\mathcal{P}} &= \mathcal{P}(Q) \\ \delta_{\mathcal{P}}(p, a) &= \bigcup_{q \in p} \delta(q, a) \\ q_{\mathcal{P}}^{\text{init}} &= \{q^{\text{init}}\} \\ F_{\mathcal{P}} &= \{p \in Q_{\mathcal{P}} \mid p \cap F \neq \emptyset\} \end{aligned} \quad \diamond$$

Bsp. 2.12: Der Potenzmengenautomat für den NEA aus Bsp. 2.11 hat das folgende Zustandsdiagramm.



Die vier Zustände auf der rechten Seite sind nicht erreichbar.

BEWEIS (von Satz 2.6): Zeige $L(\mathcal{A}_{\mathcal{P}}) = L(\mathcal{N})$. Dafür beweisen wir zunächst per Induktion über w die folgende Eigenschaft:

$$\forall w \in \Sigma^* : \forall p \in Q_{\mathcal{P}} \setminus \{\{\}\} : \forall q \in Q :$$

$$q \in \tilde{\delta}_{\mathcal{P}}(p, w) \Leftrightarrow \exists q_0 \in p. q_0, q_1, \dots, q_n \in \text{run}(q_0, w), \text{ sodass } q_n = q$$

I.A. ($w = \varepsilon$): $q \in \tilde{\delta}_{\mathcal{P}}(p, \varepsilon) = p$ wähle $q_0 = q$.

I.S. ($w = aw'$) ein beliebiges Wort.

$$q \in \tilde{\delta}_{\mathcal{P}}(p, w) \Leftrightarrow q \in \tilde{\delta}_{\mathcal{P}}(\delta_{\mathcal{P}}(p, a), w')$$

$$\stackrel{\text{I.V.}}{\Leftrightarrow} \exists q_1 \in \delta_{\mathcal{P}}(p, a). q_1, q_2, \dots, q_n \in \text{run}(q_1, w'), \text{ sodass } q_n = q$$

$$\Leftrightarrow \exists q_0 \in p. q_1 \in \delta_{\mathcal{P}}(p, a). q_1, q_2, \dots, q_n \in \text{run}(q_1, w'), \text{ sodass } q_n = q$$

$$\Leftrightarrow \exists q_0 \in p. q_0, q_1, q_2, \dots, q_n \in \text{run}(q_1, aw'), \text{ sodass } q_{n+1} = q$$

Mit Hilfe dieser Eigenschaft zeigen wir nun die Gleichheit $L(\mathcal{A}_P) = L(\mathcal{N})$.

$$\begin{aligned}
 w \in L(\mathcal{A}_P) &\Leftrightarrow \tilde{\delta}_P(q_P^{\text{init}}, w) \in F_P \\
 &\Leftrightarrow \exists p_f \in F_P : \tilde{\delta}_P(q_P^{\text{init}}, w) = p_f \\
 &\Leftrightarrow \exists q_f \in F : q_f \in \tilde{\delta}_P(q_P^{\text{init}}, w) \\
 &\Leftrightarrow \exists q_0 \in q_P^{\text{init}} = \{q_A^{\text{init}}\}. q_0, q_1, \dots, q_n \in \text{run}(q_0, w), \text{ sodass } q_n = q_f \in F \\
 &\Leftrightarrow \exists \text{ initialer, akzeptierender Lauf von } \mathcal{N} \text{ über } w \\
 &\Leftrightarrow w \in L(\mathcal{N}) \quad \square
 \end{aligned}$$

Bemerkung: Es gelten also die folgenden Äquivalenzen.

$$L \text{ regulär} \stackrel{\text{Def. 2.3}}{\iff} L = L(\mathcal{A}) \text{ für einen DEA } \mathcal{A} \iff L = L(\mathcal{N}) \text{ für einen NEA } \mathcal{N}$$

Bemerkung: NEAs sind eine exponentiell kompaktere Repräsentation von regulären Sprachen im folgenden Sinne:

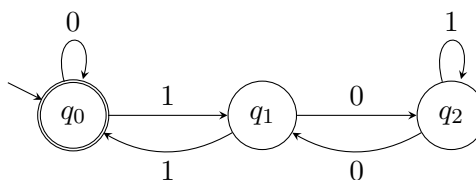
1. Es gibt mit L_n (n -letztes Zeichen) eine Menge von Sprachen, die sich durch einen NEA mit $n + 1$ Zuständen darstellen lassen, aber bei denen ein minimaler DEA mindestens 2^n Zustände hat. (Siehe Übungsblatt 3, Aufgabe 2).
2. Zu jedem NEA mit n Zuständen gibt es einen DEA mit 2^n Zuständen, der die gleiche Sprache akzeptiert. (Satz 2.6).
3. Zu jedem DEA mit n Zuständen gibt es einen NEA mit n Zuständen, der die gleiche Sprache akzeptiert.

2.4 Pumping Lemma (PL) für reguläre Sprachen

Notation: Sei $\text{bin} : \{0, 1\}^* \rightarrow \mathbb{N}$ die Decodierung von Bitstrings in natürliche Zahlen; z.B. $\text{bin}(101) = 5$, $\text{bin}(\varepsilon) = 0$.

Vorlesung:
13.05.2026

Bsp. 2.13: Betrachte den folgenden DEA, der die Sprache der Binärcodierung von durch drei teilbaren Zahlen akzeptiert: $L = \{w \in \{0, 1\}^* \mid \text{bin}(w) \equiv_3 0\}$



Beobachtungen:

- Es gilt offensichtlich, dass $11 \in L$.
- Es gilt auch, dass $1001 \in L$.
- Der Automat hat eine Schleife bei $\tilde{\delta}(q_1, 00) = q_1$, die mehrfach „abgelaufen“ werden kann, ohne die Akzeptanz zu beeinflussen.
- Also gilt auch $100001 \in L$.
- Im Allgemeinen gilt $\forall i \in \mathbb{N} : 1(00)^i 1 \in L$.

Verdacht: Alle „langen“ Wörter lassen sich in der Mitte „aufpumpen“. Wir formalisieren diesen Verdacht im folgenden Lemma.

Lemma 2.7 (Pumping Lemma): Sei L eine reguläre Sprache. Dann gilt:

$$\begin{aligned} \exists n \in \mathbb{N}, n > 0 : \quad & \forall z \in L, |z| \geq n : \\ & \exists u, v, w \in \Sigma^* : \\ & z = uvw, |uv| \leq n, |v| \geq 1 \\ & \text{und } \forall i \in \mathbb{N} : uv^i w \in L \end{aligned}$$

BEWEIS: Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein beliebiger DEA für L . Wähle $n = |Q|$ und $z \in L$ beliebig mit $|z| \geq n$.

Beim Lesen von z durchläuft $\mathcal{A}$ genau $\overbrace{|z| + 1}^{\geq n+1}$ Zustände. Somit gibt es mindestens einen Zustand $q \in Q$, der mehrmals besucht wird (Schubfachprinzip).

Wähle den Zustand q , der als Erster zweimal besucht wird.

$$\begin{aligned} \text{Nun gilt: } \quad & \exists u : \tilde{\delta}(q^{\text{init}}, u) = q && u \text{ Präfix von } z \\ & \exists v : \tilde{\delta}(q, v) = q && uv \text{ Präfix von } z \\ & \exists w : \tilde{\delta}(q, w) \in F && uvw = z \\ & |v| \geq 1 \\ & |uv| \leq n && \text{da } q \text{ als Erster zweimal besucht wird} \end{aligned}$$

$$\begin{aligned} \text{Es folgt für beliebiges } i \in \mathbb{N} : \quad & \tilde{\delta}(q^{\text{init}}, uv^i w) = \tilde{\delta}(q, v^i w) \\ & = \tilde{\delta}(q, w) && \text{denn } \forall i : \tilde{\delta}(q, v^i) = q \\ & \in F && \square \end{aligned}$$

Bsp. 2.14: Die Sprache $L_{\text{centered}} = \{0^n 10^n \mid n \in \mathbb{N}\}$ ist nicht regulär.

Wir geben hierfür einen Widerspruchsbeweis mit Hilfe des Pumping Lemmas PL.

Sei n die Konstante aus dem PL. Wähle $z = 0^n 1 0^n$. (Gültige Wahl, da $|z| = 2n + 1 \geq n$)
Laut PL existieren u, v, w , sodass $z = uvw$ mit $|v| \geq 1, |uv| \leq n$ und $\forall i \in \mathbb{N} : uv^i w \in L$.
Nach Wahl von z gilt nun

- $uv = 0^m$ mit $m \leq n$
- $v = 0^k$ mit $k \geq 1$
- $w = 0^{n-m} 1 0^n$

Betrachte $uv^2w = 0^{m-k} 0^{2k} 0^{n-m} 1 0^n = 0^{n+k} 1 0^n \notin L$. Widerspruch! Somit ist L nicht regulär.

Zur Illustration:

$$\begin{array}{c} \underbrace{0 \dots 0}_n \underbrace{1 0 \dots 0}_n \\ \hline | \text{---} u \text{---} | \text{---} v \text{---} | \text{---} w \text{---} | \end{array}$$

2.5 ε -Transitionen

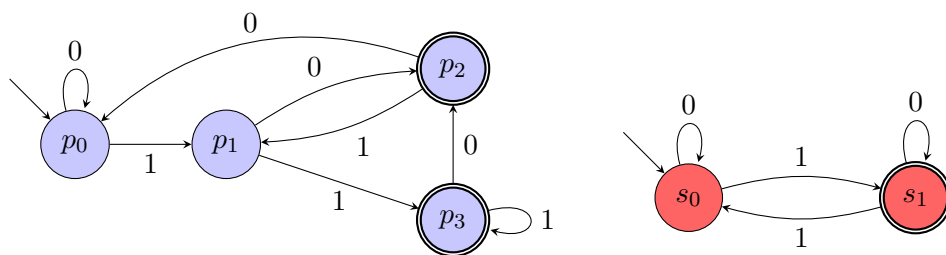
In diesem Unterkapitel führen wir mit dem ε -NEA ein weiteres Automatenmodell ein. Wir wollen ε -NEAs zunächst durch die folgende Fragestellung und anschließende Diskussion motivieren.

Frage: Gegeben zwei reguläre Sprachen L_1, L_2 , ist auch die Konkatenation $L_1 \cdot L_2$ eine reguläre Sprache?

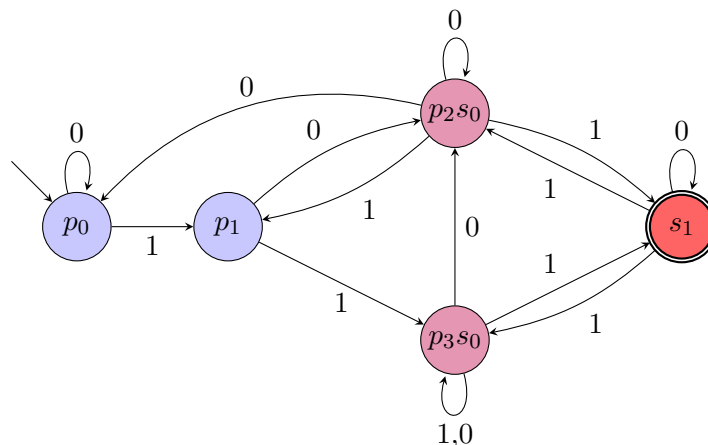
Idee: Gegeben DEA $\mathcal{A}_1$ mit $L(\mathcal{A}_1) = L_1$ und DEA $\mathcal{A}_2$ mit $L(\mathcal{A}_2) = L_2$, konstruiere NEA für $L_1 \cdot L_2$ durch „Hintereinanderschalten“ von $\mathcal{A}_1$ und $\mathcal{A}_2$; immer wenn wir in einem akzeptierenden Zustand von $\mathcal{A}_1$ sind, erlauben wir, in $\mathcal{A}_2$ zu „wechseln“.

Erste, naive (und inkorrekte) Umsetzung dieser Idee: Verschmelze akzeptierende Zustände von $\mathcal{A}_1$ mit dem Startzustand von $\mathcal{A}_2$. Wir betrachten die folgenden Automaten, um zu sehen, dass diese Umsetzung nicht zielführend ist.

Bsp. 2.15: Links: DEA $\mathcal{A}_1$, der Automat aus Bsp. 2.11 eingeschränkt auf die erreichbaren Zustände. Rechts: DEA $\mathcal{A}_2$ mit der Sprache $\{w \in \{0, 1\}^* \mid \text{Anzahl } 1 \text{ in } w \text{ ungerade}\}$.



Unten: NEA $\mathcal{N}_{\text{naiv}}$ aus der naiven und inkorrekten Konstruktion für die Konkatenation.



Dieser NEA akzeptiert nun auch das Wort $w = 11011$. Allerdings ist w nicht in der Konkatenation $L(\mathcal{A}_1) \cdot L(\mathcal{A}_2)$, denn es gibt keine Zerlegung $w = w_1 \cdot w_2$, sodass sowohl das Präfix w_1 von $\mathcal{A}_1$ als auch das Suffix w_2 von $\mathcal{A}_2$ akzeptiert wird.

Das „Verschmelzen“ von p_2 (bzw. p_3) mit s_0 war also keine gute Idee. Was uns aber helfen würde: ein Zustandsübergang, der es uns erlaubt, von Zustand p_2 (bzw. p_3) in den Zustand s_0 zu gehen, ohne dabei ein Zeichen zu lesen.

Wir nennen solch einen Zustandsübergang ε -Transition und definieren einen Automaten, der solche Zustandsübergänge haben kann, wie folgt.

Vorlesung:
19.05.2026

Def. 2.15 (ε -NEA): Ein *nichtdeterministischer endlicher Automat mit ε -Transitionen* ist ein 5-Tupel

$$\mathcal{B} = (\Sigma, Q, \delta, q^{\text{init}}, F)$$

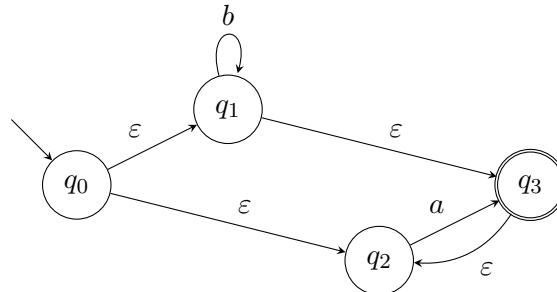
wobei $\Sigma, Q, q^{\text{init}}, F$ wie bei NEAs (bzw. DEAs) definiert sind und die Transitionsfunktion den folgenden Typ hat.

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q) \quad \diamond$$

Bemerkung: Wenn $f : A \rightarrow \mathcal{P}(B)$ ist, dann schreiben wir $\hat{f} : \mathcal{P}(A) \rightarrow \mathcal{P}(B)$ für die Fortsetzung von f auf Mengen. D.h. $\hat{f}(M) = \bigcup_{a \in M} f(a)$, falls $M \subseteq A$. Z.B. verwenden wir $\hat{\delta} : \mathcal{P}(Q) \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$.

Bemerkung: Funktionen $f, g : A \rightarrow \mathcal{P}(B)$ können bzgl. ihres Bildbereichs geordnet werden. Wir schreiben $f \subseteq g$, falls $\forall a \in A. f(a) \subseteq g(a)$. Diese Relation auf $A \rightarrow \mathcal{P}(B)$ ist eine partielle Ordnung mit kleinstem Element $f_0 : a \mapsto \emptyset$, die Funktion, die jedes $a \in A$ auf die leere Menge abbildet.

Bsp. 2.16: Zustandsdiagramm eines ε -NEA über dem Alphabet $\Sigma = \{a, b\}$.



Im Folgenden sei $\mathcal{B}$ immer ein ε -NEA.

Wie bei den bisher definierten Automaten wollen wir mit Hilfe eines ε -NEA eine Sprache definieren. Wir benötigen dafür zunächst zwei weitere Definitionen.

Der ε -Abschluss ist eine Abbildung, die jedem Zustand q die Menge der Zustände zuordnet, die von q über ε -Transitionen erreichbar sind. Wir definieren diese Abbildung formal wie folgt. Dabei verwenden wir den Abbildungsnamen ecl , um an den englischen Begriff „ ε closure“ zu erinnern.

Def. 2.16: Der ε -Abschluss $\text{ecl}_{\mathcal{B}} : Q \rightarrow \mathcal{P}(Q)$ ist die kleinste Abbildung (bzgl. $\subseteq$), die für alle $q, q', q'' \in Q$ die folgenden Eigenschaften erfüllt:

$$q \in \text{ecl}_{\mathcal{B}}(q) \qquad \frac{q' \in \text{ecl}_{\mathcal{B}}(q) \quad q'' \in \delta(q', \varepsilon)}{q'' \in \text{ecl}_{\mathcal{B}}(q)}$$

◇

Offensichtlich kann immer eine endliche explizite Repräsentation von $\text{ecl}_{\mathcal{B}}$ berechnet werden: Starte in jedem Zustand einmal und folge mit Breitensuche allen ε -Kanten im Zustandsdiagramm.

Bsp.: Für den ε -NEA aus Bsp. 2.16 sieht $\text{ecl}_{\mathcal{B}}$ wie folgt aus:

q	q_0	q_1	q_2	q_3
$\text{ecl}(q)$	$\{q_0, q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_2\}$	$\{q_2, q_3\}$

Als Nächstes definieren wir eine mengenwertige Funktion, die uns für einen Zustand q und ein Wort w sagt, welche Zustände von q durch w erreichbar sind. Der Name der Funktion „reach“ soll dabei an das englische Wort „reachability“ erinnern.

Def. 2.17: Die *Erreichbarkeitsfunktion* $\text{reach}_{\mathcal{B}} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$ ist definiert durch:

$$\begin{aligned} \text{reach}_{\mathcal{B}}(q, \varepsilon) &= \text{ecl}_{\mathcal{B}}(q) \\ \text{reach}_{\mathcal{B}}(q, a.w) &= \bigcup \{ \text{reach}_{\mathcal{B}}(q'', w) \mid q' \in \text{ecl}_{\mathcal{B}}(q), q'' \in \delta(q', a) \} \\ &= \bigcup \{ \text{reach}_{\mathcal{B}}(q'', w) \mid q'' \in \hat{\delta}(\text{ecl}_{\mathcal{B}}(q), a) \} \\ &= \widehat{\text{reach}_{\mathcal{B}}(\hat{\delta}(\text{ecl}_{\mathcal{B}}(q), a), w)} \end{aligned}$$

◇

Für den ε -NEA aus Bsp. 2.16 können wir $\text{reach}_{\mathcal{B}}$ mit Hilfe der folgenden Tabelle angeben. Dabei bedeutet der Eintrag von einer Sprache L in Zeile q_i und Spalte q_j , dass für alle $w \in L$ das Tripel $q_j \in \text{reach}_{\mathcal{B}}(q_i, w)$.

$\text{reach}_{\mathcal{B}}$	q_0	q_1	q_2	q_3
q_0	$\{\varepsilon\}$	$\{b\}^*$	$\{b\}^* \cdot \{a\}^*$	$\{b\}^* \cdot \{a\}^*$
q_1	$\{\}$	$\{b\}^*$	$\{b\}^* \cdot \{a\}^*$	$\{b\}^* \cdot \{a\}^*$
q_2	$\{\}$	$\{\}$	$\{a\}^*$	$\{a\} \cdot \{a\}^*$
q_3	$\{\}$	$\{\}$	$\{a\}^*$	$\{a\}^*$

Def. 2.18: Ein Wort $w \in \Sigma^*$ wird von $\mathcal{B}$ *akzeptiert*, wenn $\text{reach}_{\mathcal{B}}(q^{\text{init}}, w) \cap F \neq \emptyset$. Die von $\mathcal{B}$ akzeptierte Sprache $L(\mathcal{B})$ ist die Menge der von $\mathcal{B}$ akzeptierten Wörter, d.h. $L(\mathcal{B}) = \{w \in \Sigma^* \mid \text{reach}_{\mathcal{B}}(q^{\text{init}}, w) \cap F \neq \emptyset\}$. ◇

Offensichtlich gibt es zu jedem NEA $\mathcal{N}$ einen ε -NEA $\mathcal{B}$, der die gleiche Sprache akzeptiert. Die Konstruktion ist einfach: Erweitere die Transitionsfunktion um $\delta(q, \varepsilon) = \{q\}$ für alle $q \in Q$. Für die Sprachgleichheit zeigen wir mittels Induktion über w , dass für alle Wörter $w \in \Sigma^*$ gilt:

$$\exists \text{ Lauf } q_0, q_1, \dots, q_n \text{ von } \mathcal{N} \text{ über } w \iff q_n \in \text{reach}_{\mathcal{B}}(q_0, w)$$

Der folgende Satz zeigt uns, dass auch die umgekehrte Richtung gilt.

Satz 2.8: Zu jedem ε -NEA $\mathcal{B}$ gibt es einen NEA $\mathcal{N}$, sodass $L(\mathcal{N}) = L(\mathcal{B})$ gilt.

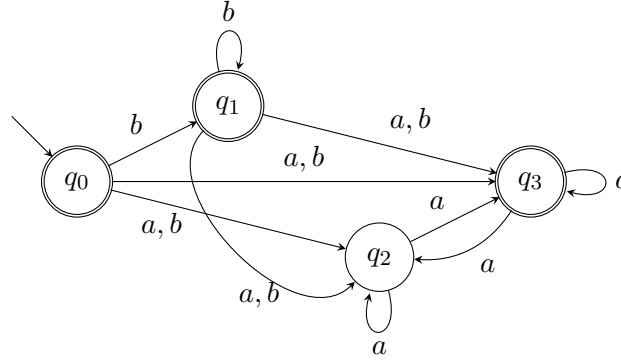
Zur Vorbereitung des Beweises machen wir zunächst die folgende Definition.

Def. 2.19 (ε -freier Automat): Für einen gegebenen ε -NEA $\mathcal{B} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ definieren wir den NEA $\mathcal{N} = (\Sigma, Q, \delta_{\mathcal{N}}, q^{\text{init}}, F_{\mathcal{N}})$ mit

$$\begin{aligned} \delta_{\mathcal{N}}(q, a) &= \bigcup \{ \text{ecl}_{\mathcal{B}}(q'') \mid q'' \in \hat{\delta}(\text{ecl}_{\mathcal{B}}(q), a) \} \\ F_{\mathcal{N}} &= \{q \in Q \mid \text{ecl}_{\mathcal{B}}(q) \cap F \neq \emptyset\} \end{aligned}$$

und nennen diesen NEA den ε -freien Automaten von $\mathcal{B}$. ◇

Bsp. 2.17: Der ε -freie Automat für den ε -NEA aus Bsp. 2.16 hat das folgende Zustandsdiagramm.



BEWEIS (von Satz 2.8: ε -Eliminierung): Zeige $L(\mathcal{N}) = L(\mathcal{B})$. Dabei verwenden wir die folgende Eigenschaft, die wir mittels Induktion über die Länge von w in den Übungen zeigen werden.

$\forall w \in \Sigma^+ \forall q, q' \in Q :$

$$q' \in \text{reach}_{\mathcal{B}}(q, w) \Leftrightarrow \exists \underbrace{q_0, q_1, \dots, q_n}_{\text{Lauf}} \in Q, \text{ sodass } n = |w|, q_0 = q \text{ und } q_n = q'$$

Mit Hilfe dieser Eigenschaft zeigen wir nun für $w \neq \varepsilon$:

$$\begin{aligned}
 w \in L(\mathcal{B}) &\stackrel{\text{Def. 2.3}}{\Leftrightarrow} \text{reach}_{\mathcal{B}}(q^{\text{init}}, w) \cap F \neq \emptyset \\
 &\Leftrightarrow \exists q_f \in F_{\mathcal{N}} : \underbrace{q_0, q_1, \dots, q_n}_{\text{Lauf}} \in Q, \text{ sodass } n = |w|, q_0 = q^{\text{init}} \text{ und } q_n = q_f \\
 &\stackrel{\text{Def. 2.18}}{\Leftrightarrow} w \in L(\mathcal{N})
 \end{aligned}$$

Der Fall $w = \varepsilon$ folgt mit $\varepsilon \in L(\mathcal{B}) \Leftrightarrow F \cap \text{ecl}_{\mathcal{B}}(q^{\text{init}}) \neq \emptyset \Leftrightarrow q^{\text{init}} \in F_{\mathcal{N}} \Leftrightarrow \varepsilon \in L(\mathcal{N})$. $\square$

Mit Hilfe der ε -NEAs greifen wir nun die zu Beginn von Abschnitt 2.5 aufgeworfene Fragestellung wieder auf und zeigen, dass für je zwei reguläre Sprachen auch die Konkatination regulär ist.

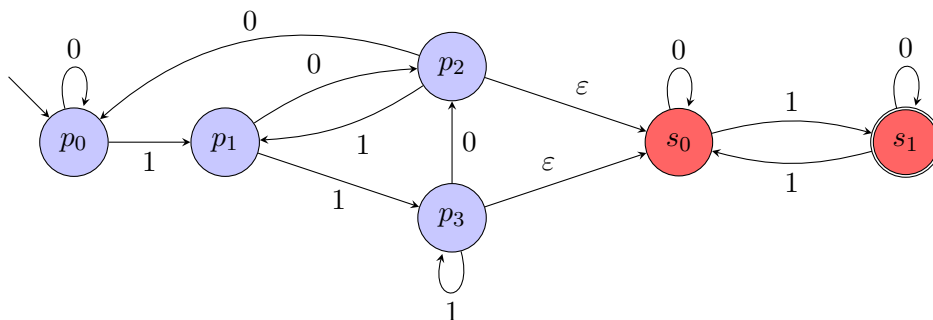
Wir geben hierfür zunächst eine Konstruktion an.

Def. 2.20: Gegeben zwei NEA- ε $\mathcal{B}_i = (\Sigma, Q_i, \delta_i, q_i^{\text{init}}, F_i)$, $i = 1, 2$, definieren wir den ε -NEA für Konkatination $\mathcal{B} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ wie folgt.

$$\begin{aligned}
 Q &= Q_1 \dot{\cup} Q_2 \\
 \delta(q, a) &= \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \end{cases} & \delta(q, \varepsilon) &= \begin{cases} \delta_1(q, \varepsilon) \cup \{q_2^{\text{init}}\} & q \in F_1 \\ \delta_1(q, \varepsilon) & q \in Q_1 \setminus F_1 \\ \delta_2(q, \varepsilon) & q \in Q_2 \end{cases} \\
 q^{\text{init}} &= q_1^{\text{init}} \\
 F &= F_2
 \end{aligned}$$

◇

Bsp. 2.18: Der ε -NEA für Konkatination für die beiden Automaten aus Bsp. 2.15 hat das folgende Zustandsdiagramm.



Lemma 2.9: Die vom ε -NEA für Konkatination akzeptierte Sprache ist $L(\mathcal{B}_1) \cdot L(\mathcal{B}_2)$.

BEWEIS: Zeige mittels Induktion über w_1 , dass $\forall w_1, w_2 \in \Sigma^*, \forall q_1 \in Q_1, \forall q'_1 \in F, \forall q'_2 \in Q_2$ die folgende Eigenschaft gilt.

$$q'_1 \in \text{reach}_{\mathcal{B}_1}(q_1, w_1) \text{ und } q'_2 \in \text{reach}_{\mathcal{B}_2}(q^{\text{init}}, w_2) \Leftrightarrow q'_2 \in \text{reach}_{\mathcal{B}}(q_1, w_1 w_2)$$

Mit dieser Eigenschaft folgt leicht für alle $w_1, w_2 \in \Sigma^*$:

$$w_1 \in L(\mathcal{B}_1) \text{ und } w_2 \in L(\mathcal{B}_2) \Leftrightarrow w_1 \cdot w_2 \in L(\mathcal{B}) \quad \square$$

2.6 Abschlusseigenschaften

Def. 2.21: Eine Menge X heißt *abgeschlossen* unter Operation $f : X^n \rightarrow X$, falls $\forall x_1, \dots, x_n \in X : f(x_1, \dots, x_n) \in X$. ◇

Vorlesung:
20.05.2026

Zum Beispiel sind die natürlichen Zahlen abgeschlossen unter Addition, aber nicht abgeschlossen unter Subtraktion.

Im Folgenden schreiben wir *REG* für die Menge aller regulären Sprachen.

Lemma 2.10: Die Menge *REG* der regulären Sprachen ist abgeschlossen unter Komplement.

BEWEIS: Sei L eine reguläre Sprache. Dann gibt es (per Definition) einen DEA $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ der L akzeptiert. Wir konstruieren den DEA $\bar{\mathcal{A}} = (\Sigma, Q, \delta, q^{\text{init}}, Q \setminus F)$ für $\bar{L}$, bei dem ein Zustand genau dann akzeptierend ist, wenn der Zustand in $\mathcal{A}$ nicht akzeptierend ist. Man kann leicht zeigen, dass $L(\bar{\mathcal{A}}) = \bar{L}$; somit ist auch $\bar{L}$ regulär. $\square$

Lemma 2.11: Die Menge *REG* der regulären Sprache ist abgeschlossen unter dem Stern-Operator.

BEWEIS: Sei L eine reguläre Sprache. Dann gibt es einen ε -NEA $\mathcal{B} = (\Sigma, Q, \delta, q^{\text{init}}, F)$, der L akzeptiert. Wir konstruieren einen NEA $\mathcal{B}_* = (\Sigma, Q \cup \{q_*^{\text{init}}\}, \delta_*, q_*^{\text{init}}, F \cup \{q_*^{\text{init}}\})$ für L^* , indem wir

- einen neuen akzeptierenden Startzustand einführen,
- vom neuen Startzustand einen ε -Transition zum alten Startzustand hinzufügen und
- von jedem akzeptierenden Zustand einen ε -Transition zum alten Startzustand hinzufügen.

$$\delta_*(q, x) = \begin{cases} \delta(q, x) & q \notin F \text{ oder } x \neq \varepsilon \\ \delta(q, x) \cup \{q^{\text{init}}\} & q \in F \text{ und } x = \varepsilon \\ \{\} & q = q_*^{\text{init}} \text{ und } x \in \Sigma \\ \{q^{\text{init}}\} & q = q_*^{\text{init}} \text{ und } x = \varepsilon \end{cases}$$

Man kann via Induktion über die Länge von Wörtern zeigen, dass $L(\mathcal{B}_*) = L^*$ gilt. $\square$

Bemerkung: Die Menge der regulären Sprachen *REG* ist unter den folgenden Operationen abgeschlossen.

$\cap$	(Durchschnitt)	Satz 2.1
$\cup$	(Vereinigung)	Präsenzübungen erste Woche, Aufgabe 3
$\overline{}$	(Komplement)	Lemma 2.10
$\cdot$	(Konkatenation)	Lemma 2.9
$*$	(Kleene-Stern)	Lemma 2.11

2.7 Reguläre Ausdrücke

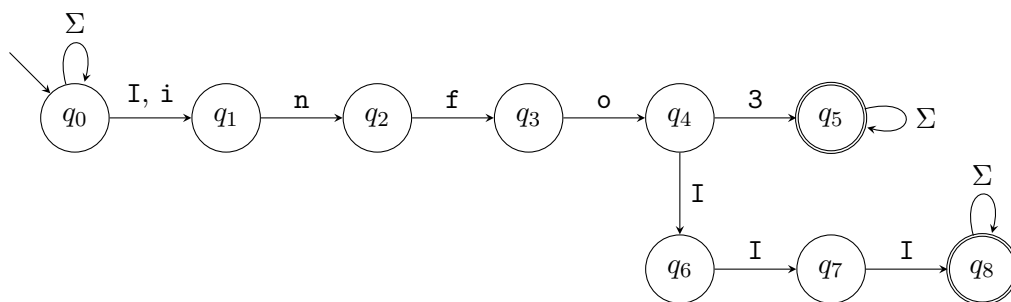
Problemstellung: Sie haben auf Ihrem Rechner irgendwo eine Textdatei mit Notizen zu dieser Vorlesung, können sich aber gerade nicht an den Pfad erinnern. Sie wissen aber noch, dass die Zeichenkette `info3` oder `infoIII` enthalten ist. Möglicherweise war das `i` am Anfang aber auch groß geschrieben. Da Sie mehrere Dateien mit dieser Zeichenkette haben, soll der Rechner jeweils Dateinamen und die entsprechenden Zeilen ausgeben.

Auf Systemen, auf denen die GNU Tools installiert sind, kann dieses Problem mit dem folgenden Befehl gelöst werden.

```
grep -r "(I|i)nfo(III|3)" /
```

Konzeptuell soll Ihr Rechner hier Instanzen des Wortproblems lösen. Ihre Sprache besteht aus allen Zeichenketten, die `info3` in den oben beschriebenen Varianten enthalten. Die Wörter, die getestet werden sollen, sind die Zeilen aller Dateien auf dem Rechner.

Eine Umsetzung: Gib dem Rechner den folgenden NEA, welchen er dann in einen DEA konvertiert, um das Wortproblem effizient zu lösen.



Problem: Dem Rechner einen Graphen als Eingabe zu übermitteln ist unkomfortabel oder zeitaufwändig. Wir führen deshalb einen weiteren textbasierten Formalismus für reguläre Sprachen ein. Dieser hat große Ähnlichkeiten zum dem in der Praxis verwendeten Input von `grep`.

Def. 2.22: Die Menge $RE(\Sigma)$ der *regulären Ausdrücke über Σ* ist induktiv definiert durch:

- $\emptyset \in RE(\Sigma)$
- $\varepsilon \in RE(\Sigma)$
- $a \in RE(\Sigma)$ für alle $a \in \Sigma$

- falls $r, s \in RE(\Sigma)$
 - $(r + s) \in RE(\Sigma)$
 - $(r \cdot s) \in RE(\Sigma)$
 - $r^* \in RE(\Sigma)$

◇

Konventionen: Wir möchten nicht immer alle Klammern schreiben müssen. Wir führen deshalb die folgenden Präzedenzregeln ein: „ $*$ “ bindet stärker als „ $\cdot$ “ und „ $\cdot$ “ bindet stärker als „ $+$ “. Nach Definition der Semantik werden wir außerdem sehen, dass „ $\cdot$ “ und „ $+$ “ assoziativ sind. Unsere Konvention ist: Sofern mit Hilfe dieser Regeln der reguläre Ausdruck zweifelsfrei rekonstruiert werden kann, dürfen wir Klammern und „ $\cdot$ “ weglassen. Wir schreiben z.B. $110 + 0$ statt $((1 \cdot (1 \cdot 0)) + 0)$.

Bsp. 2.19:

1. $(\emptyset\varepsilon)^* + a$ ist ein regulärer Ausdruck über $\Sigma = \{a\}$.
2. $\emptyset\varepsilon\emptyset\varepsilon$ ist ein regulärer Ausdruck über jedem Alphabet.
3. $aaaabbbbbbb$ ist ein regulärer Ausdruck über $\Sigma = \{a, b\}$.
4. a^4b^7 ist **kein** regulärer Ausdruck
5. $(0 + 1 \cdot (01^*0)^* \cdot 1)^*$ ist ein regulärer Ausdruck über $\Sigma = \{0, 1\}$.
6. $(A + \dots + Z + a + \dots + z + 0 + \dots + 9)^*(I + i) \text{ n f o } (3 + I I I) (A + \dots + Z + a + \dots + z + 0 + \dots + 9)^*$ ist ein regulärer Ausdruck⁶ über dem Alphabet $\Sigma = \{A, \dots, Z, a, \dots, z, 0, \dots, 9\}$.

Wir geben regulären Ausdrücken mit Hilfe der folgenden Definition eine Semantik.

Def. 2.23: Die durch einen regulären Ausdruck *beschriebene Sprache* $\llbracket \cdot \rrbracket : RE(\Sigma) \rightarrow \mathcal{P}(\Sigma^*)$ ist induktiv definiert durch:

$$\begin{aligned}
 \llbracket \emptyset \rrbracket &= \emptyset \\
 \llbracket \varepsilon \rrbracket &= \{\varepsilon\} \\
 \llbracket a \rrbracket &= \{a\} \quad a \in \Sigma \\
 \llbracket (r + s) \rrbracket &= \llbracket r \rrbracket \cup \llbracket s \rrbracket \\
 \llbracket (r \cdot s) \rrbracket &= \llbracket r \rrbracket \cdot \llbracket s \rrbracket \\
 \llbracket r^* \rrbracket &= \llbracket r \rrbracket^*
 \end{aligned}$$

◇

⁶Wir verwenden hier $A + \dots + Z$ als Abkürzung für die Disjunktion aus 26 Zeichen. Der eigentliche reguläre Ausdruck besteht aus der Disjunktion; die drei Punkte kommen darin nicht vor.

Bsp. 2.20:

$$\llbracket a + (\emptyset\varepsilon)^* \rrbracket = \llbracket a \rrbracket \cup (\llbracket \emptyset \rrbracket \cdot \llbracket \varepsilon \rrbracket)^* = \{a\} \cup (\emptyset)^* = \{a\} \cup \{\varepsilon\} = \{a, \varepsilon\}$$

Bsp. 2.21: Die aus Bsp. 2.11 bekannte Sprache

$$L_2 = \{w \in \{0, 1\}^* \mid \text{das zweitletzte Zeichen von } w \text{ ist } 1\}$$

wird durch den regulären Ausdruck $(0 + 1)^* 1 (0 + 1)$ beschrieben.

Satz 2.12 (Kleene): L ist regulär $\Leftrightarrow L$ ist Sprache eines regulären Ausdrucks.

BEWEIS (Kleene, $\Leftarrow$): Betrachte zu einem regulären Ausdruck $r \in RE(\Sigma)$ die durch diesen erzeugte Sprache $L = \llbracket r \rrbracket$. Zeige via strukturelle Induktion über den Aufbau regulärer Ausdrücke, dass $\llbracket r \rrbracket$ regulär ist.

- I.A.: • $r = \emptyset$, $\llbracket r \rrbracket = \emptyset$ ist regulär (NEA: $\rightarrow \bigcirc$)
- $r = \varepsilon$, $\llbracket r \rrbracket = \{\varepsilon\}$ ist regulär (NEA: $\rightarrow \bigcirc\bigcirc$)
- $r = a$, $\llbracket r \rrbracket = \{a\}$ ist regulär (NEA: $\rightarrow \bigcirc \xrightarrow{a} \bigcirc\bigcirc$)

I.V.: Für $i \in \{1, 2\}$ gilt: $\llbracket r_i \rrbracket$ ist regulär.

- I.S.: • $r = r_1 + r_2$, $\llbracket r \rrbracket = \llbracket r_1 \rrbracket \cup \llbracket r_2 \rrbracket$ ist regulär nach I.V. und Abschluss unter Vereinigung (gezeigt in Präsenzübungen, Aufgabe 3)
- $r = r_1 \cdot r_2$, $\llbracket r \rrbracket = \llbracket r_1 \rrbracket \cdot \llbracket r_2 \rrbracket$ ist regulär nach I.V. und Lemma 2.9
- $r = r_1^*$, $\llbracket r \rrbracket = \llbracket r_1 \rrbracket^*$ ist regulär nach I.V. und Lemma 2.11 □

Im verbleibenden Teil des Abschnitts zu regulären Ausdrücken werden wir zeigen, dass wir für jeden DEA einen regulären Ausdruck mit der gleichen Sprache konstruieren können. (Richtung „ $\Rightarrow$ “ von Satz 2.12). Um unsere Idee zu beschreiben, benötigen wir zunächst die folgende Definition.

Def. 2.24: Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein DEA. Für einen Zustand $q \in Q$ ist die *Sprache des Zustands* $L_q = \{w \in \Sigma^* \mid \tilde{\delta}(q, w) \in F\}$ die Sprache der Wörter, die von Zustand q aus in einen akzeptierenden Zustand führen. ◇

Im Folgenden betrachten wir einen beliebigen DEA und nummerieren dessen Zustände $Q = \{q_0, q_1, \dots, q_n\}$. Wir leiten nun ein Gleichungssystem zwischen den Sprachen L_{q_i} her.

$$L_{q_i} = \{w \in \Sigma^* \mid \tilde{\delta}(q_i, w) \in F\}$$

Zunächst teilen wir L_{q_i} in den Teil, der (potentiell) das leere Wort enthält, und den Teil, der die nicht-leeren Wörter enthält.

$$= \{\varepsilon \mid q_i \in F\} \cup \bigcup_{a \in \Sigma} \{a\} \{w' \in \Sigma^* \mid \tilde{\delta}(\delta(q_i, a), w') \in F\}$$

Die nicht-leeren Wörter hängen von den Sprachen der Folgezustände ab

$$= \{\varepsilon \mid q_i \in F\} \cup \bigcup_{a \in \Sigma} \{a\} L_{\delta(q_i, a)}$$

Anstatt die Vereinigung über die Transitionen a und Folgezustandssprachen L_{q_j} zu bilden, lassen sich die nicht-leeren Wörter von L_{q_i} auch als Vereinigung über alle Zustände mit entsprechend gewählten *Koeffizienten* A_{ij} formulieren; die Zustände, die keine Folgezustände sind, haben den Koeffizienten $\emptyset$.

$$= \{\varepsilon \mid q_i \in F\} \cup \bigcup_{j=0}^n \underbrace{\{a \in \Sigma \mid \delta(q_i, a) = q_j\}}_{A_{ij} \neq \varepsilon} L_{q_j}$$

Diese Gleichungen lassen sich analog als Gleichungen von regulären Ausdrücken r_i formulieren:

$$r_i = N(q_i) + \sum_{j=0}^n R_{ij} r_j \quad (\text{RegExGlSys})$$

wobei $\llbracket r_i \rrbracket = L_{q_i}$ und $R_{ij} = \sum \{a \in \Sigma \mid \delta(q_i, a) = q_j\}$ mit $\varepsilon \notin \llbracket R_{ij} \rrbracket$ und

$$N(q_i) = \begin{cases} \varepsilon & q_i \in F \\ \emptyset & q_i \notin F \end{cases}$$

Bsp. 2.22: Wir wollen diese Gleichungen zunächst an einem Beispiel betrachten und verwenden dafür den aus Bsp. 2.13 bekannten DEA, der die Sprache der Binärcodierung von durch drei teilbaren Zahlen akzeptiert: $L = \{w \in \{0, 1\}^* \mid \text{bin}(w) \equiv_3 0\}$.

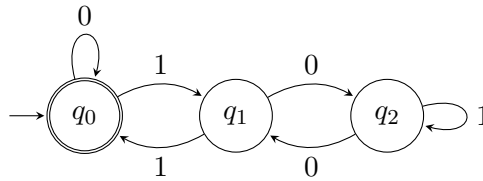


Abb. 4: DEA „modulo 3“

Wir erhalten das folgende Gleichungssystem mit drei Unbekannten.

$$\begin{aligned} r_0 &= \varepsilon + 0 \cdot r_0 + 1 \cdot r_1 \\ r_1 &= 1 \cdot r_0 + 0 \cdot r_2 \\ r_2 &= 0 \cdot r_1 + 1 \cdot r_2 \end{aligned}$$

Wir kennen bisher kein systematisches Verfahren zum Lösen solcher Gleichungen und betrachten deshalb das folgende Lemma.

Lemma 2.13 (Ardens Lemma):

Für die Gleichung $X = A \cdot X \cup B$ über den Sprachen $A, B, X \subseteq \Sigma^*$ gilt:

1. Die Sprache A^*B ist eine Lösung für X .
2. Falls $\varepsilon \notin A$, so ist diese Lösung eindeutig.

BEWEIS (Teil 1): Durch Einsetzen in die Gleichung rechnen wir nach, dass A^*B tatsächlich eine Lösung ist.

$$\underbrace{A^*B}_X = (AA^* \cup \{\varepsilon\})B = A \cdot \underbrace{(A^*B)}_X \cup B.$$

Daraus folgt $A^*B \subseteq X$. □

BEWEIS (Beschreibung des Agda-Beweises von **ardens-left**): Sei $\varepsilon \notin A$ und sei X eine Lösung der Gleichung $X = A \cdot X \cup B$. Der Agda-Beweis zeigt die Inklusion

$$X \subseteq A^*B.$$

Zusammen mit der bereits im manuellen Beweis verwendeten Inklusion $A^*B \subseteq X$ folgt daraus die Eindeutigkeit der Lösung.

Der eigentliche Induktionsbeweis steckt in der Hilfsaussage **ardens-left-gen**. Gezeigt wird die stärkere Aussage:

$$\forall n \in \mathbb{N} : \forall w \in \Sigma^* : |w| < n \text{ und } w \in X \quad \Rightarrow \quad w \in A^*B.$$

Wir führen eine Induktion über n durch.

I.A. ($n = 0$): Es gibt kein Wort w mit $|w| < 0$, also ist nichts zu zeigen.

I.S. ($n \rightsquigarrow n + 1$): Sei $w \in \Sigma^*$ mit $|w| < n + 1$ und $w \in X$. Wegen $X = A \cdot X \cup B$ gibt es zwei Möglichkeiten.

- Falls $w \in B$, dann ist $w = \varepsilon \cdot w$ mit $\varepsilon \in A^0$. Also gilt $w \in A^*B$.

- Falls $w \in A \cdot X$, dann gibt es Wörter u, v mit $w = uv$, $u \in A$ und $v \in X$. Der Fall $u = \varepsilon$ ist unmöglich, denn daraus würde $\varepsilon \in A$ folgen, im Widerspruch zur Annahme. Also ist $u \neq \varepsilon$. Dann ist v ein echtes Restwort von w ; aus $|w| < n + 1$ folgt $|v| < n$. Nach Induktionsvoraussetzung gilt daher $v \in A^*B$. Also gibt es ein j sowie Wörter y, z mit $v = yz$, $y \in A^j$ und $z \in B$. Wegen $u \in A$ folgt $uy \in A^{j+1}$ und damit $w = uv = uyz \in A^{j+1}B \subseteq A^*B$.

Damit ist die Hilfsaussage bewiesen. Für **ardens-left** selbst wählt der Agda-Beweis zu einem beliebigen $w \in X$ den Wert $n = |w| + 1$. Dann gilt $|w| < n$, und die Hilfsaussage liefert sofort $w \in A^*B$. Also gilt $X \subseteq A^*B$. $\square$

Wir können Ardens Lemma wie folgt auch für reguläre Ausdrücke formulieren:

Korollar 2.14: Seien r_X, r_A, r_B reguläre Ausdrücke mit $\varepsilon \notin \llbracket r_A \rrbracket$, sodass die folgende Gleichung gilt:

$$\llbracket r_X \rrbracket = \llbracket r_A \cdot r_X + r_B \rrbracket$$

Dann ist der reguläre Ausdruck

$$r_A^* r_B$$

eine Lösung für r_X , welche die Gleichung erfüllt. Außerdem erzeugen alle anderen Lösungen die gleiche Sprache wie $r_A^* r_B$. $\diamond$

Wir haben nun alle Hilfsmittel, um den Beweis für die fehlende Richtung zu führen.

BEWEIS (Kleene, $\Rightarrow$): Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ der oben diskutierte DEA mit $Q = \{q_0, q_1, \dots, q_n\}$ und $q^{\text{init}} = q_0$, für dessen Sprache wir einen regulären Ausdruck konstruieren möchten.

Mit Hilfe von Ardens Lemma (und weiteren Rechenregeln für Sprachen) können wir nun das Gleichungssystem (RegExGlsys) iterativ lösen. Wir beginnen mit Gleichung r_n :

$$\begin{aligned} r_n &= N(q_n) + \sum_{j=0}^n R_{nj} r_j \\ &= \underbrace{N(q_n) + \left(\sum_{j=0}^{n-1} R_{nj} r_j \right)}_{r_B} + \underbrace{R_{nn}}_{r_A} r_n \end{aligned}$$

Wie oben angedeutet ist nach dem Herausziehen des n -ten Summenglieds Ardens Lemma anwendbar (beachte: $\varepsilon \notin \llbracket R_{nn} \rrbracket$); wir erhalten:

$$r_n := R_{nn}^* \left(N(q_n) + \sum_{j=0}^{n-1} R_{nj} r_j \right)$$

Dieses Ergebnis in $r_0, \dots, r_{n-1}$ eingesetzt ergibt:

$$r_i = N(q_i) + \left(\sum_{j=0}^{n-1} R_{ij} r_j \right) + \underbrace{R_{in} R_{nn}^* \left(N(q_n) + \sum_{j=0}^{n-1} R_{nj} r_j \right)}_{r_n}$$

(Distributivgesetz: $R_{in} R_{nn}^*$)

$$= N(q_i) + \left(\sum_{j=0}^{n-1} R_{ij} r_j \right) + R_{in} R_{nn}^* N(q_n) + \sum_{j=0}^{n-1} R_{in} R_{nn}^* R_{nj} r_j$$

(Zusammenlegen der Summen und Ausklammern von r_j)

$$= N(q_i) + R_{in} R_{nn}^* N(q_n) + \sum_{j=0}^{n-1} (R_{ij} + R_{in} R_{nn}^* R_{nj}) r_j$$

Nach diesen Umformungen ergeben sich ε -freie Koeffizienten $R_{nj} + R_{in} R_{nn}^* R_{nj}$ für r_j und wir können mit dem Auflösen der Summe von $n-1$ analog zu n fortfahren. Am Ende erhalten wir einen regulären Ausdruck als Lösung für r_0 . Per Konstruktion gilt $\llbracket r_0 \rrbracket = L_{q_0} = L_{q^{\text{init}}} = L(\mathcal{A})$. $\square$

Bsp. 2.23: Wir betrachten noch einmal Bsp. 2.22 und lösen das Gleichungssystem auf die im Beweis beschriebene Weise.

$$\begin{aligned} r_0 &= \varepsilon + 0 \cdot r_0 + 1 \cdot r_1 \\ r_1 &= 1 \cdot r_0 + 0 \cdot r_2 \\ r_2 &= \underbrace{0 \cdot r_1}_B + \underbrace{1}_A \cdot r_2 \end{aligned}$$

Ardens Lemma auf r_2 anwenden:

$$r_2 = 1^* \cdot 0 \cdot r_1$$

Einsetzen in r_1 :

$$r_1 = \underbrace{1 \cdot r_0}_B + \underbrace{0 \cdot 1^* \cdot 0 \cdot r_1}_A$$

Ardens Lemma auf r_1 anwenden:

$$r_1 = (01^*0)^* \cdot 1 \cdot r_0$$

Einsetzen in r_0 :

$$\begin{aligned} r_0 &= \varepsilon + 0 \cdot r_0 + 1 \cdot (01^*0)^* \cdot 1 \cdot r_0 \\ &= \underbrace{\varepsilon}_B + \underbrace{(0 + 1 \cdot (01^*0)^* \cdot 1)}_A \cdot r_0 \end{aligned}$$

Ardens Lemma auf r_0 anwenden:

$$\begin{aligned} r_0 &= (0 + 1 \cdot (01^*0)^* \cdot 1)^* \cdot \underline{\varepsilon} \\ &= (0 + 1(01^*0)^*1)^* \end{aligned}$$

3 Grammatiken und kontextfreie Sprachen

Def. 3.1: Eine *Grammatik* ist ein 4-Tupel (Σ, N, P, S) mit folgenden Komponenten:

Vorlesung:
02.06.2026

- Σ ist ein Alphabet, dessen Elemente wir *Terminalsymbole* nennen.
- N ist eine endliche Menge, $N \cap \Sigma = \emptyset$, deren Elemente wir *Nichtterminalsymbole* oder *Variablen* nennen.
- $P \subseteq (N \cup \Sigma)^* N (N \cup \Sigma)^* \times (N \cup \Sigma)^*$ ist eine endliche Relation, deren Elemente wir *Regeln* oder *Produktionen* nennen.
- $S \in N$ ist ein Nichtterminalsymbol, das *Startsymbol*.

◇

Bsp. 3.1: $\mathcal{G} = (\Sigma, N, P, S)$ mit⁷

$$\begin{aligned}\Sigma &= \{0, 1\} \\ N &= \{S\} \\ P &= \{S \rightarrow 1S0S \\ &\quad, S \rightarrow 0S1S \\ &\quad, S \rightarrow \varepsilon\}\end{aligned}$$

Def. 3.2 (Ableitungsrelation, Ableitung, Sprache einer Grammatik): Die *Ableitungsrelation*⁸ zu $\mathcal{G} = (\Sigma, N, P, S)$ ist $\cdot \vdash_{\mathcal{G}} \cdot \subseteq (N \cup \Sigma)^* \times (N \cup \Sigma)^*$ definiert durch

$$\frac{\alpha \rightarrow \beta \in P}{\gamma \cdot \alpha \cdot \delta \vdash_{\mathcal{G}} \gamma \cdot \beta \cdot \delta}$$

Die *Ableitung von β aus α* , geschrieben $\alpha \vdash_{\mathcal{G}}^* \beta$ ist induktiv definiert als die reflexive, transitive Hülle von „ $\vdash_{\mathcal{G}}$ “:

$$\frac{}{\alpha \vdash_{\mathcal{G}}^* \alpha} \qquad \frac{\alpha \vdash_{\mathcal{G}} \beta \quad \beta \vdash_{\mathcal{G}}^* \gamma}{\alpha \vdash_{\mathcal{G}}^* \gamma}$$

Falls $\alpha \vdash_{\mathcal{G}}^* \beta$, so ist β eine *Satzform von $\mathcal{G}$* . Oft wird die Ableitung durch eine Liste von Satzformen $\alpha_0, \dots, \alpha_n$ angegeben mit $\alpha_i \vdash_{\mathcal{G}} \alpha_{i+1}$ für $0 \leq i < n$.

Ein Wort $w \in \Sigma^*$ wird von $\mathcal{G}$ *erzeugt*, wenn $S \vdash_{\mathcal{G}}^* w$ gilt. Die von $\mathcal{G}$ *erzeugte Sprache* ist definiert als:

$$L(\mathcal{G}) = \{w \in \Sigma^* \mid S \vdash_{\mathcal{G}}^* w\}$$

Wir nennen zwei Grammatiken *äquivalent*, falls sie die gleiche Sprache erzeugen.

◇

⁷ P ist eine „normale“ binäre Relation, doch wir verwenden statt „ $(x, y) \in P$ “ meist „ $x \rightarrow y$ “, also einen Pfeil und Infix-Notation, um die Lesbarkeit zu erhöhen.

⁸ Analog zur Relation P verwenden wir auch für die Ableitungsrelation wieder Infix-Notation, also $\alpha \vdash \beta$ statt $(\alpha, \beta) \in \vdash$.

Bsp. 3.2: Wir betrachten nochmal die Grammatik aus Bsp. 3.1. Es gilt: $1001 \in L(\mathcal{G})$.

$$S \vdash_{\mathcal{G}} 1S0S \vdash_{\mathcal{G}} 10S \vdash_{\mathcal{G}} 100S1S \vdash_{\mathcal{G}} 100S1 \vdash_{\mathcal{G}} 1001$$

Es gilt: $L(\mathcal{G})$ ist die Sprache der Wörter über $\{0, 1\}$, die gleich viele Nullen wie Einsen haben:

$$L = \{w \in \Sigma^* \mid \#_0(w) = \#_1(w)\}$$

Die Funktion $\#_a(w)$ berechnet hierbei die Anzahl der Vorkommen von $a \in \{0, 1\}$ in w .

Dass $L(\mathcal{G}) \subseteq L$, lässt sich mittels Induktion über die Ableitung $S \vdash_{\mathcal{G}}^* w$ zeigen. Der Beweis wird als Übung dem Leser überlassen.

Wir zeigen $L \subseteq L(\mathcal{G})$, d.h. $\forall w \in \Sigma^*. w \in L \Rightarrow w \in L(\mathcal{G})$. Hierzu definieren wir eine Hilfsfunktion $d : \Sigma^* \rightarrow \mathbb{Z}$:

$$d(\varepsilon) = 0 \qquad d(1w) = d(w) + 1 \qquad d(0w) = d(w) - 1$$

Mittels Induktion über u zeige, dass $u \in L \Leftrightarrow d(u) = 0$ und $d(u \cdot v) = d(u) + d(v)$.

Wir betrachten nun die folgende Aussage

$$P(n) = \forall w. |w| = n \Rightarrow w \in L \Rightarrow w \in L(\mathcal{G}).$$

und beweisen sie mit Hilfe des Beweisprinzips der vollständigen (bzw. starken) Induktion:

$$\frac{\forall n. (\forall j. j < n \Rightarrow P(j)) \Rightarrow P(n)}{\forall n. P(n)}$$

Die Ausformulierung des Beweises erfolgt in der Übung.

Bsp. 3.3: $\mathcal{G} = (\Sigma, N, P, S)$ mit

$$\Sigma = \{a, b, c\}$$

$$N = \{S, B, C\}$$

$$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$$

Es gilt z.B. $aaabbbccc \in L(\mathcal{G})$.

Außerdem gilt $L(\mathcal{G}) = \{a^n b^n c^n \mid n \geq 1\}$. (Ohne Beweis)

Die Chomsky-Hierarchie teilt die Grammatiken in vier Typen unterschiedlicher Mächtigkeit ein.

Def. 3.3 (Chomsky-Hierarchie):

- Jede Grammatik ist eine *Typ-0-Grammatik*.
- Eine Grammatik ist *Typ-1* oder *kontextsensitiv*, falls alle Regeln expansiv sind, d.h., für alle Regeln $\alpha \rightarrow \beta \in P$ ist $|\alpha| \leq |\beta|$. Ausnahme: falls S nicht in einer rechten Regelseite auftritt, dann ist $S \rightarrow \varepsilon$ erlaubt.
- Eine Grammatik heißt *Typ-2* oder *kontextfrei*, falls alle Regeln die Form $A \rightarrow \alpha$ mit $A \in N$ und $\alpha \in (N \cup \Sigma)^*$ haben.
- Eine Grammatik heißt *Typ-3* oder *regulär*, falls alle Regeln die folgende Form haben:

$$\begin{array}{l} A \rightarrow w \quad w \in \Sigma^*, A \in N \\ \text{oder} \quad A \rightarrow aB \quad a \in \Sigma, A, B \in N \end{array}$$

Eine Sprache heißt *Typ- i -Sprache*, falls es eine *Typ- i -Grammatik* für sie gibt. ◇

Beobachtung 3.1: Jede *Typ- $(i + 1)$ -Sprache* ist auch eine *Typ- i -Sprache*.

- Jede *Typ-3-Grammatik* ist eine *Typ-2-Grammatik*.
- Wir werden im folgenden Unterkapitel zeigen, dass jede *Typ-2-Grammatik* in eine äquivalente ε -freie⁹ *Typ-2-Grammatik* transformiert werden kann. Diese erfüllt dann auch alle Bedingungen einer *Typ-1-Grammatik*.
- Jede *Typ-1-Grammatik* ist auch eine *Typ-0-Grammatik*.

Im Lauf der Vorlesung werden wir die folgende Aussage zeigen:

Sei CH_i die Menge der *Typ- i -Sprachen*; dann gilt: $CH_3 \subsetneq CH_2 \subsetneq CH_1 \subsetneq CH_0$.

3.1 Kontextfreie Sprachen

Die Sprache aus Bsp. 3.1 ist kontextfrei. Weitere Beispiele sind:

Vorlesung:
skipped

Bsp. 3.4: Arithmetische Ausdrücke ohne Klammern: $\mathcal{G} = (\{E\}, \{a, +, *\}, P, E)$ mit

$$P = \{E \rightarrow a \mid E + E \mid E * E\}.$$
¹⁰

⁹Auf keiner rechten Seite steht ε ; Ausnahme: Startsymbol S ; vgl. Elimination von ε -Produktionen.

¹⁰Notation: Wenn wir mehrere Regeln mit gleicher linker Seite wie z.B. „ $A \rightarrow \alpha$ “ und „ $A \rightarrow \beta$ “ haben, dann dürfen wir diese auch mit Hilfe eines senkrechten Striches als „ $A \rightarrow \alpha \mid \beta$ “ schreiben.

Bsp. 3.5: Arithmetische Ausdrücke mit Klammern: $\mathcal{G} = (\{E\}, \{a, +, *, (,)\}, P, E)$ mit

$$P = \{E \rightarrow a \mid (E + E) \mid (E * E)\}.$$

Bsp. 3.6: Syntax von Programmiersprachen. Wir zeigen hier nur exemplarisch Produktionen für einige typische Bestandteile einer Programmiersprache.¹¹ Wörter in spitzen Klammern sind hier Nichtterminalsymbole. Terminalsymbole sind grau unterlegt.

$$\begin{aligned} \langle \text{Stmt} \rangle &\rightarrow \langle \text{Var} \rangle \text{ = } \langle \text{Exp} \rangle \\ &\mid \langle \text{Stmt} \rangle \text{ ; } \langle \text{Stmt} \rangle \\ &\mid \text{ if } (\langle \text{Exp} \rangle) \langle \text{Stmt} \rangle \text{ else } \langle \text{Stmt} \rangle \\ &\mid \text{ while } (\langle \text{Exp} \rangle) \langle \text{Stmt} \rangle \end{aligned}$$

Für den arithmetischen Ausdruck ohne Klammern $a + a$ gibt es in der Grammatik aus Bsp. 3.4 zwei verschiedene Ableitungen:

- $E \vdash_{\mathcal{G}} E + E \vdash_{\mathcal{G}} a + E \vdash_{\mathcal{G}} a + a$
- $E \vdash_{\mathcal{G}} E + E \vdash_{\mathcal{G}} E + a \vdash_{\mathcal{G}} a + a$

Beide Ableitungen unterscheiden sich nur durch die Reihenfolge, in der Variablen ersetzt werden. Die folgende Definition erlaubt es uns, beide Ableitungen durch das gleiche Objekt, den sogenannten Ableitungsbaum, darzustellen.

Vorlesung:
03.06.2026

Def. 3.4 (Ableitungsbaum): Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine kontextfreie Grammatik und $A \in N$. Wir definieren die Menge der *in A beginnenden Ableitungsbäume von $\mathcal{G}$* , $\text{Abl}(A)$, als Menge von beschrifteten, geordneten Bäumen induktiv wie folgt:

Falls $\pi = A \rightarrow w_0 A_1 w_1 \dots A_n w_n \in P$ mit $A_i \in N$, $w_i \in \Sigma^*$, $0 \leq i \leq n$ und $\mathcal{T}_i \in \text{Abl}(A_i)$, dann ist

$$\begin{array}{c} \pi \\ \swarrow \quad \searrow \\ \mathcal{T}_1 \quad \dots \quad \mathcal{T}_n \end{array} \in \text{Abl}(A)$$

Da es manchmal aufwendig ist, Bäume zu zeichnen, verwenden wir alternativ die folgende Notation: $\pi(\mathcal{T}_1, \dots, \mathcal{T}_n)$

¹¹Sie finden am Ende der Java Language Specification <https://docs.oracle.com/javase/specs/> auf ca. 25 Seiten die kontextfreie Grammatik für diese Programmiersprache.

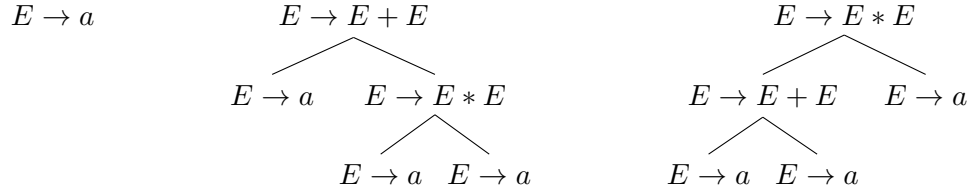
Das *abgeleitete Wort* zu einem $\mathcal{T} \in \text{Abl}(A)$, $Y : \text{Abl}(A) \rightarrow \Sigma^*$ ¹², ist definiert durch

$$Y \left(\begin{array}{c} \pi \\ \swarrow \quad \searrow \\ \mathcal{T}_1 \quad \dots \quad \mathcal{T}_n \end{array} \right) = w_0 Y(\mathcal{T}_1) w_1 \dots Y(\mathcal{T}_n) w_n$$

wobei $\pi = A \rightarrow w_0 A_1 w_1 \dots A_n w_n \in P$. ◇

Bemerkung: Nach dieser Definition gibt es für jede Regel π , bei der nur Terminalsymbole auf der rechten Seite vorkommen, einen Ableitungsbaum, der die ein-elementige Knotenmenge $\{\pi\}$ und eine leere Menge von Kanten hat.

Bsp. 3.7: Für die Grammatik aus Bsp. 3.4 sind die folgenden drei Beispiele in E beginnende Ableitungsbäume.



Lemma 3.1: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine kontextfreie Grammatik.

$$w \in L(\mathcal{G}) \quad \text{gdw} \quad \exists \mathcal{T} \in \text{Abl}(S) \text{ mit } Y(\mathcal{T}) = w$$

(Ohne Beweis.)

3.2 Die Chomsky-Normalform für kontextfreie Sprachen

Wir lernen in diesem Unterkapitel eine spezielle Form von kontextfreien Grammatiken (Chomsky-Normalform, CNF) kennen, für die gilt:

- Es gibt einen einfachen Algorithmus für das Wortproblem ($w \in L(\mathcal{G})?$).
- Jede kontextfreie Grammatik lässt sich effektiv in CNF transformieren.

Def. 3.5 (CFG in CNF): Eine kontextfreie Grammatik (CFG) $\mathcal{G} = (\Sigma, N, P, S)$ ist in Chomsky-Normalform (CNF), falls jede Regel die Form $A \rightarrow a$, $A \rightarrow BC$, oder $S \rightarrow \varepsilon$ hat, wobei $A, B, C \in N$, $a \in \Sigma$. Falls $S \rightarrow \varepsilon \in P$, dann darf S auf keiner rechten Seite einer Regel vorkommen. ◇

¹²Wir verwenden $Y(\mathcal{T})$, um an das englische Wort „yield“ zu erinnern.

Wir stellen im Folgenden vier Transformationen (SEP, BIN, DEL, UNIT) vor und wollen dabei jeweils den Zeitaufwand und die Größe der resultierenden CFG analysieren.

Dafür definieren wir die Größe einer Grammatik wie folgt.

$$|\mathcal{G}| = \sum_{A \in N} \sum_{A \rightarrow \alpha \in P} |A\alpha|$$

Die Größe ist also genau die Anzahl an Zeichen aus $\Sigma \cup N$, die man benötigt, um die Regeln der Grammatik aufzuschreiben.

Def. 3.6: Eine CFG heißt *separiert*, wenn jede Regel eine der folgenden Formen hat.

$$\begin{aligned} A &\rightarrow A_1 \dots A_n && \text{für } A \in N, A_i \in N, n \geq 0 \\ A &\rightarrow a && \text{für } A \in N, a \in \Sigma \end{aligned} \quad \diamond$$

Lemma 3.2 (SEP): Zu jeder CFG gibt es eine äquivalente separierte CFG.

BEWEIS: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine CFG. Konstruiere $\mathcal{G}' = (\Sigma, N', P', S)$ mit

- $N' = N \dot{\cup} \{Y_a \mid a \in \Sigma\}$ ¹³
- $P' = \{Y_a \rightarrow a \mid a \in \Sigma\} \cup \{A \rightarrow \beta[a \rightarrow Y_a] \mid A \rightarrow \beta \in P\}$.

Die Schreibweise $\beta[a \rightarrow Y_a]$ bedeutet hier, dass alle $a \in \Sigma$, die in β vorkommen, durch Y_a ersetzt werden.

Offenbar gilt $L(\mathcal{G}) = L(\mathcal{G}')$ und $\mathcal{G}'$ ist separiert (ohne Beweis). □

Bemerkung: SEP berechnet in $O(|\mathcal{G}|^2)$ eine Grammatik der Größe $O(|\mathcal{G}|)$.

- Laufe einmal über alle Regeln und ersetze jeweils $a \mapsto Y_a$.
- Für jedes neue Nichtterminalsymbol fügen wir eine Regel der Größe 2 hinzu.

Lemma 3.3 (BIN): Zu jeder CFG gibt es eine äquivalente CFG, bei der für alle Regeln $A \rightarrow \alpha$ gilt, dass $|\alpha| \leq 2$.

BEWEIS: Ersetze jede Regel der Form

$$A \rightarrow X_1 X_2 \dots X_n, \quad n \geq 3$$

¹³ „ $\dot{\cup}$ “ steht für „disjunkte Vereinigung“

mit $X_i \in N \cup \Sigma$, $1 \leq i \leq n$, durch die Regeln

$$\begin{aligned} A &\rightarrow X_1 \langle X_2 \dots X_n \rangle \\ \langle X_2 \dots X_n \rangle &\rightarrow X_2 \langle X_3 \dots X_n \rangle \\ &\vdots \\ \langle X_{n-1} X_n \rangle &\rightarrow X_{n-1} X_n \end{aligned}$$

Dabei sind $\langle X_2 \dots X_n \rangle, \dots, \langle X_{n-1} \dots X_n \rangle$ neue Nichtterminalsymbole. $\square$

Bemerkung: BIN berechnet in $O(|\mathcal{G}|)$ eine Grammatik der Größe $O(|\mathcal{G}|)$.

- Laufe einmal über alle Regeln.
- Für jede Regel der Größe $n + 1$, $n \geq 3$ fügen wir höchstens $n - 1$ neue Regeln der Größe 3 hinzu.

Wir definieren die Menge der Nichtterminalsymbole, aus denen das leere Wort abgeleitet werden kann, formal wie folgt:

Def. 3.7: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine CFG. Definiere die Menge $\text{Nullable}(\mathcal{G}) \subseteq N$ als

$$\text{Nullable}(\mathcal{G}) = \{A \in N \mid A \vdash_{\mathcal{G}}^* \varepsilon\}. \quad \diamond$$

Satz 3.4: Es gibt einen Algorithmus, der $\text{Nullable}(\mathcal{G})$ in $O(|\mathcal{G}|^3)$ berechnet.

BEWEIS: Definiere M_i als die Menge der Nichtterminalsymbole, aus denen sich ε mit einem Ableitungsbaum der Höhe $< i$ ableiten lässt:¹⁴

$$\begin{aligned} M_0 &= \emptyset \\ M_{i+1} &= \{A \mid A \rightarrow \alpha \in P \text{ und } \alpha \in M_i^*\} \end{aligned}$$

Es gilt für alle $i \in \mathbb{N}$, dass $M_i \subseteq M_{i+1} \subseteq N$. Da $|N|$ endlich ist, existiert ein $m \in \mathbb{N}$, sodass

$$M_m = M_{m+1} = \bigcup_{i \in \mathbb{N}} M_i$$

Wir können daher $\bigcup_{i \in \mathbb{N}} M_i$ mit dem Algorithmus in Abb. 5 in $O(|\mathcal{G}|^3)$ berechnen.

Wir zeigen nun, dass $\text{Nullable}(\mathcal{G}) = \bigcup_{i \in \mathbb{N}} M_i$.

„ $\supseteq$ “ Wir zeigen mittels Induktion, dass

$$\forall i \in \mathbb{N} : M_i \subseteq \text{Nullable}(\mathcal{G})$$

¹⁴Im Folgenden ist mit $*$ bei M_i^* und M^* der Kleene-Stern gemeint.

```

M = {}
repeat
  changed = false
  Mold = M
  foreach A → α ∈ P:
    if (A ∉ M ∧ α ∈ Mold*):
      M = M ∪ {A}
      changed = true
until not changed
return M

```

Abb. 5: Berechnung von Nullable($\mathcal{G}$)**IA** $i = 0$: $M_0 = \emptyset \subseteq \text{Nullable}(\mathcal{G})$ **IS** $i \rightsquigarrow i + 1$

Wenn $A \in M_{i+1}$, dann existiert $A \rightarrow A_1 \dots A_n \in P$ mit $A_j \in M_i$ für alle $1 \leq j \leq n$.
 Nach IV existieren Ableitungsbäume $\mathcal{T}_j$ für $A_j \vdash_{\mathcal{G}}^* \varepsilon$, die zu einem Ableitungsbaum von A nach ε kombiniert werden können:

$$\mathcal{T} = \begin{cases} A \rightarrow A_1 \dots A_n \\ \swarrow \quad \searrow \\ \mathcal{T}_1 \quad \dots \quad \mathcal{T}_n \end{cases}$$

Wegen $Y(\mathcal{T}_j) = \varepsilon$, $1 \leq j \leq n$, gilt auch $Y(\mathcal{T}) = \varepsilon$. Also gilt $A \in \text{Nullable}(\mathcal{G})$.

„ $\subseteq$ “

Wir zeigen mittels Induktion über einen Ableitungsbaum von $A \vdash_{\mathcal{G}}^* \varepsilon$, dass $\exists i. A \in M_i$.

IA $A \rightarrow \varepsilon$ ist eine Produktion. Also ist $A \in M_1$.**IS** Die Wurzel des Ableitungsbaums ist mit $A \rightarrow A_1 \dots A_n$ beschriftet und die Teilbäume entsprechen den Ableitungen $A_j \vdash_{\mathcal{G}}^* \varepsilon$, $1 \leq j \leq n$.

Die IV liefert für jeden Teilbaum ein i_j , sodass $A_j \in M_{i_j}$. Also gilt für $i = \max\{i_1, \dots, i_n\}$, dass $A_j \in M_i$. Daher ist $A \in M_{i+1}$. □

Vorlesung:
09.06.2026

Korollar 3.5: Man kann zu einer CFG $\mathcal{G}$ entscheiden, ob $\varepsilon \in L(\mathcal{G})$. ◇BEWEIS: Prüfe, ob $S \in \text{Nullable}(\mathcal{G})$. □

Lemma 3.6 (DEL): Zu jeder CFG $\mathcal{G} = (\Sigma, N, P, S)$ gibt es eine äquivalente CFG $\mathcal{G}' = (\Sigma, N', P', S')$, bei der die einzige ε -Regel $S' \rightarrow \varepsilon$ ist (falls $\varepsilon \in L(\mathcal{G})$) und bei der S' auf keiner rechten Seite einer Regel vorkommt.

BEWEIS:

1. Definiere $\mathcal{G}'$ wie folgt:
 - Erweitere $\mathcal{G}$ um ein neues Startsymbol $S' \notin N$ mit $S' \rightarrow S$ als neue Regel. Dieser Schritt stellt sicher, dass S' auf keiner rechten Regelseite vorkommt.
 - Wende erst SEP, dann BIN an. Nun hat jede rechte Regelseite von $\mathcal{G}'$ die Form $a \in \Sigma$ oder ε oder A oder BC .
2. Für alle Regeln $A \rightarrow BC \in P$:
 - Falls $B \in \text{Nullable}(\mathcal{G}')$ füge $A \rightarrow C$ hinzu.
 - Falls $C \in \text{Nullable}(\mathcal{G}')$ füge $A \rightarrow B$ hinzu.
3. Falls $S \in \text{Nullable}(\mathcal{G}')$, füge $S' \rightarrow \varepsilon$ hinzu.
4. Entferne alle Regeln $A \rightarrow \varepsilon$ für $A \neq S'$. □

Bemerkung: DEL kann in $O(|\mathcal{G}|^3)$ berechnet werden und die Größe der resultierenden Grammatik ist $O(|\mathcal{G}|)$.

- Die höchsten Kosten entstehen beim Berechnen von Nullable.
- In Schritt 3 verdoppelt sich die Anzahl der Regeln höchstens.

Def. 3.8: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine Grammatik. Eine Regel der Form $A \rightarrow B \in P$ mit $A, B \in N$ heißt *Kettenregel*. ◇

Lemma 3.7 (UNIT): Es gibt einen Algorithmus der für eine CFG $\mathcal{G}$ in $O(|\mathcal{G}|^3)$ eine äquivalente CFG $\mathcal{G}'$ ohne Kettenregeln mit $|\mathcal{G}'| \in O(|\mathcal{G}|^2)$ berechnet.

BEWEIS: Beweisskizze zum Algorithmus in Abb. 6.

- Zur Äquivalenz:

Wir zeigen, dass jeder Schritt, der die CFG ändert, eine äquivalente CFG liefert. Dazu zeigen wir, dass jedes Wort, das in der alten CFG abgeleitet werden konnte, auch in der neuen CFG abgeleitet werden kann und umgekehrt. Für Schritt 6 gilt die Äquivalenz sogar für jede Iteration der mittleren for-Schleife.

- Zur Kettenregelfreiheit:

In Schritt 3 wird mindestens ein Kettenregelzykel entfernt. Da eine Grammatik nur endlich viele Regeln hat, wird nach spätestens $|\mathcal{G}|$ Anwendungen von Schritt 3 der Schritt 5, in dem alle Zyklen entfernt sind, erreicht.

In der mittleren for-Schleife von Schritt 6 gilt aufgrund der topologischen Sortierung immer $i < k$. Die äußere Schleife hat die Invariante, dass am Ende jedes Durchlaufs für $n \geq i \geq j$ keine Kettenregel $A_i \rightarrow \dots$ in P' existiert. Beim Verlassen der Schleife ist $j = 1$ und es existieren überhaupt keine Kettenregeln mehr.

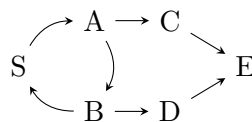
- Zu Laufzeit und Größe des Resultates:

Vor Schritt 6 wird die Grammatik nie vergrößert (wichtig sowohl für Laufzeit als auch für Größe!) und die Kosten übersteigen $O(|\mathcal{G}|^3)$ nicht. Die Anzahl der Schleifendurchläufe ist jeweils durch $|\mathcal{G}|$ beschränkt. Seien $B \rightarrow \alpha_1, \dots, B \rightarrow \alpha_n$ alle neuen Regeln, die für eine Variable $B \in N$ in Schritt 6 hinzugefügt werden. Dann gilt, dass die Summe der Größe der α_i die Größe der Grammatik $|\mathcal{G}|$ nicht übersteigt. Somit ist die Größe des Resultates durch $|\mathcal{G}|^2$ beschränkt. $\square$

Bsp. 3.8: Sei $\Sigma = \{0, 1\}$. Betrachte die CFG $\mathcal{G} = (\Sigma, \{S, A, B, C, D, E\}, P, S)$ deren Regeln P wie folgt definiert sind.

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow B \mid 0A1B \mid C \\ B &\rightarrow S \mid 1B0A \mid D \\ C &\rightarrow E \\ D &\rightarrow E \\ E &\rightarrow \epsilon \end{aligned}$$

Nach der erstmaligen Anwendung von Schritt 2 erhalten wir den folgenden gerichteten Graphen.



¹⁵Topologisches Sortieren ist ein aus der Informatik 2 Vorlesung bekannter Begriff und bezeichnet das Erweitern einer gegebenen partiellen Ordnung zu einer totalen Ordnung.

Eingabe: CFG $\mathcal{G} = (\Sigma, N, P, S)$

Ausgabe: CFG $\mathcal{G}' = (\Sigma, N', P', S)$, die äquivalent zu $\mathcal{G}$ ist, aber keine Kettenregeln enthält.

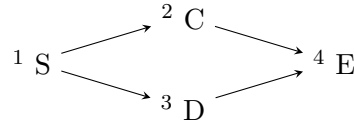
1. Setze $N' = N$ und $P' = P$.
2. Betrachte den gerichteten Graphen G , dessen Knotenmenge N und Kantenmenge $\{(A, B) \mid A \rightarrow B \in P'\}$ ist. (Es gibt also genau eine Kante pro Kettenregel.)
3. Suche, z.B. mittels Tiefensuche, einen Zyklus in G . Wenn kein Zyklus gefunden wurde, fahre mit Schritt 5 fort.
4. Wurde ein Zyklus $A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_k \rightarrow A_1$ gefunden, dann sei o.B.d.A. $A_1 = S$ falls S im Zyklus enthalten. Ersetze nun in P' alle Vorkommen von A_j mit $j > 1$ durch A_1 (auf linker *und* rechter Regelseite). Entferne dann alle Regeln der Form $A \rightarrow A$. Fahre fort mit Schritt 2.
5. Sortiere die Knoten des nun azyklischen Graphen G topologisch¹⁵ als $A_1, \dots, A_n$, so dass A_n auf keiner linken Seite einer Kettenregel vorkommt.
6. **for** $j = n - 1, \dots, 1$
 - for each** $A_j \rightarrow A_k \in P'$
 - entferne $A_j \rightarrow A_k$ aus P'
 - for each** $A_k \rightarrow \alpha \in P'$
 - füge $A_j \rightarrow \alpha$ zu P'

Abb. 6: Algorithmus UNIT

In Schritt 2 wählen wir den Zykel $S \rightarrow A \rightarrow B \rightarrow S$ und erhalten für P' die folgenden Regeln.

$$\begin{aligned} S &\rightarrow C \mid D \mid 0S1S \mid 1S0S \\ C &\rightarrow E \\ D &\rightarrow E \\ E &\rightarrow \varepsilon \end{aligned}$$

Nach dem zweiten Anwenden von Schritt 2 erhalten wir den folgenden gerichteten Graphen und wählen in Schritt 5 die durch die Zahlen angedeutete topologische Sortierung.



Am Ende von Schritt 6 sieht die Regelmengende P' wie folgt aus¹⁶.

$$\begin{aligned} S &\rightarrow \varepsilon \mid 0S1S \mid 1S0S \\ C &\rightarrow \varepsilon \\ D &\rightarrow \varepsilon \\ E &\rightarrow \varepsilon \end{aligned}$$

Satz 3.8: Es gibt einen Algorithmus der für eine CFG $\mathcal{G}$ in $O(|\mathcal{G}|^3)$ Rechenschritten eine äquivalente CFG $\mathcal{G}'$ in CNF mit $|\mathcal{G}'| \in O(|\mathcal{G}|^2)$ berechnet.

BEWEIS: (Beweisskizze) Ein Algorithmus der diese Eigenschaften erfüllt ist das Hintereinanderschalten von SEP, BIN, DEL und UNIT.¹⁷

- Zur Korrektheit:

Wir zeigen dass jedes dieser vier Verfahren die vom vorherigen Verfahren etablierte Eigenschaft nicht zerstört und dass die vier etablierten Eigenschaften für die CNF hinreichend sind.

¹⁶Das Beispiel zeigt dass das Resultat des Algorithmus völlig unnötige Regeln enthalten kann.

¹⁷Es genügt sogar nur DEL und dann UNIT anzuwenden, da DEL schon SEP und BIN ausführt.

- Zu Laufzeit und Größe des Resultates:

Die ersten drei Verfahren erhöhen die Größe der Grammatik nur linear und die Laufzeiten sind durch $O(|\mathcal{G}|^3)$ beschränkt. Für UNIT sind die in diesem Satz genannten Kosten eine obere Schranke. $\square$

Lemma 3.9: Die Menge der Typ-2-Sprachen ist eine Teilmenge der Typ-1-Sprachen.

BEWEIS: Zu jeder Typ-2-Grammatik existiert eine äquivalente ε -freie Typ-2-Grammatik. Diese ist nach Definition auch eine Typ-1-Grammatik. $\square$

3.3 Wortproblem und Leerheitsproblem für kontextfreie Sprachen

Satz 3.10: Es gibt einen Algorithmus, der für eine CFG $\mathcal{G}$ in CNF und ein Wort w in $O(|w|^3 \cdot |\mathcal{G}|)$ Schritten und mit $O(|w|^2 \cdot |\mathcal{G}|)$ Speicherplatz entscheidet ob $w \in L(\mathcal{G})$ gilt.

BEWEIS (CYK-Algorithmus): Der CYK-Algorithmus¹⁸ in Abb. 7 erfüllt diese Eigenschaften. Wir wollten zunächst die Idee des Algorithmus erklären.

Sei $w = a_1 \dots a_n \in \Sigma^*$. Definiere für $1 \leq i \leq j \leq n$:

$$M_{ij} = \{A \in N \mid A \vdash_{\mathcal{G}}^* a_i \dots a_j\} \quad (\text{EntryDeriv})$$

Es gilt: $w \in L(\mathcal{G})$ genau dann, wenn $S \vdash_{\mathcal{G}}^* w$ genau dann, wenn $S \in M_{1n}$.

Es gilt für $i = j$

$$M_{ii} = \{A \mid A \rightarrow a_i \in P\}$$

Es gilt für $i < j$

$$\begin{aligned} M_{ij} &= \{A \mid A \rightarrow BC \in P \text{ und } BC \vdash_{\mathcal{G}}^* a_i \dots a_j\} \\ &= \{A \mid A \rightarrow BC \in P \text{ und } \exists k \in \{i, \dots, j-1\} \text{ sodass } B \vdash_{\mathcal{G}}^* a_i \dots a_k \text{ und } C \vdash_{\mathcal{G}}^* a_{k+1} \dots a_j\} \\ &= \{A \mid A \rightarrow BC \in P \text{ und } \exists k \in \{i, \dots, j-1\} \text{ sodass } B \in M_{ik} \text{ und } C \in M_{(k+1)j}\} \end{aligned}$$

Beobachtung: Der Wert von M_{ij} hängt nur von $M_{i'j'}$ mit $i' > i, j' \leq j$ oder $i' \geq i, j' < j$ ab. Wir können die M_{ij} also mit dem Verfahren aus Abb. 7 berechnen.

Vorlesung:
10.06.2026

¹⁸Benannt nach Cocke, Younger und Kasami, die den Algorithmus unabhängig voneinander entdeckt haben. Manchmal auch YCK.

Eingaben: CFG $\mathcal{G} = (\Sigma, N, P, S)$ in CNF und Wort $a_1 \dots a_n \in \Sigma^*$

Ausgabe: $a_1 \dots a_n \in L(\mathcal{G})$?

Sei M eine $n \times n$ -Matrix mit (initial) $M_{ij} = \emptyset$ für alle i, j // $O(n^2)$

```

for  $i = 1, \dots, n$  do //  $O(|\mathcal{G}|) \cdot O(n)$ 
     $M_{ii} = \{A \mid A \rightarrow a_i\}$ 
for  $i = n - 1, \dots, 1$  do
    for  $j = i + 1, \dots, n$  do
        for  $k = i, \dots, j - 1$  do //  $O(|\mathcal{G}|) \cdot O(n^3)$ 
             $M_{ij} = M_{ij} \cup \{A \mid A \rightarrow BC, B \in M_{ik}, C \in M_{(k+1)j}\}$ 
return  $S \in M_{1n}$ 

```

Abb. 7: CYK Algorithmus

Skizze der verbleibenden Beweisschritte.

Zur Korrektheit: Zeige mittels einer verschachtelten Induktion über Zeilen und Spalten, dass am Ende der zweiten for-Schleife für alle $M_{i'j'}$ mit $i' \in \{i, \dots, n - 1\}$ und $j' \in \{i + 1, \dots, j\}$ die oben definierte Gleichung (EntryDeriv) gilt.

Zur Laufzeitabschätzung: Die Iterationen jeder for-Schleife sind durch die Länge des Wortes $|w|$ beschränkt. In der innersten Schleife genügt es ein mal über die Regeln der Grammatik zu iterieren.

Zum Speicherplatz: Wir benötigen weniger als $|w|^2$ Matrixeinträge und jeder Eintrag enthält höchstens $|N|$ Variablen. $\square$

Bsp. 3.9: Sei $\Sigma = \{a, b, c\}$. Betrachte das Wort $w = aaabbbcc$ und die CFG $\mathcal{G} = (\Sigma, \{S, X, Y\}, P, S)$ ¹⁹ deren Regeln P wie folgt definiert sind.

$$\begin{aligned}
 S &\rightarrow XY \\
 X &\rightarrow ab \mid aXb \\
 Y &\rightarrow c \mid cY
 \end{aligned}$$

Damit wir den CYK Algorithmus anwenden können transformieren wir $\mathcal{G}$ in eine äquivalente Grammatik in CNF. Z.B. in die CFG $\mathcal{G}' = (\Sigma, \{S, X, Y, Z, A, B, C\}, P', S)$, deren

¹⁹Es gilt $L(\mathcal{G}) = \{a^n b^n c^m \mid n, m \geq 1\}$ aber das bestimmen der erzeugten Sprache ist für das Entscheiden des Wortproblems natürlich nicht erforderlich.

Regeln P' wie folgt definiert sind.

$$\begin{aligned} S &\rightarrow XY \\ X &\rightarrow AB \mid AZ \\ Z &\rightarrow XB \\ Y &\rightarrow CY \mid c \\ A &\rightarrow a, B \rightarrow b, C \rightarrow c \end{aligned}$$

Wir wenden den CYK Algorithmus an und erhalten die folgende Matrix.

	A	•	•	•	•	X_6	S_7	S_8
a	A	•	•	X_4	Z_5	•	•	
	a	A	X_2	Z_3	•	•	•	
		a	B	•	•	•	•	
			b	B	•	•	•	
				b	B	•	•	
					b	C, Y	Y_1	
						c	C, Y	
							c	

Zellen, die mit „•“ markiert sind, sind leer. Die Reihenfolge, in der die nicht-diagonalen, nicht leeren Zellen eingetragen wurden, ist mit kleinen Zahlen markiert. Beispielsweise wurde $M_{35} = \{Z\}$ mit Zahl „3“ nach $M_{78} = \{Y\}$ mit Zahl „1“ eingetragen.

Da $S \in M_{1n}$, gilt $w \in L(\mathcal{G}')$.

Korollar 3.11: Es gibt einen Algorithmus, der für eine CFG $\mathcal{G}$ und ein Wort w in $O(|w|^3 \cdot |\mathcal{G}|^2)$ Schritten und mit $O(|w|^2 \cdot |\mathcal{G}|^2)$ Speicherplatz entscheidet, ob $w \in L(\mathcal{G})$. $\diamond$

BEWEIS: Nach Satz 3.8 kann eine CFG in CNF konvertiert werden, sodass deren Größe höchstens quadratisch wächst. Wende anschließend den CYK Algorithmus auf das Resultat an. $\square$

Satz 3.12: Es gibt einen Algorithmus, der das Leerheitsproblem²⁰ für kontextfreie Grammatiken in $O(|\mathcal{G}|^2)$ entscheidet.

Def. 3.9: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine CFG. Ein Nichtterminalsymbol $A \in N$ ist *co-erreichbar*, falls es ein Wort $w \in \Sigma$ existiert, sodass $A \vdash_{\mathcal{G}}^* w$ gilt. $\diamond$

BEWEIS: Die M die Menge der co-erreichbaren Nichtterminale von $\mathcal{G}$ ist gegeben durch

$$M = \{A \mid A \vdash_{\mathcal{G}}^* w, w \in \Sigma^*\}.$$

Dann gilt: $L = \emptyset \Leftrightarrow S \notin M$.

²⁰Gefragt ist, ob $L(\mathcal{G}) = \{w \in \Sigma^* \mid S \vdash_{\mathcal{G}}^* w\} \stackrel{?}{=} \emptyset$.

Wir können M wie folgt berechnen:

$$M_0 = \emptyset$$

$$M_{i+1} = \{A \mid A \rightarrow \alpha \in P, \alpha \in (\Sigma \cup M_i)^*\}$$

Es gilt für alle $i \in \mathbb{N}$, $M_i \subseteq M_{i+1} \subseteq M$. Da $M \subseteq N$ endlich ist, gibt es ein n , sodass $M_n = M_{n+1}$ (hier ohne Beweis). Setze $M = M_n$.

Wir können daher M analog zu $\text{Nullable}(\mathcal{G})$ (siehe Satz 3.4) in $O(|\mathcal{G}|)$ berechnen. $\square$

Bem.: Wenn wir in einer Grammatik $\mathcal{G}$ ein nicht co-erreichbares Nichtterminalsymbol A und alle Regeln, bei denen A auf der linken oder rechten Regelseite auftritt, entfernen, ändert sich $L(\mathcal{G})$ nicht. Wir können eine gegebene Grammatik also optimieren, indem wir alle nicht co-erreichbaren Nichtterminalsymbole entfernen.

3.4 Das Pumping Lemma für kontextfreie Sprachen

Satz 3.13 (Pumping Lemma für CFL): Sei L eine kontextfreie Sprache. Dann gilt:

$$\begin{aligned} \exists n \in \mathbb{N}, n > 0 : \quad & \forall z \in L, |z| \geq n : \\ & \exists u, v, w, x, y \in \Sigma^* : \\ & z = uvwxy, |vwx| \leq n, |vx| \geq 1 \\ \text{und } \forall i \in \mathbb{N} : & uv^iwx^iy \in L \end{aligned}$$

Lemma 3.14: In jedem Binärbaum mit $\geq 2^k$ Blättern gibt es mindestens ein Blatt sodass der einen Pfad von der Wurzel zu diesem Blatt Länge $\geq k$ hat. (ohne Beweis)

BEWEIS (von Satz 3.13): ²¹ Sei $\mathcal{G} = (\Sigma, N, P, S)$ in CNF mit $L(\mathcal{G}) = L$. Wähle $n = 2^{|N|}$.

Betrachte den Ableitungsbaum $\mathcal{T} \in \text{Abl}(S)$ für ein Wort z mit $|z| \geq n$. $\mathcal{T}$ ist ein Binärbaum mit $|z| \geq n = 2^{|N|}$ Blättern. In einem solchen Binärbaum existiert nach Lemma 3.14 ein Pfad ζ der Länge $\geq |N|$. Auf ζ liegen $\geq |N| + 1$ Knoten. Jeder Knoten ist mit einer Regel beschriftet deren linke Seite ein Nichtterminalsymbol ist. Es muss also mindestens ein Nichtterminalsymbol A mehrmals als linke Seite einer Regel vorkommen.

Folge ζ rückwärts vom Blatt in Richtung Wurzel, bis sich eine linke Regelseite A das erste Mal wiederholt. Das geschieht nach $\leq |N|$ Schritten. Teile $z = uvwxy$ wie in Abb. 8a skizziert. Es gilt:

²¹Dieser Beweis wurde in der Vorlesung mit Hilfe von Abb. 8 erklärt aber nicht an die Tafel geschrieben.

- $|vx| \geq 1$, da ζ entweder durch den B -Nachfolger oder durch C -Nachfolger läuft. Angenommen, ζ verläuft durch den B -Nachfolger. Somit muss $C \vdash^* x$ gelten. In CNF ist $C \vdash^* \varepsilon$ nicht möglich und somit ist $|x| \geq 1$. Der Fall, dass ζ durch den C -Nachfolger verläuft, ist analog.
- $|vwx| \leq 2^{|N|} = n$, da der obere Knoten dessen linke Regelseite mit A beschriftet ist, höchstens $|N|$ Schritte von der Blattebene entfernt und $\mathcal{T}$ ein Binärbaum ist.
- Für die Aussage $\forall i \in \mathbb{N} : uv^iwx^iy \in L$ unterscheiden wir drei Fälle:
 - $i = 0$: Aus dem „unteren“ A haben wir w abgeleitet, aus dem oberen A haben wir vwx abgeleitet. Wir könnten also auch aus dem oberen A direkt w ableiten und erhalten $S \vdash^* uwy$. (Siehe Abb. 8b).
 - $i = 1$: Gilt trivialerweise, da $uv^1wx^1y = z$.
 - $i \geq 1$: Aus dem „oberen“ A haben wir vAx abgeleitet, aus dem unteren A haben wir w abgeleitet. Wir könnten also auch aus dem unteren A nochmal vAx ableiten. Wenn wir dies oft genug wiederholen, können wir jedes uv^iwx^iy mit $i \geq 2$ erzeugen (siehe auch Abb. 8c).

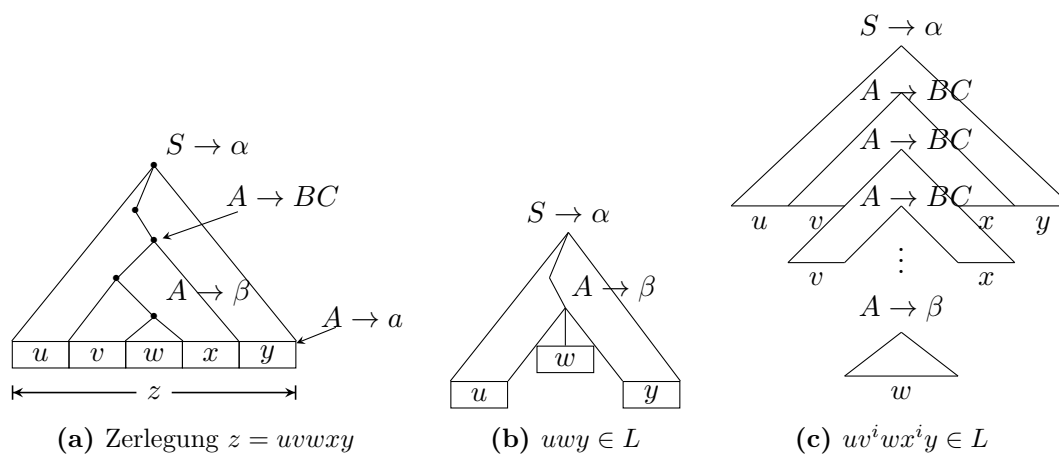


Abb. 8: Illustration des Argumentes aus dem Beweis von Satz 3.13

□

Vorlesung:
16.06.2026

Lemma 3.15: Die Sprache $L = \{a^n b^n c^n \mid n \geq 1\}$ ist nicht kontextfrei.

BEWEIS: Wir nehmen an, L wäre kontextfrei. Sei $n \in \mathbb{N}$ mit $n > 0$ die Konstante aus dem PL. Wir betrachten $z = a^n b^n c^n$. Sei $z = uvwxy$ eine Zerlegung mit $|vx| \geq 1$ und $|vwx| \leq n$.

Wegen $|vwx| \leq n$ trifft einer folgenden fünf Fälle zu.

- $vw x = a^j$ mit $j > 0$: Es gilt $|v^0 w x^0| < j$ und somit $uv^0 w x^0 y \notin L$, da die Anzahl der a nicht mit der Anzahl der b übereinstimmt.
- $vw x = a^k b^j$ mit $k > 0$ und $j > 0$.
 - Falls $v = a^{k'}$ und $x = b^{j'}$, dann gilt $uv^0 w x^0 y \notin L$ da die Anzahl der a oder die Anzahl der b nicht mehr mit der Anzahl der c übereinstimmt.
 - Falls v oder x die Form $a^+ b^+$ hat, gilt $uv^2 w x^2 y \notin L$ da $uv^2 w x^2 y$ ein Teilwort der Form $a^+ b^+ a^+$ enthält.
- $vw x = b^j$: analog zu Fall a^j
- $vw x = b^k c^j$: analog zu Fall $a^k b^j$
- $vw x = c^j$: analog zu Fall a^j

Die Sprache L ist also nicht kontextfrei. □

Lemma 3.16: Die Menge der kontextfreien Sprachen (CH2) ist eine echte Teilmenge der Menge der kontextsensitiven Sprachen (CH1).

BEWEIS: Aus Beobachtung 3.1 wissen wir bereits, dass $CH2 \subseteq CH1$ gilt. Aus Bsp. 3.3 wissen wir, dass $L = \{a^n b^n c^n \mid n \geq 1\}$ von einer kontextsensitiven Grammatik erzeugt wird. Mit Hilfe von Lemma 3.15 folgern wir $CH2 \subsetneq CH1$. □

3.5 Abschlusseigenschaften für kontextfreie Sprachen

Satz 3.17: Die Menge der kontextfreien Sprachen (CFL) ist abgeschlossen unter Vereinigung ($\cup$), Konkatenation ($\cdot$) und dem Sternoperator ($*$).

BEWEIS: Für zwei CFG $\mathcal{G}_i = (\Sigma, N_i, P_i, S_i)$, $i = 1, 2$, konstruiere $\mathcal{G} = (\Sigma, N, P, S)$ mit

$$\begin{aligned}
 \text{„}\cup\text{“: } N &= N_1 \dot{\cup} N_2 \dot{\cup} \{S\} \\
 P &= \{S \rightarrow S_1, S \rightarrow S_2\} \cup P_1 \cup P_2 \\
 \text{„}\cdot\text{“: } N &= N_1 \dot{\cup} N_2 \dot{\cup} \{S\} \\
 P &= \{S \rightarrow S_1 S_2\} \cup P_1 \cup P_2 \\
 \text{„}\ast\text{“: } N &= N_1 \dot{\cup} \{S\} \\
 P &= \{S \rightarrow \varepsilon, S \rightarrow S_1 S\} \dot{\cup} P_1
 \end{aligned}$$
□

Satz 3.18: CFL ist *nicht* abgeschlossen unter Durchschnitt ($\cap$).

BEWEIS: für $n \geq 1$

$$\underbrace{\{a^n b^n c^m \mid n, m \geq 1\}}_{\in \text{CFL}} \cap \underbrace{\{a^m b^n c^n \mid m, n \geq 1\}}_{\in \text{CFL}} = \underbrace{\{a^n b^n c^n\}}_{\notin \text{CFL}}$$

Also CFL ist nicht abgeschlossen unter Durchschnitt (siehe Lemma 3.15). $\square$

Satz 3.19: CFL ist *nicht* unter Komplement ($\overline{\cdot}$) abgeschlossen.

BEWEIS: Angenommen, CFL wäre abgeschlossen unter Komplement.

Falls $L_1, L_2 \in \text{CFL}$, dann ist $\overline{L_1}, \overline{L_2} \in \text{CFL}$ nach Annahme.

$\Rightarrow \overline{L_1} \cup \overline{L_2} \in \text{CFL}$ wegen Abschluss unter „ $\cup$ “.

$\Rightarrow \overline{L_1} \cup \overline{L_2} = \overline{L_1 \cap L_2} \in \text{CFL}$ – Widerspruch zu Satz 3.18. $\square$

Satz 3.20: CFL ist abgeschlossen unter Schnitt mit der Menge der regulären Sprachen (REG).

BEWEIS: Sei $L \in \text{CFL}$, R eine reguläre Sprache, $\mathcal{G} = (\Sigma, N, P, S)$ eine CFG in CNF mit $L(\mathcal{G}) = L$ und $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein NEA mit $L(\mathcal{A}) = R$.

Konstruiere $\mathcal{G}' = (\Sigma, N', P', S')$ wie folgt:

- $N' = Q \times N \times Q \dot{\cup} \{S'\}$,
- P' enthält folgende Regeln
 - $S' \rightarrow (q^{\text{init}}, S, q_f)$, für jedes $q_f \in F$;
 - $(q_0, A, q_2) \rightarrow (q_0, B, q_1)(q_1, C, q_2)$, für alle $q_0, q_1, q_2 \in Q$, $A \rightarrow BC \in P$;
 - $(q, A, q') \rightarrow a$, für alle $q \in Q$, $q' \in \delta(q, a)$, $A \rightarrow a \in P$.

Jede Ableitung $S' \vdash_{\mathcal{G}'}^* w$ kann durch Löschen der Q -Komponenten und Überspringen des ersten Ableitungsschritts in eine $\mathcal{G}$ -Ableitung transformiert werden. Ebenso können wir aus der Ableitung einen akzeptierenden Lauf von $\mathcal{A}$ auf w extrahieren. Also ist $w \in L \cap R$.

Umgekehrt: Gegeben einen $\mathcal{G}$ -Ableitungsbaum $\mathcal{T}$ für w und einen akzeptierenden Lauf von $\mathcal{A}$ auf w , konstruiere einen $\mathcal{G}'$ -Ableitungsbaum für w durch eine in-order Traversierung von $\mathcal{T}$. Der Lauf gibt an den Produktionen an den Blättern eindeutig den jeweiligen Folgezustand vor. $\square$

3.6 Einschub: Typ-3-Sprachen

Satz 3.21: L ist regulär $\Leftrightarrow L$ ist Typ-3-Sprache.

BEWEIS: „ $\Rightarrow$ “ Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ DEA für L .

Konstruiere Typ-3-Grammatik $\mathcal{G} = (\Sigma, N, P, S)$ mit

$$N = Q$$

$$S = q^{\text{init}}$$

$$P = \{q \rightarrow aq' \mid q, q' \in Q, a \in \Sigma, \delta(q, a) = q'\} \cup \{q \rightarrow \varepsilon \mid q \in F\}$$

Es gilt $L(\mathcal{G}) = L(\mathcal{A})$. (ohne Beweis)

„ $\Leftarrow$ “ Sei $\mathcal{G} = (\Sigma, N, P, S)$ Typ-3-Grammatik. Dann erhalten wir durch Anwenden von BIN eine äquivalente Grammatik, in der jede Regel eine der folgenden Formen hat:

$$A \rightarrow \varepsilon \quad \text{für } A \in N$$

$$A \rightarrow a \quad \text{für } A \in N, a \in \Sigma$$

$$A \rightarrow a_1 a_2 \quad \text{für } A \in N, a_1, a_2 \in \Sigma$$

$$A \rightarrow aB \quad \text{für } A \in N, a \in \Sigma, B \in N$$

Wir machen nun die folgenden Modifikationen.

- Wir fügen eine frische Variable Y_ε und die Regel $Y_\varepsilon \rightarrow \varepsilon$ hinzu.
- Wir löschen jede Regel der zweiten Form und führen stattdessen die Regel $A \rightarrow aY_\varepsilon$ ein.
- Wir löschen jede Regel der dritten Form und führen stattdessen eine frische Variable Y_{a_2} und die Regeln $A \rightarrow a_1 Y_{a_2}$ und $Y_{a_2} \rightarrow a_2 Y_\varepsilon$ ein.

Die resultierende Grammatik $\mathcal{G} = (\Sigma, N', P', S)$ ist äquivalent zu $\mathcal{G}$. (Ohne Beweis)

Wir konstruieren nun den folgenden NEA. $\mathcal{N} = (\Sigma, Q, \delta, q^{\text{init}}, F)$

$$Q = N'$$

$$B \in \delta(A, a) \text{ gdw } A \rightarrow aB \in P'$$

$$q^{\text{init}} = S$$

$$F = \{A \in N' \mid A \rightarrow \varepsilon \in P'\}$$

Es gilt $L(\mathcal{G}) = L(\mathcal{A})$. (ohne Beweis) □

Lemma 3.22: $CH3 \subsetneq CH2$ (d.h., die regulären Sprachen sind eine *echte* Teilmenge der kontextfreien Sprachen).

BEWEIS: Betrachte $L_{\text{centered}} = \{0^n 10^n \mid n \in \mathbb{N}\} \subseteq \{0, 1\}^*$

Sowohl aus Bsp. 2.9 als auch aus Bsp. 2.14 ist bekannt, dass L_{centered} nicht regulär ist.

Es gibt aber eine Typ-2-Grammatik für L_{centered} :

$$\mathcal{G} = (\{0, 1\}, \{S\}, \{S \rightarrow 1, S \rightarrow 0S0\}, S)$$

(ohne Beweis) □

4 Kellerautomaten pushdown automaton (PDA)

Die im letzten Kapitel vorstellten kontextfreien Sprachen können wir bisher nur mit Hilfe von kontextfreien Grammatiken darstellen. Für die regulären Sprachen aus Kapitel 2 haben wir dagegen mit DEAs, NEAs, ε -NEAs, regulären Ausdrücken und regulären Grammatiken verschiedene Darstellungsformen kennengelernt. Die Automaten hatten dabei für uns einen besonderen Reiz, da diese Darstellungsform Ähnlichkeiten mit Computern hat. In diesem Kapitel werden wir nun den Kellerautomaten, ein Automatenmodell für die kontextfreien Sprachen, kennenlernen.

Ein Kellerautomat ist im wesentlichen ein ε -NEA, der um einen Stapelspeicher mit unbeschränkter Kapazität erweitert wurde.

Bei jedem Zustandsübergang darf der Kellerautomat das oberste Element des Stapelspeichers inspizieren und durch eine (potentiell leere) Sequenz von Elementen ersetzen. Der Automat darf dabei die Entscheidung über den Folgezustand vom inspizierten Element abhängig machen. Zu Beginn liegt genau ein Element, das *Kellerbodensymbol*, auf dem Stapelspeicher.

Der Kellerautomat unterscheidet sich noch in einem weiteren Detail vom ε -NEA: Es gibt keine akzeptierende Zustände. Der Kellerautomat akzeptiert stattdessen, wenn am Ende eines Zustandsübergangs das Eingabewort komplett gelesen wurde und der Stapelspeicher leer ist.

Analog zu endlichen Automaten können wir auch Kellerautomaten mit Hilfe eines Zustandsdiagramms beschreiben. Ein Beispiel ist in Abb. 9 dargestellt. Alle Transitionen sind mit einem Tripel beschriftet. Die erste Komponente des Tripels ist dabei das gelesene Zeichen der Eingabe, die zweite Komponente ist das vom Stapelspeicher genommene Element und die dritte Komponente ist die Sequenz, die auf den Stapelspeicher gelegt wird.

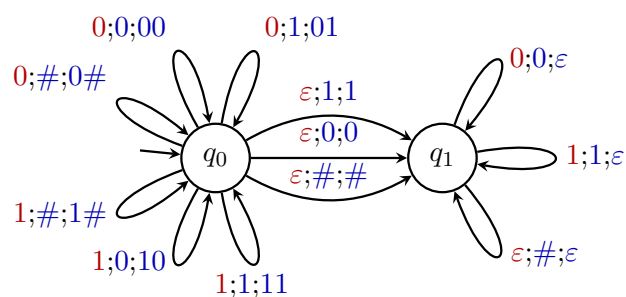


Abb. 9: Zustandsdiagramm eines Kellerautomaten

Der in Abb. 9 dargestellte Kellerautomat akzeptiert die Sprache $L = \{ww^R \mid w \in \{0,1\}^*\}$

und verwendet das Zeichen $\#$ als Kellerbodensymbol. In Zustand q_0 wird für jedes Eingabezeichen das entsprechende Symbol auf den Stapelspeicher gelegt. Zu jedem Zeitpunkt kann der Automat nichtdeterministisch (mit Hilfe einer ε -Transition) in den Zustand q_1 wechseln. Diese Transition sollte genau bei der Hälfte des Eingabeworts genommen werden. In Zustand q_1 vergleicht der Automat sukzessive die verbleibenden Eingabesymbole mit dem Inhalt des Stapelspeichers. Nur bei Übereinstimmung wird der Stapelspeicher bis zum Kellerbodensymbol geleert. Falls nur noch das Kellerbodensymbol auf dem Stapelspeicher liegt, kann der Automat dieses mit der Transition „ $\varepsilon; \#; \varepsilon$ “ entfernen und die Eingabe akzeptieren.

Auf Englisch heißen Kellerautomaten *pushdown automata* und wir werden die Abkürzung PDA für Kellerautomaten verwenden. In diesem Kapitel befassen wir uns zunächst nur mit nichtdeterministischen Kellerautomaten und definieren diese formal wie folgt.

Def. 4.1: Ein nichtdeterministischer Kellerautomat (NPDA)²² ist ein 6-Tupel

$$(\Sigma, Q, \Gamma, q^{\text{init}}, Z^{\text{init}}, \delta).$$

Dabei ist

- Σ , das *Eingabealphabet*,
- Q eine endliche Menge von *Zuständen*,
- Γ , das *Kelleralphabet*,
- $q^{\text{init}} \in Q$, der Startzustand,
- $Z^{\text{init}} \in \Gamma$, das *Kellerbodensymbol*,
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ die *Transitionsfunktion*. ◇

Bsp. 4.1: Formale Beschreibung des in Abb. 9 abgebildeten Kellerautomaten.

$$\begin{array}{ll} \Sigma = \{0, 1\} & \text{Eingabealphabet} \\ \Gamma = \{0, 1, \#\} & \text{Kelleralphabet} \\ Q = \{q_0, q_1\} & \# \triangleq \text{Kellerbodensymbol} \\ \delta(q_0, a, Z) = \{(q_0, aZ)\} & a \in \{0, 1\}, Z \in \Gamma \quad (3) \\ \delta(q_0, \varepsilon, Z) = \{(q_1, Z)\} & (4) \\ \delta(q_1, a, a) = \{(q_1, \varepsilon)\} & (5) \\ \delta(q_1, \varepsilon, \#) = \{(q_1, \varepsilon)\} & (6) \end{array}$$

Hat die graphische Darstellung eines Kellerautomaten sehr viele Transitionen müssen wir diese nicht alle explizit zeichnen. Statt dessen können wir auch wie in Abb. 10 Variablen verwenden um mehrere Transitionen zusammenzufassen.

²²Englisch: Nondeterministic Pushdown Automaton

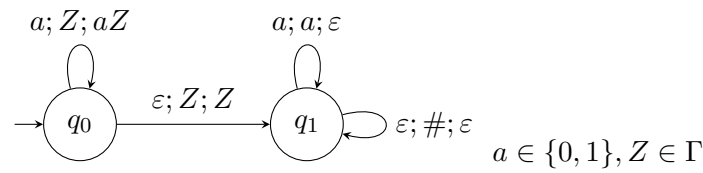


Abb. 10: Zustandsdiagramm der Kellerautomaten aus Abb. 9. Statt alle Transitionen explizit zu zeichnen verwenden wir die Variablen a und Z um eine Menge von Transitionen durch eine einzige Kante zu repräsentieren.

Im Weiteren sei $\mathcal{K} = (\Sigma, Q, \Gamma, q^{\text{init}}, Z^{\text{init}}, \delta)$ ein NPDA.

Vorlesung:
17.06.2026

Def. 4.2: Die Menge der *Konfigurationen* von $\mathcal{K}$ ist $\text{Konf}(\mathcal{K}) = Q \times \Sigma^* \times \Gamma^*$.

Die *Schrittrelation* von $\mathcal{K}$

$$\triangleright \subseteq \text{Konf}(\mathcal{K}) \times \text{Konf}(\mathcal{K})$$

ist definiert durch

$$\begin{aligned} (q, aw, Z\gamma) &\triangleright (q', w, \beta\gamma) && \text{falls } \delta(q, a, Z) \ni (q', \beta) \\ (q, w, Z\gamma) &\triangleright (q', w, \beta\gamma) && \text{falls } \delta(q, \varepsilon, Z) \ni (q', \beta). \end{aligned}$$

Wir schreiben $(q, w, \gamma) \triangleright^n (q', w', \gamma')$, wenn $\mathcal{K}$ in $n \in \mathbb{N}$ Schritten von Konfiguration (q, w, γ) zur Konfiguration (q', w', γ') gelangt.

Wir schreiben $\triangleright^*$ für die reflexive, transitive Hülle von $\triangleright$.

Eine *akzeptierende Konfiguration* hat die Form $(q, \varepsilon, \varepsilon)$ mit $q \in Q$.

Die von $\mathcal{K}$ *akzeptierte Sprache* ist

$$L(\mathcal{K}) = \{w \in \Sigma^* \mid \exists q \in Q : (q^{\text{init}}, w, Z^{\text{init}}) \triangleright^* (q, \varepsilon, \varepsilon)\}$$

◇

Bsp.: Die folgenden Schritte von $\mathcal{K}$ aus Beispiel 4.1 zeigen, dass $w = 0110 \in L(\mathcal{K})$:

$$\begin{aligned} &(q_0, 0110, \#) \\ \triangleright &(q_0, 110, 0\#) \quad (\text{„pushen“ des Eingabesymbols 0}) \\ \triangleright &(q_0, 10, 10\#) \quad (\text{„pushen“ des Eingabesymbols 1}) \\ \triangleright &(q_1, 10, 10\#) \quad (\varepsilon\text{-Übergang von } q_0 \text{ nach } q_1) \\ \triangleright &(q_1, 0, 0\#) \quad (\text{„poppen“ des Eingabesymbols 1}) \\ \triangleright &(q_1, \varepsilon, \#) \quad (\text{„poppen“ des Eingabesymbols 0}) \\ \triangleright &(q_1, \varepsilon, \varepsilon) \quad (\varepsilon\text{-Übergang zum Entfernen von } \#) \end{aligned}$$

Lemma 4.1: Zu jeder kontextfreien Sprache L gibt es einen NPDA $\mathcal{K}$, sodass $L(\mathcal{K}) = L$.

Zum Führen des Beweises benötigen wir folgende Lemmas.

Lemma 4.2: Für alle $q, q' \in Q$, $w \in \Sigma^*$, $Z \in \Gamma$ und $n \in \mathbb{N}$ gilt:

Wenn $(q, w, Z) \triangleright^n (q', \varepsilon, \varepsilon)$, dann $\forall v \in \Sigma^*, \gamma \in \Gamma^* : (q, wv, Z\gamma) \triangleright^n (q', v, \gamma)$.

BEWEIS: Übungsaufgabe. □

Lemma 4.3: Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine CFG und $\beta \vdash_{\mathcal{G}}^* \gamma$ eine Ableitung.

Dann ist auch $\alpha\beta\delta \vdash^* \alpha\gamma\delta$ eine Ableitung, für alle $\alpha, \delta \in (N \cup \Sigma)^*$.

BEWEIS: Übung oder selbst. □

BEWEIS (von Lemma 4.1): Sei $\mathcal{G} = (\Sigma, N, P, S)$ eine kontextfreie Grammatik für L in CNF. Definiere einen NPDA $\mathcal{K} = (\Sigma, Q, \Gamma, q^{\text{init}}, Z^{\text{init}}, \delta)$ durch:

- $Q = \{q^{\text{init}}\}$
- $\Gamma = \Sigma \cup N$
- $Z^{\text{init}} = S$
- $\delta(q^{\text{init}}, a, a) = \{(q^{\text{init}}, \varepsilon)\}$ für $a \in \Sigma$
- $\delta(q^{\text{init}}, \varepsilon, A) = \{(q^{\text{init}}, \alpha)\}$ für $A \rightarrow \alpha \in P$

Wir zeigen nun, dass $L(\mathcal{K}) = L(\mathcal{G})$.

- Beweisrichtung „ $\supseteq$ “

Sei $w \in L(\mathcal{G})$, dann gilt $S \vdash^* w$ bzw. $w = Y(\mathcal{T})$ für ein $\mathcal{T} \in \text{Abl}(S)$. Hierfür schreiben wir abkürzend $\mathcal{T} \in \text{Abl}(S, w)$. Wir wollen zeigen, dass $(q^{\text{init}}, w, S) \triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon)$ folgt und beweisen dafür mittels Induktion über den zugehörigen Ableitungsbaum $\mathcal{T}$ die folgende stärkere Eigenschaft.

$$\forall A \in N : \text{ wenn } \mathcal{T} \in \text{Abl}(A, w), \text{ dann } (q^{\text{init}}, w, A) \triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon)$$

Der Ableitungsbaum für w hat die Form $\mathcal{T} = \pi(\mathcal{T}_1 \dots \mathcal{T}_n)$ und wir führen eine Fallunterscheidung nach der Produktion π durch.

- $\pi = S \rightarrow \varepsilon$ mit $n = 0$. Nach Konstruktion gilt $(q^{\text{init}}, \varepsilon) \in \delta(q^{\text{init}}, \varepsilon, S)$ und somit $(q^{\text{init}}, \varepsilon, S) \triangleright (q^{\text{init}}, \varepsilon, \varepsilon)$.

- $\pi = A \rightarrow a$, $a \in \Sigma$ mit $n = 0$. Nach Konstruktion gilt $(q^{\text{init}}, a) \in \delta(q^{\text{init}}, \varepsilon, A)$ und $(q^{\text{init}}, \varepsilon) \in \delta(q^{\text{init}}, a, a)$.

Somit gilt $(q^{\text{init}}, a, A) \triangleright (q^{\text{init}}, a, a) \triangleright (q^{\text{init}}, \varepsilon, \varepsilon)$.

- $\pi = A \rightarrow BC$ mit $B, C \in N$ und $n = 2$.

Also ist $\mathcal{T} = \pi(\mathcal{T}_1, \mathcal{T}_2)$, $\mathcal{T}_1 \in \text{Abl}(B, u_1)$, $\mathcal{T}_2 \in \text{Abl}(C, u_2)$ mit $w = u_1 u_2$.

Nach Konstruktion gilt $(q^{\text{init}}, BC) \in \delta(q^{\text{init}}, \varepsilon, A)$.

Nach I.V. gilt $(q^{\text{init}}, u_1, B) \triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon)$ und $(q^{\text{init}}, u_2, C) \triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon)$.

Mit Lemma 4.2 folgt schließlich:

$$\begin{aligned} (q^{\text{init}}, w, A) &= (q^{\text{init}}, u_1 u_2, A) \triangleright (q^{\text{init}}, u_1 u_2, BC) \\ &\triangleright^* (q^{\text{init}}, u_2, C) \\ &\triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon). \end{aligned}$$

- Beweisrichtung „ $\subseteq$ “

Sei $w \in L(\mathcal{K})$, dann gilt $(q^{\text{init}}, w, S) \triangleright^* (q^{\text{init}}, \varepsilon, \varepsilon)$. Wir wollen $S \vdash^* w$ folgern und beweisen dafür mittels vollständiger Induktion über die Anzahl der Berechnungsschritte n die folgende Aussage:

$$\forall n \in \mathbb{N} : \forall w \in \Sigma^*, \alpha \in \Gamma^* : \text{wenn } (q^{\text{init}}, w, \alpha) \triangleright^n (q^{\text{init}}, \varepsilon, \varepsilon), \text{ dann } \alpha \vdash^* w$$

I.A. $n = 0$. $w = \varepsilon$, $\alpha = \varepsilon$: Es gilt $\varepsilon \vdash^* \varepsilon$.

I.S. $n \rightsquigarrow n + 1$, $\alpha = Z\alpha'$: Betrachte den ersten Schritt der Berechnung

$$(q^{\text{init}}, w, Z\alpha') \triangleright (q^{\text{init}}, w', \beta\alpha') \triangleright^n (q^{\text{init}}, \varepsilon, \varepsilon),$$

mit $\delta(q^{\text{init}}, x, Z) \ni (q^{\text{init}}, \beta)$, $x \in \Sigma \cup \{\varepsilon\}$.

Es gibt zwei Fälle für Z :

- $Z = a$ für $a \in \Sigma$. Es folgt $\beta = \varepsilon$, $w = aw'$ und $x = a$.

Nach I.V. gilt $\alpha' \vdash^* w'$ und mit Lemma 4.3 auch $\alpha = a\alpha' \vdash^* aw' = w$.

- $Z = A$ mit $A \rightarrow \beta \in P$. Es folgt $w = w'$ und $x = \varepsilon$.

Nach I.V. gilt $\beta\alpha' \vdash^* w'$ und somit auch $A\alpha \vdash^* \beta\alpha' \vdash^* w' = w$. □

Bei einem NPDA nehmen wir in jedem Schritt ein Zeichen vom Kellerspeicher herunter, aber dürfen beliebig viele Symbole auf den Kellerspeicher zurücklegen. Was wäre, wenn die Höhe des Kellers pro Schritt höchstens um eins größer werden könnte? Verlören wir dadurch nur Komfort oder könnten wir dadurch weniger Sprachen akzeptieren?

Das folgende Lemma beantwortet diese Frage.

Lemma 4.4: Zu jedem NPDA gibt es einen äquivalenten NPDA, bei dem für alle $q, q' \in Q, Z \in \Gamma, \gamma \in \Gamma^*, x \in \Sigma \cup \{\varepsilon\}$ gilt: Falls $(q', \gamma) \in \delta(q, x, Z)$, dann $|\gamma| \leq 2$.

BEWEIS: Für jede Transition der Form $(q', \gamma) \in \delta(q, x, Z)$ mit $\gamma = Z_n \dots Z_1$ und $n > 2$:

- Erzeuge neue Zustände $q_2 \dots q_{n-1}$;
- Ersetze (q', γ) durch $(q_2, Z_2 Z_1)$;
- Definiere $\delta(q_i, \varepsilon, Z) = \begin{cases} \{(q_{i+1}, Z_{i+1} Z_i)\} & \text{falls } x = \varepsilon \text{ und } Z = Z_i \\ \emptyset & \text{sonst;} \end{cases}$
- Setze $\delta(q_{n-1}, x, Z) = \begin{cases} \{(q', Z_n Z_{n-1})\} & \text{falls } x = \varepsilon \text{ und } Z = Z_{n-1} \\ \emptyset & \text{falls } x \in \Sigma \text{ oder } Z \neq Z_{n-1}. \end{cases}$

Wiederhole, bis alle Transitionen die gewünschte Form haben. Die Sprache des NPDA ändert sich durch diese Transformation nicht (ohne Beweis). $\square$

Lemma 4.5: Zu jedem NPDA $\mathcal{K}$ gibt es eine CFG $\mathcal{G}$, sodass $L(\mathcal{K}) = L(\mathcal{G})$.

BEWEIS: Ohne Einschränkung (Lemma 4.4) sei $\mathcal{K} = (Q, \Sigma, \Gamma, q^{\text{init}}, Z^{\text{init}}, \delta)$ ein NPDA mit $|\gamma| \leq 2$ für alle $(q', \gamma) \in \delta(q, x, Z), q, q' \in Q, x \in \Sigma \cup \{\varepsilon\}, Z \in \Gamma$.

Definiere $\mathcal{G} = (\Sigma, N, P, S)$ mit $N = (Q \times \Gamma \times Q) \cup \{S\}$. Wir wählen diese Tripel als Nichtterminalsymbole, da wir eine Grammatik konstruieren wollen, welche die folgende Eigenschaft hat:

$$[q, Z, q'] \vdash^n w \text{ gdw } (q, w, Z) \triangleright^n (q', \varepsilon, \varepsilon) \quad (*)$$

In Worten: Wir können vom Nichtterminalsymbol $[q, Z, q']$ das Wort w genau dann in n Schritten ableiten, wenn der NPDA die Konfiguration (q, w, Z) in n Schritten in eine akzeptierende Konfiguration mit Zustand q' überführen kann.

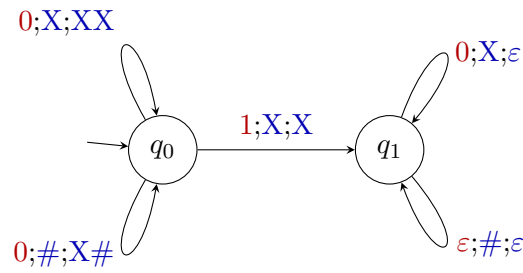
Als Menge der Regeln wählen wir $P = P_S \cup P_0 \cup P_1 \cup P_2$ mit:

$$\begin{aligned} P_S &= \{S \rightarrow [q^{\text{init}}, Z^{\text{init}}, q'] \mid q' \in Q\}, \\ P_0 &= \{[q, Z, q'] \rightarrow x \mid (q', \varepsilon) \in \delta(q, x, Z), x \in \Sigma \cup \{\varepsilon\}\}, \\ P_1 &= \{[q, Z, q'] \rightarrow x[q'', Z', q'] \mid (q'', Z') \in \delta(q, x, Z), q' \in Q, x \in \Sigma \cup \{\varepsilon\}\}, \\ P_2 &= \{[q, Z, q'] \rightarrow x[q_1, Z_1, q_2][q_2, Z_2, q'] \mid (q_1, Z_1 Z_2) \in \delta(q, x, Z), q', q_2 \in Q, x \in \Sigma \cup \{\varepsilon\}\} \end{aligned}$$

Dabei unterscheiden P_0 , P_1 und P_2 die Fälle, dass der Stack kleiner wird (P_0), gleich hoch bleibt (P_1) oder größer wird (P_2).

Beispiel:

NPDA für $L_{\text{centered3}} := \{0^n 10^n \mid n \in \mathbb{N}, n > 0\} \subseteq \{0, 1\}^*$:



010 wird akzeptiert, weil $(q_0, 010, \#) \triangleright (q_0, 10, X\#) \triangleright (q_1, 0, X\#) \triangleright (q_1, \varepsilon, \#) \triangleright (q_1, \varepsilon, \varepsilon)$.

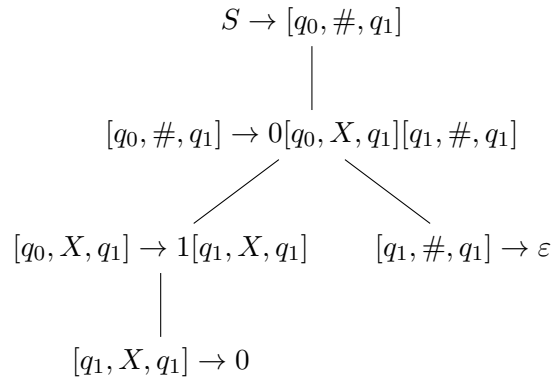
$$P_S = \{S \rightarrow [q_0, \#, q_0] \\ S \rightarrow [q_0, \#, q_1]\}$$

$$P_0 = \{[q_1, X, q_1] \rightarrow 0 \\ [q_1, \#, q_1] \rightarrow \varepsilon\}$$

$$P_1 = \{[q_0, X, q_0] \rightarrow 1[q_1, X, q_0] \\ [q_0, X, q_1] \rightarrow 1[q_1, X, q_1]\}$$

$$P_2 = \{[q_0, \#, q_0] \rightarrow 0[q_0, X, q_0][q_0, \#, q_0] \\ [q_0, \#, q_0] \rightarrow 0[q_0, X, q_1][q_1, \#, q_0] \\ [q_0, \#, q_1] \rightarrow 0[q_0, X, q_0][q_0, \#, q_1] \\ [q_0, \#, q_1] \rightarrow 0[q_0, X, q_1][q_1, \#, q_1] \\ [q_0, X, q_0] \rightarrow 0[q_0, X, q_0][q_0, X, q_0] \\ [q_0, X, q_0] \rightarrow 0[q_0, X, q_1][q_1, X, q_0] \\ [q_0, X, q_1] \rightarrow 0[q_0, X, q_0][q_0, X, q_1] \\ [q_0, X, q_1] \rightarrow 0[q_0, X, q_1][q_1, X, q_1]\}$$

Ableitungsbaum für 010:



Offensichtlich folgt aus dieser Eigenschaft $L(\mathcal{G}) = L(\mathcal{K})$, da wir vom Startsymbol S genau die Nichtterminalsymbole der Form $[q^{\text{init}}, Z^{\text{init}}, q']$ ableiten können und der NPDA genau die Wörter w akzeptiert, für die $(q^{\text{init}}, w, Z^{\text{init}})$ in eine akzeptierende Konfiguration überführt werden kann.

Es bleibt zu zeigen, dass die Eigenschaft (*) gilt. Wir zeigen dafür mittels Induktion über n dass für alle $n \in \mathbb{N}$ die folgende Eigenschaft gilt:

$$\forall n' < n : [q, Z, q'] \vdash^{n'} w \text{ gdw } (q, w, Z) \triangleright^{n'} (q', \varepsilon, \varepsilon) \quad (*)$$

I.A. $n = 1$: Folgt direkt aus den P_0 -Regeln (und der Tatsache, dass keine der anderen Regeln ein einzelnes Symbol oder ε ableiten kann).

I.S. $n \rightsquigarrow n + 1$:

- Beweisrichtung „ $\Rightarrow$ “

Sei $[q, Z, q'] \vdash^{n+1} w$ eine Ableitung für w . Für den ersten Ableitungsschritt kommen zwei Fälle in Frage:

- Fall 1: α hat die Form $x[q'', Z', q']$ mit $q'' \in Q$ ($[q, Z, q'] \rightarrow \alpha \in P_1$).

Dann existiert $x \in \Sigma \cup \{\varepsilon\}$ und $w' \in \Sigma^*$ sodass $w = xw'$ und $[q'', Z', q'] \vdash^n w'$. Nach I.V. gilt nun, dass $(q'', w', Z') \triangleright^n (q', \varepsilon, \varepsilon)$.

Nach Konstruktion gilt $(q'', Z') \in \delta(q, x, Z)$ und somit $(q, w, Z) \triangleright (q'', w', Z') \triangleright^n (q', \varepsilon, \varepsilon)$.

- Fall 2: α hat die Form $x[q_1, Z_1, q_2][q_2, Z_2, q']$ ($[q, Z, q'] \rightarrow \alpha \in P_2$).

Es gibt also einen Ableitungsbaum $\mathcal{T} = \pi(\mathcal{T}_1, \mathcal{T}_2)$, mit $\mathcal{T}_1 \in \text{Abl}([q_1, Z_1, q_2])$ und $\mathcal{T}_2 \in \text{Abl}([q_2, Z_2, q'])$, sodass $w = xY(\mathcal{T}_1)Y(\mathcal{T}_2)$.

Es gilt also $[q_1, Z_1, q_2] \vdash^{k_1} Y(\mathcal{T}_1)$ und $[q_2, Z_2, q'] \vdash^{k_2} Y(\mathcal{T}_2)$ für $k_1, k_2 \in \mathbb{N}$, sodass $k_1 + k_2 = n$.

Per I.V. folgt

$$(q_1, Y(\mathcal{T}_1), Z_1) \triangleright^{k_1} (q_2, \varepsilon, \varepsilon).$$

Mit Lemma 4.2 folgt auch

$$(q_1, Y(\mathcal{T}_1)Y(\mathcal{T}_2), Z_1Z_2) \triangleright^{k_1} (q_2, Y(\mathcal{T}_2), Z_2).$$

Auch hier können wir wieder I.V. anwenden und erhalten:

$$(q_2, Y(\mathcal{T}_2), Z_2) \triangleright^{k_2} (q', \varepsilon, \varepsilon)$$

Somit gilt:

$$(q, \underbrace{xY(\mathcal{T}_1)Y(\mathcal{T}_2)}_w, Z) \triangleright (q_1, Y(\mathcal{T}_1)Y(\mathcal{T}_2), Z_1Z_2) \triangleright^{k_1} (q_2, Y(\mathcal{T}_2), Z_2) \triangleright^{k_2} (q', \varepsilon, \varepsilon)$$

- Beweisrichtung „ $\Leftarrow$ “ Sei $(q, w, Z) \triangleright^{n+1} (q', \varepsilon, \varepsilon)$ eine Berechnung für w . Da wir annehmen, dass der NPDA in einem Schritt höchstens zwei Symbole auf den Stack legen darf, kommen für den ersten Berechnungsschritt nur zwei Fälle in Frage:

– Fall 1: $(q, w, Z) \triangleright (q'', w', Z')$ mit $w = xw'$ für $x \in \Sigma \cup \{\varepsilon\}$

Aus $(q'', w', Z') \triangleright^n (q', \varepsilon, \varepsilon)$ folgern wir mit I.V. $[q'', Z', q'] \vdash^n w'$. Nach Konstruktion können wir $[q, Z, q'] \vdash x[q'', Z', q']$ ableiten.

– Fall 2: $(q, w, Z) \triangleright (q_1, w', Z_1 Z_2)$ mit $w = xw'$ für $x \in \Sigma \cup \{\varepsilon\}$

Aus $(q_1, w', Z_1 Z_2) \triangleright^n (q', \varepsilon, \varepsilon)$ folgern wir: $\exists k_1, k_2 \in \mathbb{N}, q_2 \in Q, w_1, w_2 \in \Sigma^*$, sodass gilt:

1. $k_1 + k_2 = n$
2. $w' = w_1 w_2$
3. $(q_1, w', Z_1 Z_2) \triangleright^{k_1} (q_2, w_2, Z_2)$
4. $(q_2, w_2, Z_2) \triangleright^{k_2} (q', \varepsilon, \varepsilon)$

Wir wählen k_1 als die kleinste Zahl, welche die obigen Eigenschaften erfüllt. Damit wissen wir, dass wir Z_2 noch nie vom Keller genommen haben.

Aus 3. folgern wir $(q_1, w_1, Z_1) \triangleright^{k_1} (q_2, \varepsilon, \varepsilon)$ und mit I.V. $[q_1, Z_1, q_2] \vdash^{k_1} w_1$.

Aus 4. folgern wir mit I.V. $[q_2, Z_2, q'] \vdash^{k_2} w_2$.

Es gibt also einen Ableitungsbaum $\mathcal{T} = \pi(\mathcal{T}_1, \mathcal{T}_2)$, mit $\mathcal{T}_1 \in \text{Abl}([q_1, Z_1, q_2])$ und $\mathcal{T}_2 \in \text{Abl}([q_2, Z_2, q'])$, sodass $w = xY(\mathcal{T}_1)Y(\mathcal{T}_2)$.

Somit gibt es auch eine Ableitung

$$[q, Z, q'] \vdash x[q_1, Z_1, q_2][q_2, Z_2, q'] \vdash^{k_1+k_2} xw_1w_2. \quad \square$$

Satz 4.6: Die Menge der von Kellerautomaten akzeptierten Sprachen und die Menge der kontextfreien Sprachen sind identisch.

BEWEIS: Folgt direkt aus Lemma 4.1 und Lemma 4.5. $\square$

Bemerkung: Zu jedem NPDA gibt es einen äquivalenten NPDA, der nur einen Zustand hat. (Folgt aus Lemma 4.5 und der Konstruktion aus dem Beweis von Lemma 4.1.)

5 Rekursive Funktionen

Turingmaschinen beschreiben Berechnungen durch einzelne Konfigurationsübergänge. In diesem Kapitel nehmen wir eine funktionale Perspektive ein: Aus wenigen einfachen Funktionen über $\mathbb{N}$ erzeugen wir durch fest vorgegebene Konstruktionsprinzipien immer größere Klassen berechenbarer Funktionen.

Zunächst führt primitive Rekursion ausschließlich zu totalen Funktionen. Die Sprache LOOP liefert dafür ein imperatives Berechnungsmodell. Durch Minimierung kommt partielle Berechenbarkeit hinzu; die Sprache WHILE bildet das imperative Gegenstück. Anschließend vergleichen wir diese Berechenbarkeitsbegriffe mit Turingmaschinen. Zum Abschluss zeigt die Ackermannfunktion, dass nicht jede totale berechenbare Funktion bereits primitiv rekursiv ist.

Notation: Mit $f : A \rightarrow B$ bezeichnen wir eine *totale* Funktion von A nach B und mit $f : A \hookrightarrow B$ eine *partielle* Funktion von A nach B . Jede totale Funktion kann auch als partielle Funktion betrachtet werden.

5.1 Primitive Rekursion

Als erstes Konstruktionsprinzip betrachten wir die primitive Rekursion. Dabei wird das erste Argument schrittweise aufgebaut, während die übrigen Argumente als Parameter unverändert bleiben.

Def. 5.1 (Schema der primitiven Rekursion):

$$\text{Sei } g : \mathbb{N}^k \rightarrow \mathbb{N} \quad , \quad k \geq 0$$

$$h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$$

Falls eine Funktion $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ die folgenden Gleichungen erfüllt:

$$\begin{aligned} f(0, \bar{x}) &= g(\bar{x}) \quad , \quad \bar{x} \in \mathbb{N}^k \\ f(n+1, \bar{x}) &= h(f(n, \bar{x}), n, \bar{x}) \end{aligned} \tag{7}$$

Dann sei

$$f =: \text{PR}(g, h)$$

aus g und h durch primitive Rekursion definiert. $\diamond$

Die Rekursionsgleichungen sollen nicht nur eine Berechnungsvorschrift andeuten, sondern tatsächlich genau eine totale Funktion festlegen. Dies ist Inhalt des folgenden Lemmas.

Lemma 5.1: $\text{PR}(g, h)$ ist wohldefiniert.

BEWEIS: Seien $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ wie in der Definition. Wir definieren $F \subseteq (\mathbb{N} \times \mathbb{N}^k) \times \mathbb{N}$ als die von den folgenden Regeln induktiv erzeugte Relation:

$$\frac{}{((0, \bar{x}), g(\bar{x})) \in F} \quad \frac{((n, \bar{x}), y) \in F}{((n+1, \bar{x}), h(y, n, \bar{x})) \in F}.$$

Dabei besteht F genau aus den Tupeln, die sich durch endlich viele Anwendungen dieser Regeln herleiten lassen. Insbesondere gelten die folgenden Inversionseigenschaften:

$$\begin{aligned} ((0, \bar{x}), y) \in F &\implies y = g(\bar{x}), \\ ((n+1, \bar{x}), y) \in F &\implies \exists z \in \mathbb{N} : ((n, \bar{x}), z) \in F \wedge y = h(z, n, \bar{x}). \end{aligned}$$

Totalität Wir zeigen durch Induktion über n , dass für alle $\bar{x} \in \mathbb{N}^k$ ein $y \in \mathbb{N}$ mit $((n, \bar{x}), y) \in F$ existiert. Für $n = 0$ liefert die erste Regel das Element $((0, \bar{x}), g(\bar{x}))$. Gibt es für n ein y mit $((n, \bar{x}), y) \in F$, so liefert die zweite Regel $((n+1, \bar{x}), h(y, n, \bar{x})) \in F$.

Funktionalität Wir zeigen ebenfalls durch Induktion über n , dass aus

$$((n, \bar{x}), y) \in F \quad \text{und} \quad ((n, \bar{x}), y') \in F$$

stets $y = y'$ folgt. Für $n = 0$ ergeben die Inversionseigenschaften $y = g(\bar{x}) = y'$. Für den Induktionsschritt gibt es $z, z' \in \mathbb{N}$ mit

$$\begin{aligned} ((n, \bar{x}), z) \in F, & \quad y = h(z, n, \bar{x}), \\ ((n, \bar{x}), z') \in F, & \quad y' = h(z', n, \bar{x}). \end{aligned}$$

Nach Induktionsvoraussetzung ist $z = z'$ und damit auch $y = y'$.

Da F total und funktional ist, definiert

$$f(n, \bar{x}) = y \iff ((n, \bar{x}), y) \in F$$

eine totale Funktion $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$.

Rekursionsgleichungen Nach der ersten Erzeugungsregel gilt $((0, \bar{x}), g(\bar{x})) \in F$ und somit $f(0, \bar{x}) = g(\bar{x})$. Aus $((n, \bar{x}), f(n, \bar{x})) \in F$ folgt nach der zweiten Regel

$$((n+1, \bar{x}), h(f(n, \bar{x}), n, \bar{x})) \in F.$$

Aufgrund der Funktionalität von F gilt daher

$$f(n+1, \bar{x}) = h(f(n, \bar{x}), n, \bar{x}).$$

Damit erfüllt f beide Gleichungen aus (7).

Eindeutigkeit Sei $f' : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ eine weitere Funktion, die (7) erfüllt. Ihr Graph

$$G_{f'} := \{((n, \bar{x}), f'(n, \bar{x})) \mid n \in \mathbb{N}, \bar{x} \in \mathbb{N}^k\}$$

ist unter beiden Erzeugungsregeln abgeschlossen: Dies folgt für die erste Regel aus der Anfangsgleichung und für die zweite Regel aus der Rekursionsgleichung von f' . Da F die kleinste unter diesen Regeln abgeschlossene Relation ist, gilt $F \subseteq G_{f'}$. Folglich ist für alle n und $\bar{x}$ auch $f(n, \bar{x}) = f'(n, \bar{x})$. □

Mit dem wohldefinierten Rekursionsschema können wir nun zunächst eine Sprache primitiv rekursiver Terme festlegen. Ihre Gesamtheit bezeichnen wir als Menge der primitiv rekursiven Terme (PREC); ihre Bedeutung als Funktionen definieren wir anschließend getrennt.

Def. 5.2 (Syntax der PREC): Die Mengen PREC^k der *primitiv rekursiven Terme der Stelligkeit k* sind für alle $k \in \mathbb{N}$ simultan als die kleinste Familie von Mengen definiert, welche die folgenden Bedingungen erfüllt:

1. Für jedes $k \in \mathbb{N}$ gilt $\mathcal{O}^k \in \text{PREC}^k$.
2. Für $k \geq 1$ und $1 \leq j \leq k$ gilt $\pi_j^k \in \text{PREC}^k$.
3. $\mathcal{S} \in \text{PREC}^1$.
4. Sind $t \in \text{PREC}^m$ und $u_1, \dots, u_m \in \text{PREC}^k$, für $m, k \in \mathbb{N}$, so gilt

$$t \circ [u_1, \dots, u_m] \in \text{PREC}^k.$$

5. Sind $t \in \text{PREC}^k$ und $u \in \text{PREC}^{k+2}$, so gilt

$$\text{PR}(t, u) \in \text{PREC}^{k+1}.$$

Die Menge aller primitiv rekursiven Terme ist

$$\text{PREC} := \bigcup_{k \in \mathbb{N}} \text{PREC}^k.$$

◇

Die Stelligkeit eines Terms ist bereits durch seine syntaktische Bildung bestimmt. Nun interpretieren wir jeden Term durch eine totale Funktion derselben Stelligkeit.

Def. 5.3 (Semantik der PREC): Für jedes $k \in \mathbb{N}$ ist die Semantikabbildung

$$\llbracket \cdot \rrbracket_k : \text{PREC}^k \rightarrow (\mathbb{N}^k \rightarrow \mathbb{N})$$

durch strukturelle Rekursion über primitiv rekursive Terme definiert:

$$\begin{aligned} \llbracket \mathcal{O}^k \rrbracket_k(\bar{x}) &= 0, \\ \llbracket \pi_j^k \rrbracket_k(x_1, \dots, x_k) &= x_j, \\ \llbracket \mathcal{S} \rrbracket_1(x) &= x + 1, \\ \llbracket t \circ [u_1, \dots, u_m] \rrbracket_k(\bar{x}) &= \llbracket t \rrbracket_m(\llbracket u_1 \rrbracket_k(\bar{x}), \dots, \llbracket u_m \rrbracket_k(\bar{x})), & t \in \text{PREC}^m, u_i \in \text{PREC}^k, \\ \llbracket \text{PR}(t, u) \rrbracket_{k+1} &= \text{PR}(\llbracket t \rrbracket_k, \llbracket u \rrbracket_{k+2}) & t \in \text{PREC}^k, u \in \text{PREC}^{k+2}. \end{aligned}$$

Die Semantik einer Menge von Termen schreiben wir als

$$\begin{aligned} \llbracket \text{PREC}^k \rrbracket_k &:= \{\llbracket t \rrbracket_k \mid t \in \text{PREC}^k\}, \\ \llbracket \text{PREC} \rrbracket &:= \bigcup_{k \in \mathbb{N}} \llbracket \text{PREC}^k \rrbracket_k. \end{aligned}$$

Die Funktionen in $\llbracket \text{PREC} \rrbracket$ heißen *primitiv rekursiv*. $\diamond$

Die folgenden Konstruktionen zeigen, wie sich vertraute arithmetische Operationen allein aus diesen Bausteinen zusammensetzen lassen.

Bsp.:

1. Der Term

$$\text{add} := \text{PR}(\pi_1^1, \mathcal{S} \circ [\pi_1^3]) \in \text{PREC}^2$$

bezeichnet die Addition. Für $\text{add} := \llbracket \text{add} \rrbracket_2$ gilt nämlich

$$\begin{aligned} \text{add}(0, x) &= x \\ \text{add}(n + 1, x) &= \text{add}(n, x) + 1. \end{aligned}$$

2. Unter Verwendung des Additionsterms erhalten wir den Multiplikationsterm

$$\text{mult} := \text{PR}(\mathcal{O}^1, \text{add} \circ [\pi_3^3, \pi_1^3]) \in \text{PREC}^2.$$

Für $\text{mult} := \llbracket \text{mult} \rrbracket_2$ gilt

$$\begin{aligned} \text{mult}(0, x) &= 0, \\ \text{mult}(n + 1, x) &= \text{add}(x, \text{mult}(n, x)). \end{aligned}$$

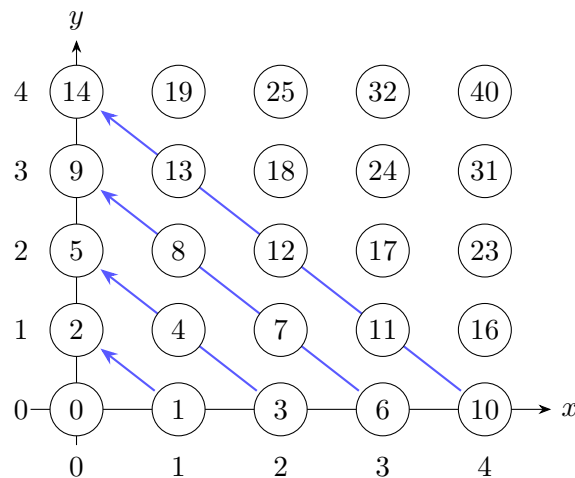


Abb. 11: Die Cantorsche Paarfunktion $cp(x, y) = tr(x + y) + y$. Die Pfeile zeigen die Aufzählungsrichtung auf den Diagonalen $x + y = \text{konstant}$.

3. Subtraktion auf natürlichen Zahlen ist nicht total. Stattdessen verwenden wir die *Monus* Operation definiert durch $x \dot{-} y = \max(x - y, 0)$ hier definiert als sub. Dazu definieren wir zunächst die Terme

Vorlesung:
24.06.2026

$$\begin{aligned} \text{pred} &:= \text{PR}(\mathcal{O}^0, \pi_2^2) && \in \text{PREC}^1, \\ \text{sub} &:= \text{PR}(\pi_1^1, \text{pred} \circ [\pi_1^3]) \circ [\pi_2^2, \pi_1^2] && \in \text{PREC}^2. \end{aligned}$$

Ihre Semantiken $\text{pred} := \llbracket \text{pred} \rrbracket_1$ und $\text{sub} := \llbracket \text{sub} \rrbracket_2$ erfüllen

$$\text{pred}(0) = 0, \quad \text{pred}(n + 1) = n, \quad \text{sub}(x, y) = x \dot{-} y.$$

Um mehrere natürliche Zahlen gemeinsam durch eine einzige natürliche Zahl zu codieren, benötigen wir Paarfunktionen.

Def. 5.4: Eine *Paarfunktion* ist eine bijektive Funktion $p : \mathbb{N}^2 \rightarrow \mathbb{N}$. ◇

Die Cantorsche Paarfunktion liefert eine konkrete primitiv rekursive Paarfunktion; auch ihre beiden Umkehrprojektionen lassen sich primitiv rekursiv berechnen.

Bsp. 5.1: Die Cantorsche Paarfunktion cp ist wie folgt definiert.

$$\begin{aligned} tr : \mathbb{N} &\rightarrow \mathbb{N} & tr(0) &= 0 & tr(n+1) &= n+1+tr(n) \\ cp : \mathbb{N}^2 &\rightarrow \mathbb{N} & cp(x, y) &= tr(x+y)+y \end{aligned}$$

Diese Funktionen werden von den folgenden primitiv rekursiven Termen bezeichnet:

$$\begin{aligned} tr &:= \text{PR}(\mathcal{O}^0, \text{add} \circ [\mathcal{S} \circ [\pi_2^2], \pi_1^2]) \in \text{PREC}^1, \\ cp &:= \text{add} \circ [\text{tr} \circ [\text{add}], \pi_2^2] \in \text{PREC}^2, \\ tr &= \llbracket \text{tr} \rrbracket_1, & cp &= \llbracket \text{cp} \rrbracket_2. \end{aligned}$$

Die diagonale Aufzählung von $\mathbb{N}^2$ durch cp ist in Abb. 11 dargestellt.

Die beiden Projektionen $px, py : \mathbb{N} \rightarrow \mathbb{N}$ mit den Eigenschaften (für alle $x, y, n \in \mathbb{N}$)

$$x = px(cp(x, y)) \quad y = py(cp(x, y)) \quad n = cp(px(n), py(n))$$

sind ebenfalls primitiv rekursiv.

Dazu definiere eine primitiv rekursive Funktion $untr : \mathbb{N} \rightarrow \mathbb{N}$, sodass $m = untr(n)$ impliziert $tr(m) \leq n$ und $n < tr(m+1)$. (Übung oder selbst)

Dann setze

$$py(n) = n \dot{-} tr(untr(n)) \quad px(n) = untr(n) \dot{-} py(n)$$

5.2 LOOP Berechenbarkeit

Die bisherigen Konstruktionsprinzipien beschreiben Funktionen auf abstrakte Weise. LOOP-Programme stellen ihnen ein einfaches imperatives Berechnungsmodell mit ausschließlich beschränkten Schleifen gegenüber.

Def. 5.5 (LOOP-Programme): Für jedes $k \geq 1$ ist LOOP^k die Menge der Programme, die nur die Register $x_1, \dots, x_k$ verwenden. Ihre Syntax ist induktiv definiert durch

$$\begin{array}{ll} P, Q ::= & x_i := x_i + 1 \quad (\text{Inkrementierung}), \\ & | \quad x_i := x_i \dot{-} 1 \quad (\text{Dekrementierung mit Abschneiden bei 0}), \\ & | \quad P; Q \quad (\text{sequenzielle Ausführung}), \\ & | \quad \text{LOOP } x_i \text{ DO } P \text{ END} \quad (\text{beschränkte Schleife}). \end{array}$$

Dabei gilt in jeder Regel $1 \leq i \leq k$. Die Menge aller LOOP-Programme ist

$$\text{LOOP} := \bigcup_{k \geq 1} \text{LOOP}^k.$$

◇

Die Syntax legt zunächst nur die erlaubten Programme fest. Ihre Bedeutung definieren wir als Transformation des gesamten Registerzustands.

Def. 5.6 (Semantik eines LOOP-Programms): Für jedes $k \geq 1$ ist die Semantikabbildung

$$\llbracket \cdot \rrbracket_k : \text{LOOP}^k \rightarrow (\mathbb{N}^k \rightarrow \mathbb{N}^k)$$

durch strukturelle Rekursion wie folgt definiert:

- $\llbracket x_i := x_i + 1 \rrbracket_k(a_1, \dots, a_i, \dots, a_k) = (a_1, \dots, a_i + 1, \dots, a_k)$
- $\llbracket x_i := x_i - 1 \rrbracket_k(a_1, \dots, a_i, \dots, a_k) = (a_1, \dots, a_i - 1, \dots, a_k)$
- $\llbracket P; Q \rrbracket_k(\bar{a}) = \llbracket Q \rrbracket_k(\llbracket P \rrbracket_k(\bar{a}))$
- $\llbracket \text{LOOP } x_i \text{ DO } P \text{ END} \rrbracket_k(\bar{a}) = \llbracket P \rrbracket_k^{(a_i)}(\bar{a})$

Für $0 \leq n \leq k$ sei

$$\text{init}_{n,k} : \mathbb{N}^n \rightarrow \mathbb{N}^k, \quad \text{init}_{n,k}(a_1, \dots, a_n) = (a_1, \dots, a_n, 0, \dots, 0).$$

Dabei werden rechts $k - n$ Nullen angefügt. Ein Programm $P \in \text{LOOP}^k$ *berechnet* eine Funktion $f : \mathbb{N}^n \rightarrow \mathbb{N}^k$, wenn

$$f = \pi_1^k \circ \llbracket P \rrbracket_k \circ \text{init}_{n,k}.$$

Eine Funktion heißt *LOOP-berechenbar*, wenn sie von einem solchen Programm berechnet wird. $\diamond$

In Worten: Die Zuweisungen und die sequenzielle Ausführung haben ihre übliche Bedeutung. Bei der Ausführung von `LOOP  $x_i$  DO  $P$  END` wird der Wert r von x_i beim Eintritt in die Schleife einmal gelesen und anschließend P genau r -mal ausgeführt. Änderungen von x_i innerhalb von P beeinflussen die Anzahl der Schleifendurchläufe nicht.

Die Definition der Semantik von LOOP-Programmen ist *kompositionell*, weil Semantik einer zusammengesetzten Anweisung als Funktion der Semantik ihrer Teilanweisungen definiert ist.

Die arithmetischen Operationen aus den vorangegangenen Beispielen lassen sich bereits durch sehr kurze LOOP-Programme berechnen.

Bsp. 5.2: Die beiden Eingaben x, y stehen jeweils in x_1, x_2 ; die Ausgabe wird aus x_1 gelesen.

Addition Das Programm

$$P_{\text{add}} := \text{LOOP } x_2 \text{ DO } x_1 := x_1 + 1 \text{ END} \in \text{LOOP}^2$$

erfüllt

$$\llbracket P_{\text{add}} \rrbracket_2(x, y) = (x + y, y).$$

Also berechnet P_{add} die Addition.

Subtraktion (Monus) Analog gilt für

$$P_{\text{sub}} := \text{LOOP } x_2 \text{ DO } x_1 := x_1 \dot{-} 1 \text{ END} \in \text{LOOP}^2$$

die Gleichung

$$\llbracket P_{\text{sub}} \rrbracket_2(x, y) = (x \dot{-} y, y).$$

Somit berechnet P_{sub} die abgeschnittene Subtraktion.

Multiplikation Hier verwenden wir x_3 als Arbeitsregister. Es enthält im initialen Zustand $\text{init}_{2,3}(x, y) = (x, y, 0)$ den Wert 0. Sei

$$\begin{aligned} P_{\text{mult}} := & \text{LOOP } x_1 \text{ DO } x_3 := x_3 + 1 \text{ END;} \\ & \text{LOOP } x_1 \text{ DO } x_1 := x_1 \dot{-} 1 \text{ END;} \\ & \text{LOOP } x_2 \text{ DO LOOP } x_3 \text{ DO } x_1 := x_1 + 1 \text{ END END.} \end{aligned}$$

Dann ist $P_{\text{mult}} \in \text{LOOP}^3$. Die drei Programmteile transformieren den initialen Zustand nacheinander gemäß

$$(x, y, 0) \mapsto (x, y, x) \mapsto (0, y, x) \mapsto (xy, y, x).$$

Folglich gilt

$$\pi_1^3(\llbracket P_{\text{mult}} \rrbracket_3(\text{init}_{2,3}(x, y))) = xy,$$

also berechnet P_{mult} die Multiplikation.

Vorlesung:
30.06.2026

5.3 Eigenschaften primitiv-rekursiver Funktionen

Die Beispiele illustrieren eine Richtung des folgenden Charakterisierungssatzes.

Satz 5.2: Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ist LOOP-berechenbar genau dann, wenn sie primitiv rekursiv ist.

Obwohl primitiv rekursive Funktionen nur einen Teil der totalen Funktionen berechnen, ist es dennoch nicht möglich, sie effektiv zu vergleichen.

Satz 5.3: Gegeben zwei primitiv rekursive Funktionen $f, g : \mathbb{N}^k \rightarrow \mathbb{N}$. Das Prädikat $P(f, g) := \forall \bar{a} \in \mathbb{N}^k. f(\bar{a}) = g(\bar{a})$ ist unentscheidbar.

BEWEIS: Wir betrachten die Funktion $h : \mathbb{N}^k \rightarrow \mathbb{N}$ definiert durch $h(\bar{a}) = (f(\bar{a}) \dot{-} g(\bar{a})) + (g(\bar{a}) \dot{-} f(\bar{a}))$. Es gilt, dass $Q(h) := \forall \bar{a} \in \mathbb{N}^k. h(\bar{a}) = 0$ genau dann, wenn $P(f, g)$.

Wir zeigen nun, dass $Q(h)$ unentscheidbar ist durch Reduktion auf das Halteproblem von Turingmaschinen. Dazu sei M eine deterministische Turingmaschine (TM). Wir skizzieren, wie die Berechnung von M für endlich viele Schritte durch eine primitiv rekursive Funktion bewerkstelligt werden kann.

1. Zustände, Bandsymbole, endliche Wörter und damit Konfigurationen (v, q, w) von M werden durch natürliche Zahlen codiert. Die Codierung kann so gewählt werden, dass die benötigten Codierungs- und Decodierungsfunktionen primitiv rekursiv sind (iterierte Paarfunktion).
2. Da M fest ist, sind die Funktionen

$$\text{start}_M : \mathbb{N}^k \rightarrow \mathbb{N} \quad \text{und} \quad \text{step}_M : \mathbb{N} \rightarrow \mathbb{N}$$

primitiv rekursiv. Sie liefern den Code der Startkonfiguration bzw. den Code der eindeutigen Folgekonfiguration. Für Haltekonfigurationen setzen wir $\text{step}_M(c) = c$.

3. Durch primitive Rekursion ist auch

$$\begin{aligned} \text{conf}_M(0, \bar{x}) &= \text{start}_M(\bar{x}), \\ \text{conf}_M(n+1, \bar{x}) &= \text{step}_M(\text{conf}_M(n, \bar{x})) \end{aligned}$$

primitiv rekursiv. $\text{conf}_M(n, \bar{x})$ codiert die Konfiguration nach n Schritten.

4. Die charakteristische Funktion halt_M der Codes von Haltekonfigurationen ist primitiv rekursiv; dabei sei $\text{halt}_M(c) = 0$ genau für Haltekonfigurationen. Dann ist

$$H_M(n, \bar{x}) := \text{halt}_M(\text{conf}_M(n, \bar{x}))$$

primitiv rekursiv.

5. Definiere nun eine weitere primitiv rekursive Funktion $h_M(n, \bar{x}) = 1 \dot{-} H_M(n, \bar{x})$. Es gilt, dass $\forall n, \bar{x}. h_M(n, \bar{x}) = 0$ genau dann, wenn für alle n die TM M keine Haltekonfiguration erreicht. Das heißt, M terminiert auf keiner Eingabe. Diese Eigenschaft ist äquivalent zum Halteproblem und damit unentscheidbar.

□

Daher kann es keine syntaktische Normalform geben, durch die ein solcher Vergleich ja durchgeführt werden könnte.

5.4 μ -rekursive Funktionen

Primitive Rekursion und die dazu äquivalenten LOOP-Programme liefern für jede Eingabe einen Funktionswert. Diese Totalität ist grundlegend, schränkt den Berechenbarkeitsbegriff aber zugleich ein. Um auch nicht terminierende Berechnungen zu beschreiben, führen wir nun eine potenziell partielle Suchoperation ein. Sie sucht die erste Nullstelle, darf dabei aber keinen undefinierten früheren Funktionswert überspringen.

Def. 5.7 (Minimierung): Sei $f : \mathbb{N}^{k+1} \hookrightarrow \mathbb{N}$ partiell. Für $\bar{x} \in \mathbb{N}^k$ sei

$$C_f(\bar{x}) := \{n \in \mathbb{N} \mid f(n, \bar{x}) = 0 \text{ und } \forall j < n : f(j, \bar{x}) \downarrow \neq 0\}.$$

Dabei bedeutet $f(j, \bar{x}) \downarrow$, dass $f(j, \bar{x})$ definiert ist. Die *Minimierung* von f ist durch die folgende Vorschrift gegeben:

$$(\mu f)(\bar{x}) = \begin{cases} \min C_f(\bar{x}), & C_f(\bar{x}) \neq \emptyset, \\ \text{undefiniert}, & C_f(\bar{x}) = \emptyset. \end{cases}$$

◇

Da die Ausgangsfunktion partiell sein darf, ist zu prüfen, ob diese Vorschrift wirklich eine eindeutig bestimmte partielle Funktion liefert.

Satz 5.4 (Wohldefiniertheit der Minimierung): Die Vorschrift aus Def. 5.7 bestimmt zu jeder partiellen Funktion $f : \mathbb{N}^{k+1} \hookrightarrow \mathbb{N}$ eindeutig eine partielle Funktion $\mu f : \mathbb{N}^k \hookrightarrow \mathbb{N}$.

BEWEIS: Sei $\bar{x} \in \mathbb{N}^k$ fest. Falls $C_f(\bar{x}) = \emptyset$ ist, weist die Vorschrift $\bar{x}$ keinen Funktionswert zu. Andernfalls besitzt $C_f(\bar{x})$ nach dem Wohlordnungsprinzip für $\mathbb{N}$ ein eindeutig bestimmtes kleinstes Element.

Tatsächlich kann $C_f(\bar{x})$ sogar höchstens ein Element enthalten: Wären $n, m \in C_f(\bar{x})$ mit $n < m$, so wäre einerseits $f(n, \bar{x}) = 0$. Andererseits würde $m \in C_f(\bar{x})$ wegen $n < m$ gerade $f(n, \bar{x}) \downarrow \neq 0$ verlangen, ein Widerspruch.

Somit wird jedem $\bar{x}$ entweder genau ein Wert aus $\mathbb{N}$ oder kein Wert zugewiesen. Die Vorschrift definiert also eine partielle Funktion. Sowohl ihr Definitionsbereich $\{\bar{x} \mid C_f(\bar{x}) \neq \emptyset\}$ als auch ihr Wert auf diesem Definitionsbereich sind durch f festgelegt. Daher ist diese partielle Funktion eindeutig bestimmt. □

Aus dem Beweis ergeben sich unmittelbar die folgenden nützlichen Fälle für definierte und undefinierte Minimierungen.

Beobachtung 5.1: Für $\bar{x} \in \mathbb{N}^k$ folgen unmittelbar aus der Definition:

1. Gibt es kein $n \in \mathbb{N}$, für das $f(n, \bar{x})$ definiert und gleich 0 ist, so ist $(\mu f)(\bar{x})$ undefiniert.
2. Sei n_0 das kleinste n , für das $f(n, \bar{x})$ definiert und gleich 0 ist. Gibt es ein $j < n_0$, für das $f(j, \bar{x})$ undefiniert ist, so ist $(\mu f)(\bar{x})$ undefiniert.
3. Gilt $f(n, \bar{x}) = 0$ und $f(j, \bar{x}) \downarrow \neq 0$ für alle $j < n$, so gilt $(\mu f)(\bar{x}) = n$.

Für die Semantik μ -rekursiver Terme verwenden wir auch Komposition und primitive Rekursion auf partiellen Funktionen.

Bemerkung: Die semantischen Konstruktionen aus Def. 5.3 können genauso auch für partielle Funktionen definiert und verwendet werden. Dabei ist ein Funktionsaufruf nur dann definiert, wenn alle seine Argumente definiert sind. Ebenso lässt sich Lemma 5.1 für partielle Funktionen formulieren.

Ergänzen wir die Syntax der primitiv rekursiven Terme um Minimierung, erhalten wir die Menge der μ -rekursiven Terme (μREC), eine Sprache, deren Terme auch partielle Funktionen bezeichnen können.

Def. 5.8 (Syntax der μREC): Die Mengen μREC^k der μ -rekursiven Terme der Stelligkeit k sind simultan als die kleinste Familie von Mengen definiert, welche alle Bildungsregeln aus Def. 5.2 erfüllt, wenn darin jede Menge PREC^ℓ durch μREC^ℓ ersetzt wird, und zusätzlich unter folgender Regel abgeschlossen ist:

$$\frac{t \in \mu\text{REC}^{k+1}}{\mu t \in \mu\text{REC}^k}.$$

Die Menge aller μ -rekursiven Terme ist

$$\mu\text{REC} := \bigcup_{k \in \mathbb{N}} \mu\text{REC}^k.$$

◇

Die Semantik erweitert nun die Semantik der primitiv rekursiven Terme. Daher muss nur die Bedeutung des neuen Minimierungsterms festgelegt werden.

Def. 5.9 (Semantik der μREC): Für jedes $k \in \mathbb{N}$ setzt die Semantikabbildung

$$\llbracket \cdot \rrbracket_k : \mu\text{REC}^k \rightarrow (\mathbb{N}^k \hookrightarrow \mathbb{N})$$

die Semantik aus Def. 5.3 fort. Für die aus Def. 5.2 übernommenen Syntaxfälle gelten dieselben Semantikgleichungen, nun für partielle Funktionen. Dabei ist zu beachten, dass die Komposition *strikt* ist. Das heißt, wenn irgendeine Komponentenfunktion kein Ergebnis liefert, dann liefert die gesamte Komposition kein Ergebnis. Für den zusätzlichen Syntaxfall gilt

$$\llbracket \mu t \rrbracket_k = \mu(\llbracket t \rrbracket_{k+1}) \quad (t \in \mu\text{REC}^{k+1}).$$

Wir setzen

$$\begin{aligned} \llbracket \mu\text{REC}^k \rrbracket_k &:= \{\llbracket t \rrbracket_k \mid t \in \mu\text{REC}^k\}, \\ \llbracket \mu\text{REC} \rrbracket &:= \bigcup_{k \in \mathbb{N}} \llbracket \mu\text{REC}^k \rrbracket_k. \end{aligned}$$

Die Funktionen in $\llbracket \mu\text{REC} \rrbracket$ heißen *μ -rekursiv*. ◇

5.5 WHILE Berechenbarkeit

Die Minimierung hat den funktionalen Formalismus um partielle Berechnungen erweitert. Auf Programmebene erreichen wir dasselbe mit der Menge der WHILE-Programme (WHILE): Dazu erweitern wir LOOP um eine Schleife mit datenabhängiger Abbruchbedingung.

Def. 5.10 (Syntax von WHILE-Programmen): Für jedes $k \geq 1$ ist WHILE^k die Menge der WHILE-Programme, die nur die Register $x_1, \dots, x_k$ verwenden. Ihre Syntax ist induktiv definiert durch

$$\begin{array}{ll} P, Q ::= & x_i := x_i + 1 \quad (\text{Inkrementierung}), \\ & | \quad x_i := x_i - 1 \quad (\text{Dekrementierung mit Abschneiden bei 0}), \\ & | \quad P; Q \quad (\text{sequenzielle Ausführung}), \\ & | \quad \text{LOOP } x_i \text{ DO } P \text{ END} \quad (\text{beschränkte Schleife}), \\ & | \quad \text{WHILE } x_i \neq 0 \text{ DO } P \text{ END} \quad (\text{unbeschränkte Schleife}). \end{array}$$

Dabei gilt in jeder Regel $1 \leq i \leq k$. Die Menge aller WHILE-Programme ist

$$\text{WHILE} := \bigcup_{k \geq 1} \text{WHILE}^k.$$

◇

Da eine unbeschränkte Schleife nicht terminieren muss, ist ihre Semantik im Allgemeinen nur eine partielle Funktion auf Registerzuständen.

Def. 5.11 (Semantik von WHILE-Programmen): Für jedes $k \geq 1$ ist die Semantikabbildung

$$\llbracket \cdot \rrbracket_k : \text{WHILE}^k \rightarrow (\mathbb{N}^k \hookrightarrow \mathbb{N}^k)$$

durch strukturelle Rekursion definiert. Für Inkrementierung und Dekrementierung gelten die Gleichungen aus Def. 5.6. Sequenzielle Ausführung und beschränkte Schleifen werden durch

$$\begin{aligned} \llbracket P; Q \rrbracket_k &= \llbracket Q \rrbracket_k \circ \llbracket P \rrbracket_k, \\ \llbracket \text{LOOP } x_i \text{ DO } P \text{ END} \rrbracket_k(\bar{a}) &= \llbracket P \rrbracket_k^{(a_i)}(\bar{a}) \end{aligned}$$

definiert, wobei Komposition und Iteration nun partiell sein können.

Für die unbeschränkte Schleife sei

$$H_{P,i}(\bar{a}) := \{n \in \mathbb{N} \mid \llbracket P \rrbracket_k^{(n)}(\bar{a}) \downarrow \text{ und } \pi_i^k(\llbracket P \rrbracket_k^{(n)}(\bar{a})) = 0\}.$$

Dann ist

$$\llbracket \text{WHILE } x_i \neq 0 \text{ DO } P \text{ END} \rrbracket_k(\bar{a}) = \begin{cases} \llbracket P \rrbracket_k^{(m)}(\bar{a}), & H_{P,i}(\bar{a}) \neq \emptyset \text{ und } m = \min H_{P,i}(\bar{a}), \\ \text{undefiniert,} & H_{P,i}(\bar{a}) = \emptyset. \end{cases}$$

Ist $m = \min H_{P,i}(\bar{a})$, so ist die i -te Komponente von $\llbracket P \rrbracket_k^{(\ell)}(\bar{a})$ für alle $\ell < m$ von 0 verschieden und von $\llbracket P \rrbracket_k^{(m)}(\bar{a})$ gleich 0.

Für $0 \leq n \leq k$ berechnet ein Programm $P \in \text{WHILE}^k$ die partielle Funktion $f : \mathbb{N}^n \hookrightarrow \mathbb{N}$, wenn als Gleichheit partieller Funktionen

$$f = \pi_1^k \circ \llbracket P \rrbracket_k \circ \text{init}_{n,k}$$

gilt, wobei $\text{init}_{n,k}$ wie in Def. 5.6 definiert ist. Insbesondere ist $f(\bar{a})$ genau dann undefiniert, wenn die Ausführung von P auf $\text{init}_{n,k}(\bar{a})$ nicht terminiert. Eine partielle Funktion heißt *WHILE-berechenbar*, wenn sie von einem solchen Programm berechnet wird. $\diamond$

Satz 5.5: Eine partielle Funktion $f : \mathbb{N}^n \hookrightarrow \mathbb{N}$ ist WHILE-berechenbar genau dann, wenn sie μ -rekursiv ist.

5.6 Turingäquivalenz

Mit μ -Rekursion und WHILE-Programmen haben wir nun zwei äquivalente Beschreibungen partieller Berechnungen. Als Nächstes vergleichen wir diesen Berechenbarkeitsbegriff mit dem Modell der Turingmaschine.

Satz 5.6: $\llbracket \mu\text{REC} \rrbracket$ stimmt mit der Klasse der TM-berechenbaren Funktionen über $\mathbb{N}$ überein.

BEWEIS:

- $\supseteq$ Sei M eine deterministische TM, die eine partielle Funktion $f : \mathbb{N}^k \hookrightarrow \mathbb{N}$ berechnet. Wir skizzieren, wie die Berechnung von M durch die Semantik eines Terms aus μREC simuliert wird. Die meisten Teile entsprechen der Konstruktion aus dem Beweis von Satz 5.3 und wir geben nur den neuen Teil an.

1. Die charakteristische Funktion halt_M der Codes von Haltekonfigurationen ist primitiv rekursiv; dabei gelte $\text{halt}_M(c) = 0$ genau für Haltekonfigurationen. Damit ist auch

$$H_M(n, \bar{x}) := \text{halt}_M(\text{conf}_M(n, \bar{x}))$$

primitiv rekursiv. Durch Minimierung erhalten wir die μ -rekursive partielle Funktion

$$t_M(\bar{x}) := (\mu H_M)(\bar{x}).$$

Diese Funktion ist genau dann undefiniert, wenn M auf $\bar{x}$ nicht hält.

2. Schließlich sei out_M die primitiv rekursive Funktion, die aus dem Code einer Haltekonfiguration die codierte Ausgabe liest. Dann gilt

$$f(\bar{x}) = \text{out}_M(\text{conf}_M(t_M(\bar{x}), \bar{x})).$$

Die rechte Seite entsteht durch Minimierung und Komposition aus primitiv rekursiven Funktionen; folglich ist $f \in \llbracket \mu\text{REC} \rrbracket$.

- $\subseteq$ Sei umgekehrt $f : \mathbb{N}^n \hookrightarrow \mathbb{N}$ eine Funktion aus $\llbracket \mu\text{REC} \rrbracket$. Nach Satz 5.5 gibt es ein $k \geq n$ und ein Programm $P \in \text{WHILE}^k$, das f berechnet. Es genügt also, WHILE-Programme durch Turingmaschinen zu simulieren.

Wir codieren eine WHILE-Konfiguration als k Registerinhalte, etwa auf k Bändern oder auf einem Band mit Trennzeichen. Zu jedem $P \in \text{WHILE}^k$ konstruieren wir dann strukturell eine TM M_P , die dieselbe partielle Registertransformation berechnet.

1. Die Grundanweisungen $x_i := 0$, $x_i := x_i + 1$ und $x_i := x_i - 1$ werden durch endliche Turingmaschinen-Makros realisiert, die nur das i -te Registerband ändern.
2. Eine Sequenz $P; Q$ wird durch Hintereinanderschalten der Maschinen M_P und M_Q simuliert.
3. Für $\text{WHILE } x_i \neq 0 \text{ DO } P \text{ END}$ testet die Maschine zunächst Register i . Ist es 0, hält sie mit der aktuellen Registerbelegung. Ist es ungleich 0, führt sie M_P

aus und springt anschließend zurück zum Test. Divergiert eine Iteration von P oder wird der Test nie mit Wert 0 erreicht, so divergiert auch die konstruierte TM.

Damit berechnet M_P genau $\llbracket P \rrbracket_k$. Die vollständige Maschine für f initialisiert zusätzlich die Register $x_{n+1}, \dots, x_k$ mit 0 und gibt nach Terminierung das Register x_1 aus. Also ist f TM-berechenbar. $\square$

Zum Abschluss der Turing-Äquivalenz halten wir noch eine Normalform fest. Sie zeigt, dass für jede μ -rekursive Funktion eine einzige unbeschränkte Suche genügt; davor und danach stehen nur primitiv rekursive Berechnungen.

Satz 5.7 (Kleenes Normalformtheorem): Für jede partielle Funktion $F : \mathbb{N}^k \hookrightarrow \mathbb{N}$ mit $F \in \llbracket \mu\text{REC} \rrbracket$ gibt es primitiv rekursive Funktionen $p : \mathbb{N} \rightarrow \mathbb{N}$ und $q : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$, sodass

$$F(\bar{x}) = p((\mu q)(\bar{x}))$$

als Gleichheit partieller Funktionen gilt.

Aus diesem Normalformtheorem und den Äquivalenzen mit WHILE-Programmen und Turingmaschinen folgt insbesondere: Jede berechenbare Funktion über $\mathbb{N}$ kann durch ein WHILE-Programm berechnet werden, das höchstens eine WHILE-Anweisung enthält.

5.7 Die Ackermannfunktion

Zum Abschluss kehren wir von partiellen Berechnungen zu totalen Funktionen zurück. Dabei gilt:

Beobachtung 5.2: Alle primitiv rekursiven Funktionen sind total.
Aber: $\exists$ totale Funktionen, die nicht primitiv rekursiv sind.

Die Ackermannfunktion ist das zentrale Beispiel für die zweite Aussage: Sie ist berechenbar, wächst jedoch zu schnell, um primitiv rekursiv zu sein.

Def. 5.12: Die *Ackermannfunktion*

$$\begin{aligned} A : \mathbb{N}^2 &\rightarrow \mathbb{N} \\ A(0, y) &= y + 1 \\ A(x + 1, 0) &= A(x, 1) \\ A(x + 1, y + 1) &= A(x, A(x + 1, y)) \end{aligned} \quad \diamond$$

Für kleine Argumente ergeben sich die folgenden Werte und geschlossenen Formen.

x	$A(x, 0)$	$A(x, 1)$	$A(x, 2)$	$A(x, 3)$	$A(x, 4)$	$y \mapsto A(x, y)$
0	1	2	3	4	5	$y + 1$
1	2	3	4	5	6	$y + 2$
2	3	5	7	9	11	$2y + 3$
3	5	13	29	61	125	$2^{y+3} - 3$

Satz 5.8: Die Ackermannfunktion A ist μ -rekursiv, also $A \in \llbracket \mu\text{REC}^2 \rrbracket_2$.

BEWEIS (Beweisskizze): Nach Satz 5.5 genügt es, ein WHILE-Programm für A zu skizzieren. Das Programm wertet die rekursive Definition mit einem Stack aus. Der aktuelle Wert steht in einem Register r , der Stack s enthält die noch ausstehenden ersten Argumente von Ackermann-Aufrufen. Wir codieren den Stack mit der Paarungsfunktion: Der leere Stack ist 0, und

$$\text{push}(m, s) = cp(m, s) + 1.$$

Für $s > 0$ liefert $\text{pop}(s) = (px(s \dot{-} 1), py(s \dot{-} 1))$ das oberste Element und den Reststack. Diese Stackoperationen sind primitiv rekursiv und damit insbesondere WHILE-berechenbar.

Initialisiere $r := y$ und $s := \text{push}(x, 0)$. Solange $s \neq 0$ gilt, wird ein Element m vom Stack genommen und einer der folgenden Schritte ausgeführt:

$m = 0$: Setze $r := r + 1$.

$m > 0$ **und** $r = 0$: Lege $m \dot{-} 1$ auf den Stack und setze $r := 1$.

$m > 0$ **und** $r > 0$: Lege zuerst $m \dot{-} 1$ und danach m auf den Stack und setze $r := r \dot{-} 1$.

Die Fallunterscheidungen sind WHILE-Syntaxzucker: Tests auf 0 beziehungsweise auf Ungleichheit zu 0 lassen sich mit Hilfsregistern und WHILE-Schleifen ausdrücken.

Jeder Schleifendurchlauf ersetzt den obersten noch ausstehenden Aufruf genau entsprechend einer der drei Definitionsgleichungen von A . Die Stackmaschine führt daher eine Tiefensuche durch den endlichen Auswertungsbaum der Ackermann-Rekursion aus. Bei leerem Stack steht in r der Wert $A(x, y)$; das Programm kopiert r abschließend nach x_1 . Somit ist A WHILE-berechenbar und folglich μ -rekursiv. $\square$

Zunächst einige Monotonieeigenschaften der Ackermannfunktion.

Lemma 5.9: Für alle $x, y \in \mathbb{N}$ gelten folgende Ungleichheiten:

$$\begin{aligned} y &< A(x, y) \\ A(x, y) &< A(x, y + 1) \\ A(x, y + 1) &\leq A(x + 1, y) \\ A(x, y) &< A(x + 1, y) \\ A(x, 2y) &< A(x + 2, y) \end{aligned}$$

Um nachzuweisen, dass A nicht primitiv rekursiv ist, vergleichen wir die Wachstumsraten primitiv rekursiver Funktionen mit der Ackermannfunktion.

Satz 5.10: Sei $\mathcal{A} = \{f : \mathbb{N}^k \rightarrow \mathbb{N} \mid \exists n \forall \bar{x} : f(\bar{x}) < A(n, \max \bar{x})\}$ die Menge der Funktionen, die durch A majorisiert werden.

Es gilt: $\llbracket \text{PREC} \rrbracket \subseteq \mathcal{A}$

BEWEIS: Strukturelle Induktion über die Terme aus PREC:

1. $\llbracket \mathcal{O}^k \rrbracket_k(\bar{x}) = 0 < A(0, \max \bar{x}) = \max \bar{x} + 1$, also $\llbracket \mathcal{O}^k \rrbracket_k \in \mathcal{A}$.
2. $\llbracket \mathcal{S} \rrbracket_1(x) = x + 1 < x + 2 = A(1, x)$, also $\llbracket \mathcal{S} \rrbracket_1 \in \mathcal{A}$.
3. $\llbracket \pi_j^k \rrbracket_k(x_1, \dots, x_k) = x_j \leq \max \bar{x} < \max \bar{x} + 1 = A(0, \max \bar{x})$, also $\llbracket \pi_j^k \rrbracket_k \in \mathcal{A}$.
4. Seien $t \in \text{PREC}^m$ und $u_1, \dots, u_m \in \text{PREC}^k$. Setze $h = \llbracket t \rrbracket_m$ und $g_i = \llbracket u_i \rrbracket_k$. Nach Induktionsvoraussetzung gilt $g_1, \dots, g_m, h \in \mathcal{A}$. Daher existieren $r_1, \dots, r_m, s_0$ mit

$$\begin{aligned} g_i(\bar{x}) &< A(r_i, \max \bar{x}) & 1 \leq i \leq m, \\ h(\bar{y}) &< A(s_0, \max \bar{y}). \end{aligned}$$

Betrachte $f = h \circ [g_1, \dots, g_m] = \llbracket t \circ [u_1, \dots, u_m] \rrbracket_k$.

Setze $r = \max\{r_1, \dots, r_m, s_0 + 1\}$.

Setze $s = r - 1$ (es gilt $s_0 \leq s$).

$\hookrightarrow \forall i. g_i(\bar{x}) < A(r, \max \bar{x})$.

$$\begin{aligned} \text{Dann } f(\bar{x}) &= h(g_1(\bar{x}), \dots, g_m(\bar{x})) \\ &< A(s_0, \max(g_1(\bar{x}), \dots, g_m(\bar{x}))) \\ &\leq A(s, \max(g_1(\bar{x}), \dots, g_m(\bar{x}))) \\ &< A(s, \max(A(r_1, \max \bar{x}), \dots, A(r_m, \max \bar{x}))) \\ &= A(s, A(s + 1, \max \bar{x})) \\ &= A(s + 1, \max \bar{x} + 1) \\ &\leq A(\underline{s + 2}, \max \bar{x}) \end{aligned}$$

Also $f \in \mathcal{A}$.

5. Seien $t \in \text{PREC}^k$ und $u \in \text{PREC}^{k+2}$. Setze $g = \llbracket t \rrbracket_k$ und $h = \llbracket u \rrbracket_{k+2}$. Nach Induktionsvoraussetzung gilt $g, h \in \mathcal{A}$.

$$\begin{aligned} \curvearrowright \exists r, s : g(\bar{x}) &< A(r, \max \bar{x}) \\ h(\bar{y}) &< A(s, \max \bar{y}) \end{aligned}$$

Sei $f = \text{PR}(g, h) = \llbracket \text{PR}(t, u) \rrbracket_{k+1}$. Zeige $f \in \mathcal{A}$.

Zunächst $\forall n \in \mathbb{N} : f(n, \bar{x}) < A(q, n + \max \bar{x})$, wobei q nicht von n oder $\bar{x}$ abhängt. Wähle $q = \max(r, s) + 1$.

Induktion über n :

$$n = 0 : \quad f(0, \bar{x}) = g(\bar{x}) < A(r, \max \bar{x}) < A(q, \max \bar{x})$$

$$n \rightarrow n + 1 : \quad f(n + 1, \bar{x}) = h(f(n, \bar{x}), n, \bar{x}) < A(s, z) \quad , \quad z = \max(f(n, \bar{x}), n, \bar{x})$$

$$\text{nach I.V.: } f(n, \bar{x}) < A(q, n + \max \bar{x})$$

$$\max(n, \bar{x}) \leq n + \max \bar{x} < A(q, n + \max \bar{x})$$

$$\curvearrowright z < A(q, n + \max \bar{x})$$

$$\begin{aligned} f(n + 1, \bar{x}) &< A(s, z) \\ &< A(s, A(q, n + \max \bar{x})) \\ &\leq A(q - 1, A(q, n + \max \bar{x})) \\ &= A(q, n + 1 + \max \bar{x}) \end{aligned}$$

□ Ind. n

Nun setze $w = \max(n, \bar{x})$

$$\begin{aligned} f(n, \bar{x}) &< A(q, n + \max \bar{x}) \\ &\leq A(q, 2w) \\ &\stackrel{!}{<} A(\underline{q+2}, w) \end{aligned}$$

Also $f \in \mathcal{A}$

□

Der Majorisierungssatz liefert unmittelbar die gewünschte echte Trennung.

Korollar 5.11: Die Ackermannfunktion A ist nicht primitiv rekursiv.

◇

BEWEIS: $A \notin \mathcal{A}$

□

Liste der Definitionen

1.1	Def. (Alphabet Σ)	3
1.2	Def. (Wort w über Σ)	3
1.3	Def. (Konkatenation von Wörtern)	4
1.4	Def.	5
1.5	Def. (Sprache über Σ)	5
1.6	Def. (Konkatenation und Potenzierung von Sprachen)	6
1.7	Def. (Kleene-Abschluss, Kleene-Stern, Kleene-Plus)	6
1.8	Def. (Alternative Definition für Σ^*)	6
2.1	Def. (DEA)	11
2.2	Def. (Induktive Erweiterung von δ auf Wörter)	11
2.3	Def. (Die durch einen DEA akzeptierte Sprache)	12
2.4	Def. (Produktautomat für Schnitt)	12
2.5	Def.	14
2.6	Def. (Äquivalenz von DEA-Zuständen)	15
2.7	Def. (Äquivalenzklassenautomat)	16
2.8	Def. (Rechtskongruente Äquivalenzrelation)	17
2.9	Def.	18
2.10	Def.	18
2.11	Def. (NEA)	22
2.12	Def. (Lauf eines Automaten)	23
2.13	Def. (NEA zu DEA)	23
2.14	Def. (Potenzmengenautomat)	24
2.15	Def. (ε -NEA)	29
2.16	Def.	29
2.17	Def.	30
2.18	Def.	30
2.19	Def. (ε -freier Automat)	31
2.20	Def.	32
2.21	Def. (Abgeschlossenheit von $\mathcal{L}$)	33
2.22	Def. ($\text{RE}(\Sigma)$)	34
2.23	Def. (Semantik eines regulären Ausdrucks)	35
2.24	Def.	36
3.1	Def.	41
3.2	Def. (Ableitungsrelation, Ableitung, Sprache einer Grammatik)	41
3.3	Def. (Chomsky-Hierarchie)	43
3.4	Def. (Ableitungsbaum)	45
3.5	Def. (Eindeutigkeit von CFG und CFL)	46
3.6	Def.	46

3.7	Def.	47
3.8	Def.	50
3.9	Def. (CFG in CNF)	53
3.10	Def.	57
4.1	Def. (NPDA)	60
4.2	Def. (Menge der Konfigurationen eines NPDA)	61
5.1	Def.	71
5.2	Def. (DPDA)	71
5.3	Def.	72
6.1	Def. (Turingmaschine (TM))	76
6.2	Def. (Konfiguration)	77
6.3	Def. (Rechenschrittrelation)	78
6.4	Def. (Akzeptanz)	78
6.5	Def. (Halten)	78
6.6	Def. (berechnete Funktion)	79
6.7	Def. (Flussdiagramm)	80
6.8	Def. (Von Flussdiagramm definierte TM)	81
6.9	Def. (normierte TM)	87
7.1	Def. (berechenbare Funktion)	91
7.2	Def. (Abzählbar, Überabzählbar)	92
7.3	Def. (Durch Binärstring codierte TM)	93
7.4	Def. (Entscheidbarkeit)	97
7.5	Def. (Charakteristische Funktion)	98
7.6	Def. (Spezielles Halteproblem)	99
7.7	Def. (Semi-Entscheidbarkeit)	100
7.8	Def. (Reduktion)	101
7.9	Def. (Halteproblem)	102
7.10	Def. (Halteproblem auf leerem Band H_ϵ)	102
7.11	Def. (Universelles Halteproblem H_{vw})	104
7.12	Def. (Berechnung)	105
8.1	Def.	108
8.2	Def. (NTIME Klasse)	108
8.3	Def.	109
8.4	Def.	110
8.5	Def.	110
8.6	Def. (Polynom)	112
8.7	Def.	112
8.8	Def. (Polynomielle Reduktion)	113
8.9	Def. (3SAT)	113
8.10	Def. (Klasse der nichtdeterministisch polynomiell berechenbaren Sprachen (NP)-schwer und NP -vollständig)	116

8.11	Def.	117
8.12	Def. (CLIQUE)	118

Liste der Sätze

2.1	Satz	13
2.2	Lemma ($\equiv$ ist Äquivalenzrelation)	15
2.3	Lemma	15
2.4	Satz (Äquivalenzklassenautomat ist wohldefiniert)	16
2.5	Satz (Myhill und Nerode)	20
2.5	Korollar	21
2.6	Satz (Rabin und Scott)	24
2.7	Lemma (Pumping Lemma)	26
2.8	Satz	30
2.9	Lemma	32
2.10	Lemma	33
2.11	Lemma	33
2.12	Satz (Kleene)	36
2.13	Lemma (Ardens Lemma)	38
2.14	Korollar	38
3.1	Lemma	45
3.2	Lemma (SEP)	46
3.3	Lemma (BIN)	47
3.4	Satz	48
3.5	Korollar	49
3.6	Lemma (DEL)	49
3.7	Lemma (UNIT)	50
3.8	Satz	53
3.9	Lemma	53
3.10	Satz (Wortproblem für CFL entscheidbar)	54
3.11	Korollar	56
3.12	Satz (Entscheidbarkeit des Leerheitsproblems für kontextfreie Sprachen)	57
3.13	Satz (Typ-3-Sprache ist regulär)	57
3.14	Lemma	58
4.1	Lemma	62
4.2	Lemma (Mehr Keller – mehr Möglichkeiten.)	62
4.3	Lemma	64
4.4	Lemma	64
4.5	Satz	67
5.1	Satz (Pumping Lemma für CFL)	68

5.2	Lemma	68
5.3	Lemma	69
5.4	Lemma	70
5.5	Satz	70
5.6	Satz	70
5.7	Satz	71
5.8	Lemma	71
5.9	Lemma (DPDA, der gesamte Eingabe verarbeitet)	71
5.10	Satz (Abgeschlossenheit der deterministischen CFL)	73
5.11	Satz	73
6.1	Lemma (normierte TM)	87
6.2	Satz (Universelle TM)	90
7.1	Lemma (Standardnummerierung stdnum)	93
7.2	Lemma (bin2tm)	94
7.3	Lemma (func2power)	94
7.4	Satz (Cantor)	94
7.5	Satz (nicht berechenbare Funktionen)	94
7.6	Satz (nicht entscheidbare Sprache)	97
7.7	Satz (Berechenbarkeit und Entscheidbarkeit)	98
7.8	Satz (spezielles Halteproblem ist unentscheidbar)	99
7.9	Korollar (Komplement des speziellen Halteproblems ist unentscheidbar) .	100
7.10	Lemma (spezielles Halteproblems ist semi-entscheidbar)	101
7.11	Satz ($L, \bar{L}$ semi-entscheidbar $\Rightarrow L$ entscheidbar)	101
7.12	Korollar (Komplement des speziellen Halteproblems ist semi-entscheidbar)	101
7.13	Lemma (Reduktion)	102
7.14	Satz (H ist unentscheidbar)	102
7.15	Satz (H ist semi-entscheidbar)	102
7.16	Satz (H_ε ist unentscheidbar)	102
7.17	Satz	103
7.18	Satz (Satz von Rice)	103
7.19	Satz (Universelles Halteproblem $H_{\forall w}$)	104
7.20	Satz	104
7.21	Satz	105
7.22	Satz	106
8.1	Lemma	114
8.2	Lemma	116
8.3	Satz	116
8.4	Lemma ($\preceq_p$ ist reflexiv und transitiv)	117
8.5	Satz (Cook, Levin)	117
8.6	Satz (3SAT ist NP -vollständig)	117
8.7	Satz (CLIQUE ist NP -vollständig)	118

8.8	Lemma	120
9.1	Lemma	123
9.2	Satz	123
9.3	Satz	123
9.4	Korollar	125
9.5	Satz	126
9.6	Satz	126
9.7	Satz	127
9.8	Satz	127
9.9	Satz	128
9.10	Satz (Abgeschlossenheit von Typ-0 Sprachen)	128

Abbildungsverzeichnis

1	Endliches Band	9
2	Schematische Darstellung eines DEA, mit $\tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v) = q_{42}$. . .	17
3	Nichtdeterministischer Automat für L_n	22
4	DEA „modulo 3“	37
5	Algorithmus UNIT	51
6	CYK Algorithmus	55
7	Zustandsdiagramm eines Kellerautomaten	59
8	Zustandsdiagramm der Kellerautomaten aus Abb. 9. Statt alle Transitionen explizit zu zeichnen verwenden wir die Variablen a und Z um eine Menge von Transitionen durch eine einzige Kante zu repräsentieren. . . .	61
9	Illustration des Argumentes aus dem Beweis von Satz 3.13	69
10	Turingband	74
11	Informelle graphische Darstellung einer Turingmaschinenkonfiguration . .	77
12	Durch Flussdiagramm beschriebene TM	82
13	Mehrspurmaschine	83
14	Simulation einer Mehrbandmaschine durch eine Mehrspurmaschine	84
15	Startkonfiguration einer Mehrbandmaschine codiert auf Mehrspurmaschine	85
16	Standardnummerierung für einige Beispielwerte.	93
17	Abbildung func2power für einige Beispielwerte	94
18	Konkatenierte Abbildung aus dem Widerspruchsbeweis zu ??	95
19	Illustration des Diagonalarguments beim Beweis des Satzes von Cantor . .	96
20	Illustration des Diagonalarguments beim Rätsel des Barbier von Waldkirch	97
21	Konkatenierte Abbildung aus dem Widerspruchsbeweis zu ??	98
22	Definition der TM $\mathcal{M}_{\text{evil}}$ aus dem Beweis von ??	99
23	Illustration des Diagonalarguments beim Beweis der Unentscheidbarkeit des speziellen Halteproblems	100

24	Illustration einer Reduktion $U \preceq V$ von einer Sprache U auf eine Sprache V	101
25	Semi-Entscheidbarkeit der in diesem Kapitel vorgestellten unentscheidbaren Sprachen	105
26	Inklusionsrelationen zwischen Komplexitätsklassen	109
27	Beispiel zu <i>Erfüllbarkeitsproblem der Aussagenlogik in 3CNF</i> ($3SAT$) $\preceq_p$ CLIQUE	120
28	Illustration des Diagonalarguments beim Beweis, dass nicht jede entscheidbare Sprache CH1 ist	126
29	Abschlusseigenschaften der Sprachklassen aus der Chomsky-Hierarchie sowie der Klasse der Entscheidbaren Sprachen	128

Abkürzungsverzeichnis

3SAT	Erfüllbarkeitsproblem der Aussagenlogik in 3CNF
AL	Aussagenlogik
CFL	Menge der kontextfreien Sprachen
CFG	kontextfreie Grammatik
CNF	Chomsky-Normalform
CP	Korrespondenzproblem
CYK	Cocke, Younger, Kasami
DAG	gerichteter azyklischer Graph
DCFG	deterministische CFG
DCFL	deterministische CFL
DEA	deterministischer endlicher Automat
DFA	engl.: deterministic finite automaton
DPDA	deterministischer Kellerautomat
DTM	deterministische TM
EA	endlicher Automat
LBA	Linear Bounded Automaton
MPCP	Das modifizierte PCP
μ REC	Menge der μ -rekursiven Terme
ND	Nicht-Determinismus

NEA	nichtdeterministischer endlicher Automat
NFA	engl.: nondeterministic finite automaton
NP	Klasse der nichtdeterministisch polynomiell berechenbaren Sprachen
NPDA	nichtdeterministischer Kellerautomat
NT	Nichtterminal
NTM	nichtdeterministische TM
P	Klasse der deterministisch polynomiell berechenbaren Sprachen
PCP	Das Postsche Korrespondenzproblem
PDA	pushdown automaton (Kellerautomat)
PL	Pumping Lemma
PREC	Menge der primitiv rekursiven Terme
RE	Menge der regulären Ausdrücke
REG	Menge der regulären Sprachen
RM	Registermaschine
SAT	Erfüllbarkeitsproblem der Aussagenlogik
TM	Turingmaschine
TT	Turingtabelle
URM	unbeschränkte Registermaschine
WHILE	Menge der WHILE-Programme

Anmerkungsverzeichnis

Vorlesung: 21.04.26	3
Vorlesung: 22.04.26	8
Vorlesung: 28.04.26	12
Vorlesung: 29.04.26	15
Vorlesung: 05.05.2026	19
Vorlesung: 12.05.2026	21
Vorlesung: 13.05.2026	25
Vorlesung: 19.05.2026	28
Vorlesung: 20.05.2026	32
Vorlesung: 02.06.2026	42

Vorlesung: skipped	44
Vorlesung: 03.06.2026	45
Vorlesung: 09.06.2026	49
Vorlesung: 10.06.2026	54
Vorlesung: 16.06.2026	58
Vorlesung: 17.06.2026	64
Vorlesung: 23.06.2026	66
Vorlesung: 24.06.2026	75
Vorlesung: 30.06.2026	78
Vorlesung: 01.07.2026	83